

# **“Smart Hostel and Mess Administration Management Information System”**

**A Project Report Submitted in the partial fulfillment of the requirements  
for the award of the degree**

**BACHELOR OF TECHNOLOGY**

**In**

**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**&**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION  
TECHNOLOGY**

**By**

**M Sai Bhavitha Reddy – 2520030551  
Ramagiri Rishik – 2520030333  
G. Hitesh – 2520030370**

**Under the Esteemed Guidance of**

**Dr Yerragudipadu Subbarayudu  
Assistant Professor**

**Department of Computer Science and Engineering**



**Koneru Lakshmaiah Education Foundation**

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, [www.klh.edu.in](http://www.klh.edu.in)

**K L (Deemed to be) University**  
**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**  
**&**  
**DEPARTMENT OF COMPUTER SCIENCE AND**  
**INFORMATIONTECHNOL**



**DECLARATION**

We hereby declare that the project report entitled “**Smart Hostel and Mess Administration Management Information System**” is a record of the work carried out by **M Sai Bhavitha Reddy – 2520030551**, **Ramagiri Rishik – 2520030333**, and **G. Hitesh – 2520030370**. The project has been prepared as an academic project with the objective of studying and developing a centralized digital approach to hostel and mess administration in educational institutions.

The work presented in this report is based on the project concept, modules, objectives, software requirements, and future scope described in the project review presentation. The report documents the proposed system in a structured manner, including its background, problem statement, scope, requirements, technology considerations, architecture, database design, module-wise implementation approach, features, testing approach, deployment considerations, limitations, future enhancements, and conclusion.

We further declare that the documentation has been prepared for academic purposes and that the names, roll numbers, project title, and project direction have been retained as provided for the project. Any implementation screenshots, source-code listings, test evidence, or output demonstrations are intended to be inserted by the project team in the designated editable sections of this report.

Place: Hyderabad,

Telangana Date: \_\_\_\_\_

**M Sai Bhavitha Reddy – 2520030551**  
**Ramagiri Rishik – 2520030333**  
**G. Hitesh – 2520030370**

# K L (Deemed to be) University

## DEPARTMENT OF COMPUTER SCIENCE ENGINEERING & DEPARTMENT OF COMPUTER SCIENCE AND INFORMATIONTECHNOLOGY



### CERTIFICATE

This is to certify that the project report entitled “**Smart Hostel and Mess Administration Management Information System**” is a bonafide academic work carried out by **M Sai Bhavitha Reddy – 2520030551**, **Ramagiri Rishik – 2520030333**, and **G. Hitesh – 2520030370**, under the guidance of Dr. Y Subbarayudu, Assistant Professor, Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation (Deemed to be University).

The report presents the proposed design and development of a centralized digital platform for managing student information, hostel accommodation, room allocation, attendance, hostel fees, mess activities, complaints, notifications, and administrative reports. The work has been documented according to the project review material supplied by the student team and is submitted as part of the academic requirements of the second year, first semester project work.

The project demonstrates the application of software engineering concepts, database management, web technologies, backend processing, user interface design, testing, and system documentation to a practical institutional-management problem.

Guide: Dr. Y Subbarayudu

Designation: Assistant

Professor

Department: Computer Science and Engineering

Guide Signature: \_\_\_\_\_

Course Coordinator Signature: \_\_\_\_\_

HOD Signature: \_\_\_\_\_

## ACKNOWLEDGEMENT

The completion of this project report has been possible through the guidance, encouragement, and support received during the study and development of the Smart Hostel and Mess Administration Management Information System. We express our sincere gratitude to our guide, **Dr. Y Subbarayudu**, Assistant Professor, Department of Computer Science and Engineering, for his guidance and academic direction throughout the project. His suggestions helped us understand how a practical institutional-management problem can be translated into a structured software solution.

We also thank the **Department of Computer Science and Engineering** and Koneru Lakshmaiah Education Foundation (Deemed to be University) for providing an academic environment in which we could study software development, databases, backend processing, frontend technologies, testing, and documentation. The project has helped us connect classroom concepts with a realistic application involving student records, accommodation, attendance, fees, mess administration, complaints, communication, and reports.

We are thankful to our faculty members, classmates, and peers for their suggestions and discussions during the preparation of the project review and documentation. Their feedback encouraged us to examine the proposed system from the perspectives of students, administrators, hostel staff, and mess staff.

Finally, we express our gratitude to our parents and family members for their encouragement and support. We acknowledge everyone who contributed directly or indirectly to the successful preparation of this project report.

M Sai Bhavitha Reddy – 2520030551

Ramagiri Rishik – 2520030333

G. Hitesh – 2520030370

## **Abstract**

The Smart Hostel and Mess Administration Management Information System is a centralized digital platform designed to simplify and automate the management of hostel and mess activities in educational institutions. Traditional hostel administration often involves manual record keeping for student accommodation, room allocation, attendance, fee payments, complaints, and mess operations, which can be time-consuming and prone to errors. The proposed system provides an efficient and user-friendly solution for managing these activities through a single integrated platform.

The system enables administrators to maintain student details, allocate rooms, manage hostel occupancy, track hostel fees, and handle student complaints. It also supports mess administration by maintaining meal schedules, student meal registrations, menu details, and mess-related records. Students can access their hostel and mess information, submit complaints or requests, and view relevant updates through the system. This improves communication between students, hostel staff, and administrators while reducing paperwork and administrative workload.

**Keywords:** Smart Hostel Management, Mess Management, Student Records, Room Allocation, Attendance, Fee Management, Complaint Management, Centralized Database, Administration Dashboard.

### Index

| S. No. | Chapters / Topics                   | Page No. |
|--------|-------------------------------------|----------|
|        | Acknowledgement                     | 4        |
|        | Abstract                            | 5        |
| 1      | Introduction                        | 7-14     |
| 1.1    | Background of the Project           | 7        |
| 1.2    | Problem Statement                   | 8        |
| 1.3    | Scope of the Project                | 9        |
| 1.4    | Objectives of the Project           | 10       |
| 1.5    | Importance of the Application       | 11       |
| 1.6    | Target Users / Audience             | 12       |
| 1.7    | Project Contribution and Team Roles | 13-14    |
| 2      | System Requirements                 | 15-19    |
| 2.1    | Hardware Requirements               | 15       |
| 2.2    | Software Requirements               | 16       |
| 2.3    | Development Tools and Frameworks    | 17       |
| 2.4    | Functional Requirements             | 18       |
| 2.5    | Non-Functional Requirements         | 19       |
| 3      | Technology Stack                    | 20-27    |
| 4      | System Architecture                 | 28-33    |
| 5      | Design                              | 34-39    |
| 6      | Implementation                      | 40-48    |
| 7      | Features                            | 49-55    |
| 8      | Testing                             | 56-61    |
| 9      | Deployment                          | 62-65    |
| 10     | Challenges & Limitations            | 66-67    |
| 11     | Future Enhancements                 | 68       |
| 12     | Conclusion                          | 69       |
| 13     | References                          | 70       |

## CHAPTER -1 INTRODUCTION

### 1.1 Background of the Project

This section documents the need for a centralized hostel and mess administration system for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Student information is often distributed across admission records, hostel registers, attendance sheets, fee records, and mess-related records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Room allocation requires a current view of available and occupied rooms so that administrators can assign accommodation efficiently. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Hostel fee tracking becomes difficult when payment history and pending amounts are maintained manually. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Mess administration involves menus, meal registration, attendance, food requirements, and related records that need coordinated handling. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Complaint handling requires a mechanism through which students can report issues and administrators can track resolution. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. A centralized database can reduce duplicate entry and make authorized information easier to access. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. The proposed platform brings hostel and mess activities into a single integrated environment. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. The project applies web technologies, backend processing, and MySQL-based data management to an institutional problem. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -1 INTRODUCTION

### 1.1 Need for Digital Hostel Administration

This section documents why manual hostel operations become difficult as student numbers and activities increase for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Increasing student populations create more records and more frequent updates to accommodation information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Manual registers require repeated writing, checking, correction, and reconciliation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Administrators need timely information about occupancy, attendance, fees, complaints, and notices. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Students benefit when their own hostel and mess information is available without repeated office visits. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Digital records can support consistent storage and retrieval of information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Centralization improves coordination between students, hostel staff, mess staff, and administrators. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Dashboards and reports can convert stored records into information useful for monitoring. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. The proposed system is intended to reduce paperwork and administrative workload. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -1 INTRODUCTION

### 1.2 Problem Statement

This section documents the major operational problems identified in the project review for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Manual maintenance of student, room, fee, attendance, and mess records consumes time and effort. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. Data-entry mistakes can affect student details, fee records, room allocations, and attendance. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. Duplicate or lost records can make reliable decision-making difficult. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Fee status and payment history are harder to monitor when records are not centralized. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Mess information such as meal attendance, menus, and food requirements can be difficult to coordinate. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Complaints handled through personal visits may not have a clear status trail. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Administrators may not have real-time information about occupancy or other operations. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. The proposed solution addresses these issues through a single integrated platform. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -1 INTRODUCTION

### 1.2 Detailed Problem Analysis

This section documents how the individual problems affect students and administrators for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Room allocation errors can result in incorrect occupancy information and delays in assigning rooms. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Attendance records may require repeated consolidation before administrators can review them. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Fee records may not immediately show which students have pending payments. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Mess planning becomes less efficient when meal participation is not captured systematically. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Complaint information may be difficult to prioritize when it is stored in informal records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Notice communication may depend on physical boards or separate communication channels. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Independent records increase the effort required to produce reports. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. A centralized system can provide a consistent source of information for authorized users. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -1 INTRODUCTION

### 1.3 Scope of the Project

This section documents the functional boundaries of the proposed Smart Hostel and Mess system for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Student registration and maintenance of student details are within the project scope. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Room allocation, room availability, and hostel occupancy management are core functions. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Hostel fee records, pending payments, and payment history are included. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Attendance management is included for hostel-related monitoring. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Mess administration covers menus, meal registration, meal attendance, and related records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Complaint management supports digital submission and administrative tracking. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Notifications, notices, dashboards, and reports provide communication and monitoring support. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. The scope can later be extended to mobile access, online payment, biometrics, AI, IoT, and cloud deployment. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -1 INTRODUCTION

### 1.3 Scope Boundaries and Assumptions

This section documents what the initial system is expected to cover and what remains for future work for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The initial system is focused on institutional hostel and mess operations rather than general-purpose campus management. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Authorized users are assumed to provide valid student and administrative information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. The central database is treated as the main source for hostel, student, fee, attendance, and mess records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Internet or local network access is assumed for web-based operation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. The system does not automatically provide physical services such as room maintenance or food preparation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Advanced integrations such as biometric devices and online payment are identified as future enhancements. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. AI-based food demand prediction is considered future scope rather than a mandatory current component. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. The design is modular so that future functions can be added without redesigning the whole concept. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -1 INTRODUCTION

### 1.4 Objectives of the Project

This section documents the objectives stated in the review presentation for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Automate hostel management activities such as student registration and room allocation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Maintain student information in a centralized database for easier and more secure access. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Manage hostel fee records including payments, pending fees, and payment history. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Reduce paperwork and human errors by replacing traditional registers with computerized records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Provide timely information about room occupancy, student records, fees, attendance, and mess activities. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Generate reports and dashboards that help administrators monitor operations. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Improve communication between students, hostel staff, mess staff, and administrators. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. Create a foundation for future mobile, payment, biometric, AI, IoT, notification, and cloud features. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -1 INTRODUCTION

### 1.4 Objective-wise Interpretation

This section documents how each objective translates into a system responsibility for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Automation means that repetitive record creation, retrieval, and status updates are handled digitally. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Centralized information means that authorized modules can work with common student and operational records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Fee management requires consistent storage of payment status and history. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Error reduction depends on structured forms, validation, and controlled database operations. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Real-time or current information depends on timely updates to the underlying records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Reports convert operational records into summaries useful to administrators. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Communication improves when notices and complaint updates can be accessed through the platform. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. Future readiness is supported by modular separation of student, hostel, mess, complaint, and reporting functions. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -1 INTRODUCTION

### 1.5 Importance of the Application

This section documents the academic and operational importance of a centralized hostel and mess platform for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Hostel management is a recurring institutional activity involving many students and administrative records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Centralized records reduce the effort required to locate information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Digital room information supports better occupancy management. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Fee tracking improves visibility into payment status and history. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Mess records support organized handling of menus and meal participation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Online complaints improve the path from issue reporting to administrative action. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Dashboards and reports can help administrators review the state of operations. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. The application demonstrates how software engineering can address a practical campus-management problem. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -1 INTRODUCTION

### 1.6 Target Users / Audience

This section documents the user groups identified by the project concept for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Students are primary users who need access to hostel and mess information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Hostel administrators manage accommodation, student records, fees, attendance, complaints, and notices. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Mess staff or responsible administrators manage menu and meal-related records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Administrative users require dashboards and reports for monitoring. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Students may submit complaints or requests through the platform. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Authorized staff require controlled access to operational data. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. The interface should be understandable to users with different levels of technical familiarity. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. The system is intended to improve communication between the different stakeholder groups. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -1 INTRODUCTION

### 1.7 Project Contribution and Team Roles

This section documents the team structure and responsibility areas shown in the review presentation for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. M Sai Bhavitha Reddy is identified as Project Lead / Backend & Database Engineer. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Ramagiri Rishik is identified as AI/Feature Development & Quality Assurance Engineer. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. G. Hitesh is identified as Frontend Developer / UI-UX Designer. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. The Student & Hostel Management module is assigned to roll number 2520030551 in the submission form. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Administration & Complaints is assigned to roll number 2520030333 in the submission form. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Mess Management is assigned to roll number 2520030370 in the submission form. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. The three responsibility areas can be coordinated through common database and interface requirements. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. Team integration is necessary because hostel, mess, complaint, and reporting functions depend on shared information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -1 INTRODUCTION

### 1.7 Chapter Summary

This section documents a consolidated view of the introduction chapter for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The project addresses manual hostel and mess administration challenges. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. It proposes a centralized digital platform rather than isolated registers. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Its scope covers student, room, fee, attendance, mess, complaint, admin, notification, and reporting functions. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Its objectives focus on automation, centralized information, reduced paperwork, and better communication. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. The system is relevant to students and institutional administrators. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. The design is intended to support current requirements while allowing future extensions. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. The team has defined distinct responsibility areas while sharing a common project goal. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. The next chapter translates the project concept into hardware, software, functional, and non-functional requirements. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -2 SYSTEM REQUIREMENTS

### 2.1 Hardware Requirements

This section documents development and user hardware considerations for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. A modern development computer is sufficient because the proposed system is a web-based application. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. A multi-core processor helps when running an editor, browser, database service, and local server together. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. At least 8 GB RAM is practical for development and testing, with additional memory useful for multitasking. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. SSD storage improves loading of project files, dependencies, and database tools. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. A stable network connection supports software downloads, repository synchronization, and testing when required. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. End users can access the system using desktops, laptops, tablets, or smartphones with a modern browser. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. The client device does not require specialized hardware for the basic web application. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. Future biometric or IoT extensions would introduce additional hardware requirements. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -2 SYSTEM REQUIREMENTS

### 2.1 Server and Network Considerations

This section documents resources required to run the application environment for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The server environment must be capable of receiving web requests and communicating with the MySQL database. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. Local development can use Apache and XAMPP when PHP is selected as the server-side technology. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. The application requires reliable communication between the user interface, backend services, and database. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Network performance affects the speed at which users receive updated information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Backups should be considered for important student and operational records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Production deployment can be scaled later if the number of users increases. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Access control should be applied so that only authorized users can perform administrative operations. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. Hardware planning should reflect the project size rather than assume enterprise-scale infrastructure for the academic prototype. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -2 SYSTEM REQUIREMENTS

### 2.2 Software Requirements

This section documents the software environment stated in the review presentation for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The review presentation specifies Ubuntu as the operating system. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. HTML5, CSS3, and JavaScript are identified as frontend technologies. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. Java, Python, and PHP are identified as possible backend technologies as required. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. MySQL is the specified database management system. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Visual Studio Code is identified as the development environment. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Apache and XAMPP are specified for web-server support when PHP is used. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Google Chrome, Mozilla Firefox, and Microsoft Edge are suitable browsers for testing. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. MySQL Workbench and phpMyAdmin are identified as database management tools, while Git and GitHub support version control. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -2 SYSTEM REQUIREMENTS

### 2.3 Development Tools and Frameworks

This section documents how the listed software tools support the project lifecycle for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Visual Studio Code provides the main environment for editing frontend and backend source files. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. MySQL stores structured records required by student, hostel, fee, attendance, mess, complaint, and administrative modules. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Git records changes and supports collaboration among the three team members. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. GitHub can provide a shared repository for source-code versioning and review. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Apache provides a local web-server environment when the selected backend requires it. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. XAMPP bundles services that are useful for local PHP and MySQL development. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Browsers provide a way to verify the rendered interface and basic client-side behavior. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. MySQL Workbench or phpMyAdmin assists with database creation, queries, inspection, and maintenance. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -2 SYSTEM REQUIREMENTS

### 2.4 Functional Requirements

This section documents the operations the system is expected to perform for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The system shall support student registration and maintenance of student information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. The system shall support room allocation and room availability information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. The system shall maintain hostel fee payment, pending-fee, and payment-history records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. The system shall support attendance management for relevant hostel activities. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. The system shall support mess registration, meal schedules, menu information, and meal-related records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. The system shall allow students to submit complaints or requests. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. The system shall allow administrators to manage notices, updates, and reports. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. The system shall provide role-appropriate access so users can work with functions relevant to them. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -2 SYSTEM REQUIREMENTS

### 2.5 Non-Functional Requirements

This section documents quality characteristics that support reliable use for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Usability requires clear navigation and understandable forms for students and administrators. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Reliability requires consistent storage and retrieval of records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Security requires appropriate authentication and authorization for protected operations. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Performance requires reasonable response times for common student and administrative operations. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Maintainability requires organized modules and readable source code. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Scalability requires a structure that can accommodate additional students and future modules. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Availability depends on the web-server and database environment being operational. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. Data integrity requires validation and controlled database operations to reduce inconsistent records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -3 TECHNOLOGY STACK

### 3.1 Frontend Technologies

This section documents HTML5, CSS3, and JavaScript as identified in the review for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. HTML5 provides the structure for pages, forms, tables, navigation elements, and content areas. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. CSS3 controls layout, spacing, typography, responsiveness, and visual consistency. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. JavaScript provides client-side interaction, validation, dynamic updates, and communication with backend endpoints when required. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. The frontend should present different navigation options according to the user role. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Forms should make student, room, fee, attendance, mess, and complaint data easy to enter. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Validation at the interface can reduce obvious input mistakes before submission. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Responsive design can support access from common desktop and mobile browser sizes. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. The frontend communicates with backend processing rather than directly modifying protected database records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -3 TECHNOLOGY STACK

### 3.1 Frontend Structure and UI Design

This section documents a practical organization for the proposed interface for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. A common navigation structure can provide links to dashboard, profile, hostel, mess, complaints, notices, and reports as appropriate. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Administrator screens can prioritize operational summaries and management functions. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Student screens can prioritize personal hostel and mess information and complaint submission. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Forms should use labels and meaningful validation messages. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Tables are useful for lists such as students, rooms, fees, menus, complaints, and attendance. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Consistent buttons and status indicators improve understanding of actions and records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. CSS media queries can adapt layouts for smaller screens. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. JavaScript can update selected interface elements without unnecessarily rebuilding the whole page. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -3 TECHNOLOGY STACK

### 3.2 Backend Technologies

This section documents the backend choices specified in the review presentation for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The review identifies Java, Python, and PHP as server-side technology options depending on project requirements. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. The backend is responsible for business rules, request handling, validation, and database communication. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Backend endpoints should receive structured requests and return appropriate responses to the frontend. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Business rules such as room availability and fee status should be enforced at the server side rather than relying only on the browser. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Authentication and authorization should be handled in the backend for protected functions. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Database operations should use controlled queries and appropriate validation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Error handling should return understandable results without exposing unnecessary internal information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. The selected backend can be changed later without changing the overall functional model if clear module boundaries are maintained. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -3 TECHNOLOGY STACK

### 3.2 Backend Request and Response Flow

This section documents how a typical operation moves through the application for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. A user begins an operation through a frontend page such as student registration or complaint submission. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. The frontend sends the relevant request to the backend service. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. The backend checks the request, validates the data, and identifies the required operation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. If the operation requires stored information, the backend communicates with MySQL. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. The database returns the requested data or confirms the update. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. The backend prepares a structured response for the frontend. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. The frontend displays a success message, updated table, or error message based on the response. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. This separation keeps database access behind the backend and provides a clear boundary for security and maintenance. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

### 3.3 MySQL Database

This section documents why the specified relational database fits the project for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. MySQL is the database specified in the project review presentation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Relational tables are suitable for structured student, room, fee, attendance, mess, complaint, and notice records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Primary keys can uniquely identify important records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Foreign keys can represent relationships between students, rooms, payments, attendance entries, and complaints. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Queries can retrieve lists for dashboards and administrative reports. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Constraints can help maintain valid relationships and reduce inconsistent data. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Indexes can improve lookup performance for frequently searched fields when required. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. phpMyAdmin and MySQL Workbench can be used to create and inspect the database during development. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -3 TECHNOLOGY STACK

### 3.3 Database Entities and Relationships

This section documents a conceptual schema for the proposed system for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. A Student entity represents the core student information used by multiple modules. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. A Room entity represents accommodation capacity, status, and allocation information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. A Hostel or allocation record can connect students with their assigned accommodation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. A Fee record can store the student reference, amount, status, and payment history details. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. An Attendance record can associate a student with a date and attendance state. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. A Mess Menu record can store menu details for relevant dates or meals. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. A Complaint record can store the student, category, description, status, and administrative response information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. Notice and user-role records support communication and controlled access. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -3 TECHNOLOGY STACK

### 3.4 Version Control and Collaboration

This section documents the role of Git and GitHub in a three-member project for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Git records source-code changes and provides a history of development. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. GitHub can act as a shared repository for the team. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Team members can work on assigned modules while keeping changes organized. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Commits should describe the actual change so that the history remains understandable. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Branches can separate experimental work from the stable project version. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Merging requires checking that changes from different modules do not conflict. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Code review can help identify defects before changes become part of the main version. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. Using version control also provides a recovery point when a later change introduces an issue. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -3 TECHNOLOGY STACK

### 3.5 Database Management Tools

This section documents the purpose of MySQL Workbench and phpMyAdmin for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. MySQL Workbench can help design tables, execute SQL statements, and inspect database structure. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. phpMyAdmin provides a browser-based interface for managing MySQL databases in common local-server environments. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Developers can create tables and test queries before integrating them into backend code. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Database tools make it easier to verify whether records are inserted or updated correctly. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Backups can be created as part of responsible database maintenance. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. The team should avoid editing production-like data casually because changes may affect test results. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Schema documentation should describe table purpose and important relationships. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. These tools support the database-focused responsibility of the project lead and the broader team. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -3 TECHNOLOGY STACK

### 3.5 Technology Stack Summary

This section documents how the selected tools work together for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. HTML5 defines the structure of the user interface. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. CSS3 provides presentation and responsive layout. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. JavaScript provides browser-side interaction. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Java, Python, or PHP can provide server-side processing as selected for the implementation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. MySQL provides persistent relational storage. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Visual Studio Code supports development. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Apache and XAMPP support local server environments when applicable. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. Git and GitHub support source-code collaboration, while browsers and database tools support testing and inspection. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -4 SYSTEM ARCHITECTURE

### 4.1 High-Level Architecture

This section documents the proposed layered organization of the system for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The presentation layer contains the student and administrator web interfaces. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. The application or backend layer processes requests and applies business rules. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. The database layer stores structured hostel and mess records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Authentication and authorization should protect administrative operations. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. The frontend should not directly connect to database credentials. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. The backend acts as the controlled gateway between users and stored information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Modular separation allows student, hostel, mess, complaint, and reporting functions to evolve independently. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. The architecture can be deployed locally during development and expanded later. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

**4.1 Logical Data Flow**

This section documents the movement of information for a typical hostel operation for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. A student or administrator enters information through a web form. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. The browser packages the information as a request to the backend. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. The backend validates required fields and checks the user permission. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. The backend executes the appropriate database operation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. MySQL stores or retrieves the relevant records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. The backend interprets the database result and prepares a response. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. The frontend displays the result to the user. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. Logging and error handling can be applied at the backend boundary to support maintenance. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

# 4.1 A System Flowchart

Smart Hostel and Mess Administration Management Information System

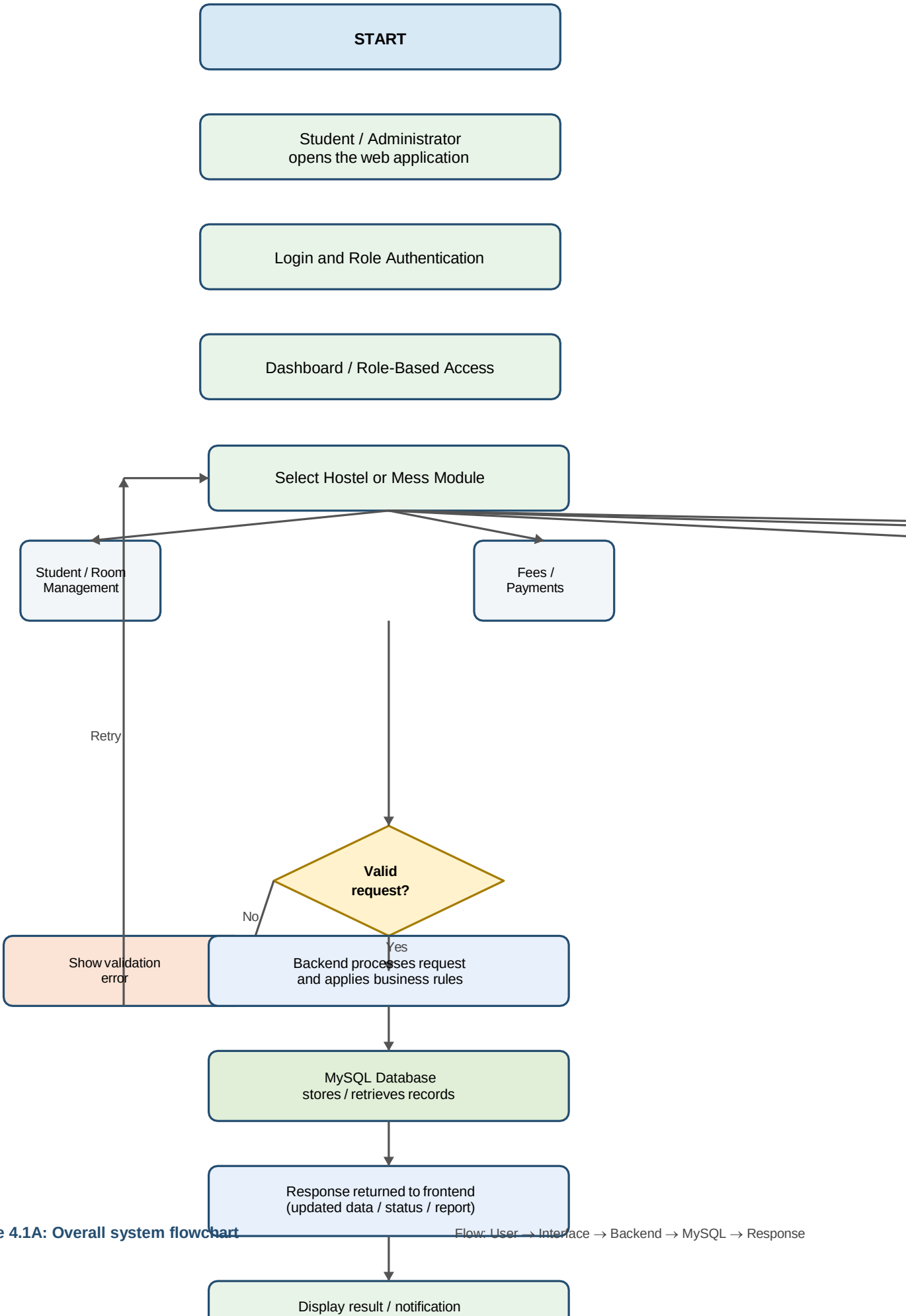


Figure 4.1A: Overall system flowchart

Flow: User → Interface → Backend → MySQL → Response

**4.2 Frontend Layer**

This section documents responsibilities of the presentation layer for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The frontend is responsible for presenting hostel and mess functions in an accessible form. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
  2. It displays dashboards, forms, tables, status information, and notices. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
  3. It can show only the navigation options relevant to the logged-in role. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
  4. Input controls should use clear labels and validation messages. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
  5. Room information can be displayed as available, occupied, or otherwise status-based records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
  6. Complaint pages can display submission and status information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
  7. Mess pages can present menus and meal-related records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
  8. The frontend should remain focused on presentation and user interaction while backend logic enforces rules. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
- Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

**4.2 Backend Layer**

This section documents responsibilities of the server-side layer for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The backend receives requests from the frontend and processes them according to the application rules. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. It validates student, room, fee, attendance, mess, and complaint data. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. It manages authentication and authorization for protected operations. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. It performs database queries and updates through the selected server-side technology. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. It prevents the frontend from directly handling database credentials. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. It can prepare responses for dashboards and reports. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. It should provide controlled error messages when operations fail. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. The backend is therefore the main point for enforcing consistent application behavior. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -4 SYSTEM ARCHITECTURE

### 4.2 Database Layer

This section documents responsibilities of MySQL storage for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The database stores persistent information used by the system. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. Student records provide identity and contact information needed by other modules. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. Room records support capacity and allocation status. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Fee records support payment status and history. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Attendance records support monitoring over time. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Mess records support menus, meal registration, and related administration. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Complaint records provide a trace of reported issues and their status. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. Notice, user, and role records support communication and access management. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

**4.3 Deployment Architecture**

This section documents local deployment and future scalability for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. During development, the application can run on a local machine using the listed development tools. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. If PHP is selected, XAMPP can provide Apache and MySQL services for local execution. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. The browser connects to the locally hosted web application. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. The backend communicates with MySQL within the local environment. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Git and GitHub can maintain the source-code repository separately from the runtime environment. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. For future use, the system can be deployed on a server or cloud environment. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Cloud-based deployment is explicitly identified as future scope in the review presentation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. Deployment planning should preserve database security, backups, access control, and reliable service availability. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -5 DESIGN

### 5.1 System Design Overview

This section documents design principles for the proposed modules for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The design is modular so that each major administrative responsibility has a clear function. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. Student Management acts as a common source of student information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. Room Management uses student and accommodation information to maintain occupancy. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Fee Management maintains payment-related records separately but connects them to students. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Mess Management handles menu and meal-related information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Complaint Management connects student reports with administrative handling. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Attendance Management records attendance information for later review. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. Admin, notification, and reporting functions provide coordination and monitoring. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

**5.2 ER Diagram / Database Schema – Conceptual Model**

This section documents entities that should appear in the database design for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The Student table is a central entity because room, fee, attendance, mess, and complaint records relate to students. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. The Room table represents hostel accommodation units and their current status. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. The Allocation table can connect a student with a room and record allocation-related information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. The Fee table can connect payments to the relevant student. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. The Attendance table records attendance entries against a student and date. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. The Menu table stores meal information that can be displayed to students. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. The Complaint table stores issue reports and their status. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. User and Role structures can separate authentication information from operational student records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -5 DESIGN

### 5.2 Database Relationships

This section documents how the conceptual tables interact for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. One student can have hostel-related records associated with accommodation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. A room can be associated with one or more students depending on the room capacity and allocation model. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. A student can have multiple fee records over time. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. A student can have multiple attendance entries. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Menu records can represent meals or menu schedules relevant to dates. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. A student can submit multiple complaints. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Administrators can update complaint status and notices. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. Foreign-key relationships should be used where appropriate to maintain referential consistency. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -5 DESIGN

### 5.3 User Interface Design

This section documents how the main screens should be organized for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The login screen provides controlled entry to the system. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. The student dashboard summarizes relevant hostel and mess information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. The administrator dashboard summarizes operational information and actions. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Student forms should avoid unnecessary fields and use clear labels. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Room pages should make availability and allocation status understandable. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Fee pages should distinguish paid, pending, and historical information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Complaint pages should show submission details and current status. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. Notice and notification areas should present important updates clearly. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

**5.4 Role-Based Access Design**

This section documents the principle of showing and protecting functions according to role for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Student users should access their own relevant information and permitted actions. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Administrators require broader access to manage records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Mess management functions should be available to the responsible administrative role. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Complaint handling requires students to submit and administrators to review or update. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Fee management should be protected because it contains administrative financial records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Authentication identifies the user while authorization decides what that user may do. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Backend authorization is necessary even when frontend menus hide restricted functions. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. Role separation reduces accidental modification of records by unauthorized users. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -5 DESIGN

### 5.5 Validation and Data Integrity Design

This section documents rules that help maintain reliable records for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Required fields should be checked before records are stored. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. Numeric values such as fees should be validated as appropriate numeric inputs. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. Dates should follow a consistent format. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Room allocation should not create an invalid occupancy state. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Complaint status should use controlled values rather than unrestricted text where possible. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Database constraints can reinforce application-level validation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Duplicate student records should be minimized through suitable identifiers. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. Validation should be performed at both the interface and backend layers. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -6 IMPLEMENTATION

### 6.1 Student Management Implementation

This section documents how student registration and records fit into the application for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The student module stores the information required to identify and manage hostel residents. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. An administrator can create or update student records through a controlled form. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. The backend validates the submitted information before database insertion or update. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. The database stores the record in the Student entity. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Student information can be reused by room, fee, attendance, mess, and complaint modules. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. List and search functions can help administrators locate records efficiently. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Access to student information should be limited to authorized users. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. The module forms the data foundation for several other project functions. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -6 IMPLEMENTATION

### 6.1 Room Management Implementation

This section documents how room allocation and availability can be handled for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Room records represent hostel accommodation and current occupancy information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. An administrator can view rooms and their availability before assigning accommodation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. When an allocation is created, the backend checks the room state and student details. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. The allocation is stored so that the relationship between student and room can be retrieved later. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. When a student leaves or changes rooms, the corresponding record can be updated. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. The room view should reflect the latest stored status. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Validation is important to avoid invalid or duplicate allocation states. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. The module supports the project objective of reducing manual room-allocation effort. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -6 IMPLEMENTATION

### 6.1 Hostel Fee Management Implementation

This section documents how fee records can be maintained for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The fee module stores hostel-related payment information against students. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Administrators can record payment entries and identify pending amounts. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Payment history allows earlier transactions to remain available for reference. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. The backend should validate fee amounts and student references before storing data. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. The student view can display relevant fee status without exposing unrelated administrative records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Administrative dashboards can summarize pending and completed payments. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Consistent status values make reports easier to generate. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. The module directly addresses the review presentation concern about poor manual fee management. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -6 IMPLEMENTATION

### 6.1 Attendance Management Implementation

This section documents how attendance records support hostel administration for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Attendance entries can associate a student with a date and attendance status. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. The system can provide a structured alternative to paper attendance registers. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Administrators can retrieve records for a selected period or student. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Validation can prevent missing student references or invalid dates. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. The stored records can support summaries for administrative review. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Attendance data should be visible only to authorized users where appropriate. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Future QR or biometric attendance is identified as an enhancement rather than a current requirement. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. The module contributes to the objective of providing current and organized attendance information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -6 IMPLEMENTATION

### 6.2 Mess Management Implementation

This section documents how menus, meal registration, and meal records fit together for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The mess module maintains menu information that can be displayed to students. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Meal schedules provide a structured representation of available meals. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Student meal registration can associate a student with a meal or schedule. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Meal attendance records can support monitoring of participation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Mess-related information is stored in the central database rather than separate informal records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Administrators can update menu information as required. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Future AI-based food demand prediction can use accumulated meal data as a foundation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. The module addresses the review presentation objective of more efficient mess administration. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -6 IMPLEMENTATION

### 6.2 Complaint Management Implementation

This section documents the digital complaint workflow for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. A student starts by entering a complaint or request through the complaint form. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. The backend validates the student identity and required complaint information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. The complaint is stored with a status that can be reviewed by administrators. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Administrators can inspect complaints and update their handling status. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. The student can view relevant updates or status information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Centralized records make it easier to identify unresolved issues. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Complaint categories can help administrators organize common issue types. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. This workflow replaces the need for every complaint to depend on a personal visit to the hostel office. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

**6.3 Admin Dashboard Implementation**

This section documents how the administrator interface brings modules together for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. The dashboard acts as a central entry point for administrative operations. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. It can provide summaries of students, rooms, fees, attendance, mess records, and complaints. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. Navigation should link administrators to the detailed management screens. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Role-based access should ensure that only authorized users reach administrative functions. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Current counts and status summaries can help administrators identify pending work. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Reports can be generated from the same centralized records used by individual modules. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. The dashboard reduces the need to search through separate registers. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. Its design should prioritize clarity over unnecessary visual complexity. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -6 IMPLEMENTATION

### 6.4 Authentication and Authorization Implementation

This section documents how controlled access should be handled for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Users should authenticate before accessing protected parts of the application. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Credentials should be processed by the backend rather than exposed to other users. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Authorization rules should identify the permitted operations for each role. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Student users should not be able to perform unrestricted administrative updates. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Administrative actions should be checked at the backend even if the interface hides restricted buttons. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Session or token handling depends on the selected backend technology. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Password storage should follow secure practices appropriate to the chosen backend implementation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. Authentication is an important part of protecting centralized student and administrative information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

**6.5 Integration of Modules**

This section documents how the separate functions operate as one system for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Student records are reused by room, fee, attendance, mess, and complaint functions. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. Room allocation depends on student and room information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. Fee records depend on identifying the correct student. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Attendance records require student references and dates. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Mess registration can use student records to identify participants. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Complaints connect a student to an issue and its administrative status. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Dashboard reports combine information from several modules. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. Integration is achieved through shared database relationships and backend services rather than independent duplicate records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -6 IMPLEMENTATION

### 6.6 Implementation Sequence

This section documents a practical order for developing the academic project for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Begin by defining the requirements and module boundaries from the project review. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. Design the database tables and relationships before implementing dependent operations. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. Create the backend connection and basic request-processing structure. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Implement authentication and role handling before exposing administrative functions. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Develop student and room management because other modules depend on student information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Add fees, attendance, mess, and complaint operations as separate modules. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Connect the dashboard and reports after the core records are available. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. Test each module individually and then test interactions between modules. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -7 FEATURES

### 7.1 Main Features Overview

This section documents the nine modules listed in the review presentation for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Student Management maintains student details. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Room Management maintains allocation and availability. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Hostel Fee Management tracks payment information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Mess Management handles menus, meals, and related records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Complaint Management supports digital complaint submission and tracking. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Attendance Management stores attendance information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Admin Management provides administrative control. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. Notification and Communication provides notices and updates. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

9. Reports and Dashboard provide summarized operational information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -7 FEATURES

### 7.2 Student Management Feature

This section documents student-facing and administrator-facing behavior for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Students are represented by structured records in the central database. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. Administrators can maintain student information through forms. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. Search and list views can make records easier to locate. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Other modules can reference the student record rather than duplicate identity information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Students can access information relevant to their own hostel and mess participation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Validation helps maintain complete and consistent records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Authorization controls who may create or modify records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. The feature supports the objective of centralized student information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -7 FEATURES

### 7.2 Room Management Feature

This section documents student and administrator use of room information for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Administrators can view room availability before allocation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. Room records can represent capacity and occupancy state. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. Allocation creates a connection between the student and accommodation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Changes to allocation can update the current state. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Students can view relevant accommodation information after assignment. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. The centralized record reduces reliance on manual vacancy lists. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Validation helps avoid inconsistent occupancy states. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. The feature supports faster and more transparent room management. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -7 FEATURES

### 7.2 Hostel Fee Management Feature

This section documents fee status, pending payments, and history for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Administrators can record hostel fee information for students. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. Payment status provides a simple way to distinguish completed and pending records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. Payment history keeps earlier entries available for reference. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Students can view relevant fee information through the system. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Dashboard summaries can help administrators identify pending payments. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Centralized storage reduces repeated manual calculations and checking. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Access control protects administrative fee information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. The feature directly addresses the review presentation problem of poor fee management. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -7 FEATURES

### 7.2 Mess Management Feature

This section documents menu and meal-related functions for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Menu information can be entered and updated by the responsible administrator. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. Students can view meal schedules and menu details. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. Meal registration can record intended participation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Meal attendance can provide a record of participation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Centralized mess records improve coordination of daily information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. The data can support later analysis of meal demand. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Future AI-based food demand prediction is proposed in the review presentation. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. The feature reduces dependence on separate manual mess records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

**7.2 Complaint Management Feature**

This section documents submission, tracking, and administrative resolution for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Students can submit hostel or mess complaints digitally. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. Each complaint can store a description and relevant status information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. Administrators can review submitted complaints from a centralized list. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Status updates provide a simple indication of progress. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Complaint history helps avoid losing previously reported issues. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Categorization can help identify repeated operational problems. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Students can view relevant updates without repeatedly visiting the office. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. The feature improves communication between students and administrators. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -7 FEATURES

### 7.2 Attendance, Notifications, and Reporting

This section documents supporting features that complete the administration workflow for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Attendance management keeps structured records instead of depending only on registers. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. Notification and communication functions can publish notices and important updates. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. Dashboards summarize operational information for administrators. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Reports can combine records from different modules. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Current room, fee, attendance, mess, and complaint information can be reviewed more efficiently. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Future real-time SMS, email, or mobile alerts are part of the proposed enhancement scope. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Future advanced analytics can improve management insight. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. These features connect daily operations with administrative monitoring. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -8 TESTING

### 8.1 Testing Strategy

This section documents a structured approach for validating the proposed system for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Testing should begin at the module level before full integration. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. Form validation should be checked using valid and invalid input values. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. Backend operations should be tested independently from the user interface when possible. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Database operations should be checked for correct insertion, update, retrieval, and relationships. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Role restrictions should be tested using different user types. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Room allocation should be tested for valid and invalid occupancy conditions. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Complaint status changes should be tested from submission through administrative handling. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. Testing evidence should be recorded with the expected and actual results. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -8 TESTING

### 8.1 Functional Test Cases – Student and Room Modules

This section documents representative tests for core hostel operations for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. Valid student registration should create a corresponding database record. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. Missing required student fields should produce a validation response. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. Updating an existing student should change the intended record only. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Viewing student records should return the expected information for an authorized user. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. Room availability should display the stored status. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Valid allocation should associate the selected student and room. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Invalid or duplicate allocation conditions should be rejected by the backend rules. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. After a successful change, the interface should reflect the updated record. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

### 8.1 Functional Test Cases – Fees and Attendance

This section documents representative tests for fee and attendance functions for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. A valid fee entry should be stored against the correct student. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
2. Invalid fee input should be rejected. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
3. Payment history should display previously stored entries. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
4. Pending fee status should be visible in the appropriate administrative view. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
5. A valid attendance entry should be stored with the correct student and date. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
6. Invalid date or missing student references should be rejected. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
7. Attendance retrieval should return the expected records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.
8. Unauthorized users should not gain access to restricted administrative fee information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component. The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

## CHAPTER -8 TESTING

### 8.2 Integration Testing – Mess and Complaints

This section documents tests that cross module boundaries for the Smart Hostel and Mess Administration Management Information System. The discussion is aligned with the project review presentation and describes the proposed behavior, structure, and academic implementation approach without assuming features that are identified only as future scope.

1. A student created in Student Management should be selectable by the mess module. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

2. A student should be able to submit a complaint using the correct identity reference. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

3. An administrator should be able to view the complaint in the administrative list. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

4. Changing complaint status should be reflected when the student checks the relevant information. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

5. Menu records created by an administrator should be visible in the intended student view. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

6. Meal registration should use a valid student and meal reference. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

7. Invalid cross-module references should not create broken records. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

8. Integration testing confirms that the modules operate as one application rather than isolated pages. In the proposed system, this point is treated as part of a controlled digital workflow rather than an isolated manual activity. The relevant user interacts with the interface, the backend applies the required validation or business rule, and the central MySQL database provides persistent storage or retrieval. This separation makes the operation easier to review, test, and maintain while keeping the responsibilities of the frontend, backend, and database distinct.

Overall, this section shows how the project requirement can be translated into a structured software component.

The emphasis is on centralized information, controlled access, consistent records, reduced paperwork, and improved communication. The implementation evidence, including actual source code, test output, and screenshots, should be inserted by the team in the editable placeholder pages provided later in the report.

**APPENDIX – CODE PLACEHOLDER 1**

This page is intentionally reserved for the project team to insert editable backend source code. Suggested material includes the database connection, authentication logic, student management operations, room allocation logic, or other server-side implementation completed by the team. Keep the code in a readable monospaced font when inserted, and add a short caption identifying the module and file name.

**INSERT BACKEND CODE HERE**

**[ EDITABLE PLACEHOLDER – INSERT YOUR PROJECT MATERIAL HERE ]**

```

1  services:
2    api-gateway:
3      image: node:20-alpine
4      working_dir: /app
5      volumes:
6        - ./app
7      command: ["node", "gateway/server.js"]
8      environment:
9        GATEWAY_PORT: 4000
10       JWT_SECRET: development-demo-secret
11       STUDENT_SERVICE_URL: http://student-service:4101
12       FEE_SERVICE_URL: http://fee-service:4102
13       COMPLAINT_SERVICE_URL: http://complaint-service:4103
14       ATTENDANCE_SERVICE_URL: http://attendance-service:4104
15       MESS_SERVICE_URL: http://mess-service:4105
16       NOTIFICATION_SERVICE_URL: http://notification-service:4106
17     ports:
18       - "4000:4000"
19     depends_on:
20       - student-service
21       - fee-service
22       - complaint-service
23       - attendance-service
24       - mess-service
25       - notification-service
26
27     student-service:
28       image: node:20-alpine
29       working_dir: /app
30       volumes:
31         - ./app
32       command: ["node", "services/student-service/server.js"]

```

Smart Hostel and Mess Administration Management Information System  
APPENDIX – CODE PLACEHOLDER 2

```
1 // Generates seed.json (and db.json) for every service with dates relative to today,
2 // so attendance, meals and dues always look current in a demo.
3 const path = require("path");
4 const { hashPassword, writeJson } = require("../shared/service-kit");
5
6 const servicesDir = path.join(__dirname, "..", "services");
7 const now = new Date();
8 const dayOffset = (n) => new Date(now.getTime() + n * 86400000).toISOString().slice(0, 10);
9 const stamp = (n, time = "09:00") => `${dayOffset(n)}T${time}:00.000Z`;
10 const weekday = (n) => ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"][n];
11 const month = dayOffset(0).slice(0, 7);
12
13 const students = [
14   ["Aarav Sharma", "B.Tech CSE", 2, "Rajesh Sharma"],
15   ["Ishita Verma", "B.Tech ECE", 1, "Sunil Verma"],
16   ["Rohan Reddy", "B.Tech CSE", 2, "Venkat Reddy"],
17   ["Sneha Iyer", "B.Tech IT", 3, "Lakshmi Iyer"],
18   ["Karthik Nair", "B.Tech ME", 2, "Suresh Nair"],
19   ["Priya Singh", "B.Tech CSE", 1, "Amit Singh"],
20   ["Mohammed Faiz", "B.Tech EEE", 3, "Imran Faiz"],
21   ["Ananya Das", "B.Tech CSE", 2, "Bikash Das"],
22   ["Vikas Patel", "B.Tech CE", 4, "Harish Patel"],
23   ["Meera Joshi", "B.Tech IT", 1, "Nitín Joshi"],
24   ["Sai Teja", "B.Tech AI&DS", 2, "Ramana Teja"],
25   ["Divya Menon", "B.Tech CSE", 1, "Gopal Menon"]
26 ].map(([name, course, year, guardianName], index) => {
27   const n = index + 1;
28   return {
29     id: `stu-${2600 + n}`,
30     rollNo: `25200301${String(n).padStart(2, "0")}`,
31     name,
32
33     phone: `98765432${String(10 + n).padStart(2, "0")}`,
34     email: `${name.split(" ")[0].toLowerCase()}@student.test`,
35     password: hashPassword("student123"),
36     role: "student",
37     course,
38     year,
39     guardianName,
40     guardianPhone: `98000000${String(10 + n).padStart(2, "0")}`,
41     status: n === 12 ? "pending" : "active",
42     joinedDate: dayOffset(-120 + n)
43   };
44 });
45
46 const staff = [
47   ["adm-0001", "Dr. Suresh Kumar", "admin@hostel.test", "admin123", "admin", "9000000001"],
48   ["msx-0001", "Lakshmi Devi", "mess@hostel.test", "mess123", "mess", "9000000002"]
49 ].map(([id, name, email, password, role, phone]) => ({
50   id, rollNo: `STAFF-${id.slice(-4)}`, name, phone, email, password: hashPassword(password), role,
51   course: role === "admin" ? "Hostel Administration" : "Mess Administration", year: 0,
52   guardianName: "", guardianPhone: "", status: "active", joinedDate: dayOffset(-400)
53 }));
54
55 const rooms = [
56   ["A-101", "A", 1, 2], ["A-102", "A", 1, 2], ["A-103", "A", 1, 3], ["A-104", "A", 1, 3],
57   ["A-201", "A", 2, 2], ["A-202", "A", 2, 2], ["B-101", "B", 1, 3], ["B-102", "B", 1, 3],
58   ["B-201", "B", 2, 2], ["B-202", "B", 2, 4]
```

## APPENDIX – CODE PLACEHOLDER 2

This page is intentionally reserved for additional editable source code. Suggested material includes fee management, mess management, complaint handling, attendance processing, reporting logic, or integration code. Insert the actual code developed by the team rather than sample code that was not implemented in the project.

**INSERT ADDITIONAL CODE HERE**

**[ EDITABLE PLACEHOLDER – INSERT YOUR PROJECT MATERIAL HERE ]**

```

1  "use strict";
2
3  /**
4   * Create the MySQL database/tables/views and import the current JSON data.
5   *
6   *   npm run db:setup      create the schema, then push each service's data
7   *   npm run db:schema    create/refresh the schema only
8   *
9   * Connection settings come from the environment (MYSQL_HOST, MYSQL_PORT,
10  * MYSQL_USER, MYSQL_PASSWORD, MYSQL_DATABASE); the XAMPP defaults are used when
11  * they are not set.
12  */
13
14  const fs = require("fs");
15  const path = require("path");
16  const mysqlStore = require("../shared/mysql-store");
17
18  function summarise(data) {
19    return Object.entries(data)
20      .filter(([, value]) => Array.isArray(value))
21      .map(([key, value]) => `${key}=${value.length}`)
22      .join(", ");
23  }
24
25  async function main() {
26    if (!mysqlStore.enabled()) {
27      console.log("MySQL is disabled (DB_BACKEND=json or the mysql driver is missing). Nothing to do.");
28      return;
29    }
30
31    const cfg = mysqlStore.config();
32    console.log(`MySQL target: ${cfg.user}@${cfg.host}:${cfg.port}/${cfg.database}`);
33
34    if (process.argv.includes("--reset")) {
35      await mysqlStore.dropDatabase(cfg.database);
36      console.log(`Dropped existing database ${cfg.database} (--reset).`);
37    }
38
39    await mysqlStore.ensureSchema();
40    console.log("Schema ready (database, tables and views).");
41
42    const importData = !process.argv.includes("--schema-only");
43    const services = Object.keys(mysqlStore.SERVICE_SPECS);
44
45    for (const name of services) {
46      const dir = path.join(__dirname, "..", "services", name);

```

**APPENDIX – OUTPUT SCREENSHOT PLACEHOLDER**

This page is intentionally reserved for editable output screenshots from the completed project. Insert screenshots of the login page, student dashboard, room management, fee management, mess management, complaint management, attendance, admin dashboard, or reports as available. Add a brief caption under each screenshot identifying the screen and the expected output.

**INSERT PROJECT OUTPUT SCREENSHOT(S) HERE**

**[ EDITABLE PLACEHOLDER – INSERT YOUR PROJECT MATERIAL HERE ]**

## CHAPTER -13 REFERENCES

1. Project Review Presentation: “Smart Hostel and Mess Administration Management Information System”, supplied by the project team.
2. Project Problem Statement Submission Form: “Smart Hostel and Mess Administration Management Information System”, supplied by the project team.
3. Sample Project Documentation supplied as the formatting and organizational example for this report.
4. HTML5, CSS3, and JavaScript documentation and learning resources used by the project team during implementation, where applicable.
5. MySQL documentation and database-management resources used for schema design and query development, where applicable.
6. Visual Studio Code documentation and development resources used during coding, where applicable.
7. Apache / XAMPP documentation used for local web-server configuration when applicable.
8. Git and GitHub documentation used for version control and collaborative development, where applicable.
9. Postman documentation used for backend request-response testing, where applicable.
10. MySQL Workbench and phpMyAdmin documentation used for database management, where applicable.

Note: The final submitted report should be updated with the exact external documentation, tutorials, APIs, libraries, and versions actually used by the team. No external technology should be presented as implemented unless it was genuinely used in the project.

# Implementation Evidence, Application Screenshots and Source Code

## Smart Hostel and Mess Administration Management Information System

This appendix supplements the original report with supplied interface screenshots, the team demonstration photograph, and source/configuration files from the project ZIP.

## Application Screenshots and Demonstration

Figure A.1: Hostel and mess administration dashboard showing student count, bed occupancy, pending fees, outpass requests, complaints, attendance and meal registrations.

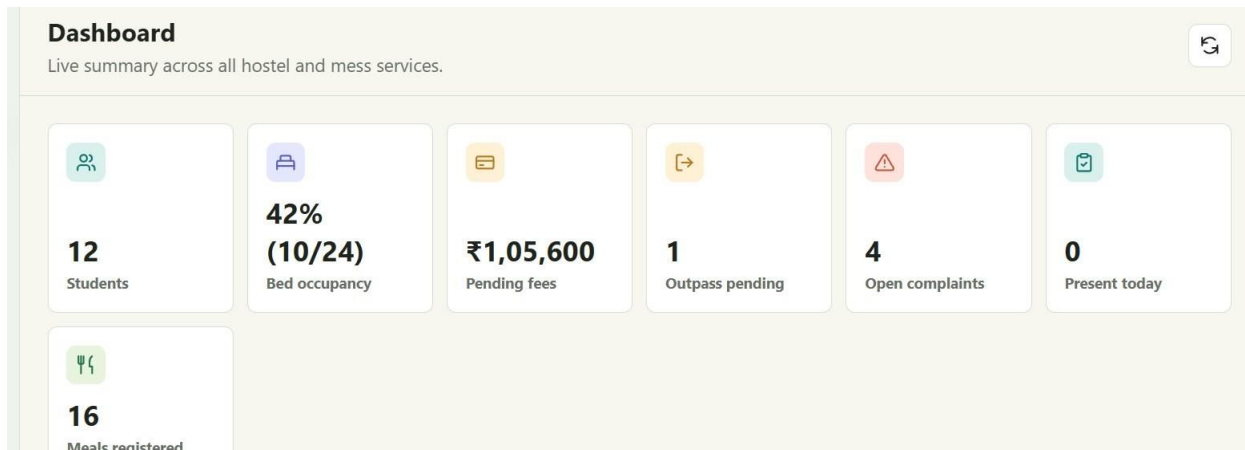


Figure A.2: System component overview showing gateway, authentication and role-based access, frontend, and service/database modules.

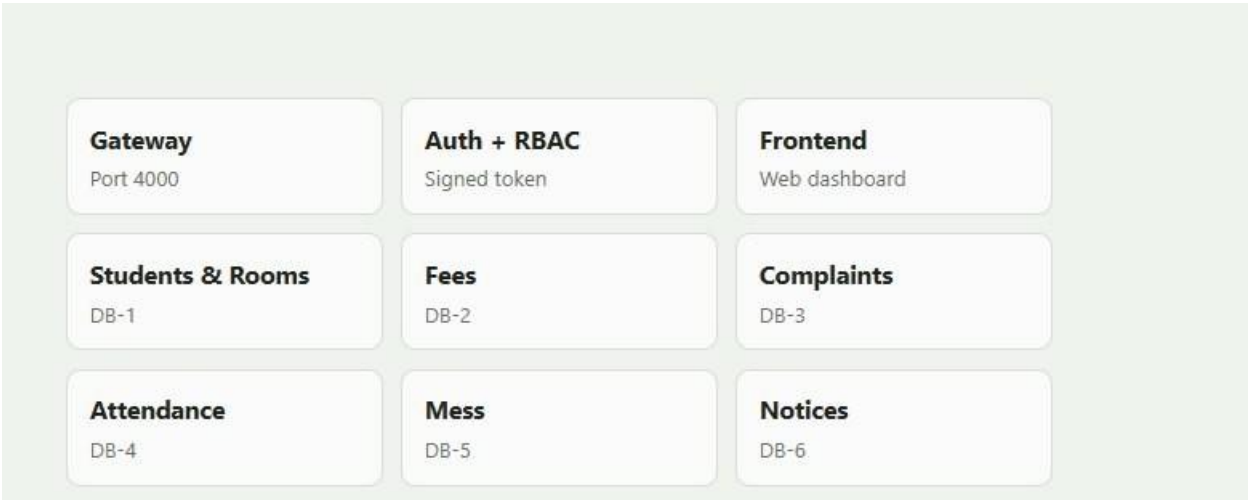


Figure A.3: Project demonstration photograph supplied by the team. GPS coordinates were not available for verification; this is not represented as a verified geotag.



# Source Code and Configuration

The following executable source and configuration files are reproduced from the submitted ZIP archive. Seed-data JSON files are not listed here.

## hostel/gateway/server.js

```
const crypto = require("crypto");
const fs = require("fs");
const http = require("http");
const path = require("path");
const {
  readBody,
  requestJson,
  sendError,
  sendJson
} = require("../shared/service-kit");

const port = Number(process.env.GATEWAY_PORT || 4000);
const frontendDir = path.join(__dirname, "..", "frontend");
const jwtSecret = process.env.JWT_SECRET || "development-demo-secret";

const services = {
  student: process.env.STUDENT_SERVICE_URL || "http://localhost:4101",
  fee: process.env.FEE_SERVICE_URL || "http://localhost:4102",
  complaint: process.env.COMPLAINT_SERVICE_URL || "http://localhost:4103",
  attendance: process.env.ATTENDANCE_SERVICE_URL || "http://localhost:4104",
  mess: process.env.MESS_SERVICE_URL || "http://localhost:4105",
  notification: process.env.NOTIFICATION_SERVICE_URL || "http://localhost:4106"
};

// Role-based access control enforced at the gateway (the UI also hides restricted screens).
// read / write : roles allowed for GET / any mutating method
// own : students are limited to their own records (studentId is forced by the gateway)
// studentWrite : mutating routes (by path suffix) a student may still call for their own record
const proxyRules = [
  { prefix: "/api/students", service: "student", target: "/students", read: ["admin", "mess", "student"],
    write: ["admin"], ownId: true },
  { prefix: "/api/rooms", service: "student", target: "/rooms", read: ["admin", "mess", "student"],
    write: ["admin"] },
  { prefix: "/api/allocations", service: "student", target: "/allocations", read: ["admin", "student"],
    write: ["admin"], own: true },
  { prefix: "/api/fees", service: "fee", target: "/fees", read: ["admin", "student"], write: ["admin"],
    own: true },
  { prefix: "/api/payments", service: "fee", target: "/payments", read: ["admin", "student"], write:
    ["admin"], own: true },
  { prefix: "/api/receipts", service: "fee", target: "/receipts", read: ["admin", "student"], write:
    ["admin"] },
  { prefix: "/api/complaints", service: "complaint", target: "/complaints", read: ["admin", "mess",
    "student"], write: ["admin", "mess"], own: true, studentWrite: [""] },
  { prefix: "/api/attendance", service: "attendance", target: "/attendance", read: ["admin", "student"],
    write: ["admin"], own: true },
  { prefix: "/api/leaves", service: "attendance", target: "/leaves", read: ["admin", "student"], write:
    ["admin"], own: true, studentWrite: [""] },
  { prefix: "/api/menu", service: "mess", target: "/menu", read: ["admin", "mess", "student"], write:
    ["admin", "mess"] },
  { prefix: "/api/registrations", service: "mess", target: "/registrations", read: ["admin", "mess",
    "student"], write: ["admin", "mess"], own: true, studentWrite: ["", "/cancel"] },
  { prefix: "/api/meal-attendance", service: "mess", target: "/meal-attendance", read: ["admin", "mess"],
    write: ["admin", "mess"] },
  { prefix: "/api/demand", service: "mess", target: "/demand", read: ["admin", "mess"], write: [] },
  { prefix: "/api/notices", service: "notification", target: "/notices", read: ["admin", "mess",
    "student"], write: ["admin"] },
  { prefix: "/api/notifications", service: "notification", target: "/notifications", read: ["admin",
    "mess", "student"], write: ["admin"], own: true, studentWrite: ["read"] }
].sort((x, y) => y.prefix.length - x.prefix.length);

function base64Url(value) {
  return Buffer.from(value)
    .toString("base64")
    .replace(/=/g, "")
    .replace(/\+/g, "-")
    .replace(/\//g, "_");
}

function fromBase64Url(value) {
  const padded = value.padEnd(value.length + ((4 - (value.length % 4)) % 4), "=");
  return Buffer.from(padded.replace(/-/g, "+").replace(/_/g, "/"), "base64").toString("utf8");
}

function signToken(user) {
  const payload = base64Url(JSON.stringify({
    sub: user.id,
    name: user.name,
    email: user.email,
    role: user.role,
    roomNo: user.roomNo,
    exp: Date.now() + 1000 * 60 * 60 * 8
  }));
  const signature = crypto.createHmac("sha256", jwtSecret).update(payload).digest("base64url");
  return `${payload}.${signature}`;
}

function verifyToken(token) {
  if (!token || !token.includes(".")) {
    return null;
  }

  const [payload, signature] = token.split(".");
  const expected = crypto.createHmac("sha256", jwtSecret).update(payload).digest("base64url");
  const expectedBuffer = Buffer.from(expected);
  const signatureBuffer = Buffer.from(signature);

  if (expectedBuffer.length !== signatureBuffer.length || !crypto.timingSafeEqual(expectedBuffer,
    signatureBuffer)) {
    return null;
  }

  const parsed = JSON.parse(fromBase64Url(payload));
}
```

```

    if (parsed.exp < Date.now()) {
      return null;
    }

    return parsed;
  }

function authUser(req) {
  const header = req.headers.authorization || "";
  const token = header.startsWith("Bearer ") ? header.slice("Bearer ".length) : "";
  return verifyToken(token);
}

function contentType(filePath) {
  const ext = path.extname(filePath).toLowerCase();
  const types = {
    ".html": "text/html; charset=utf-8",
    ".css": "text/css; charset=utf-8",
    ".js": "application/javascript; charset=utf-8",
    ".json": "application/json; charset=utf-8",
    ".svg": "image/svg+xml",
    ".png": "image/png",
    ".jpg": "image/jpeg",
    ".jpeg": "image/jpeg"
  };
  return types[ext] || "application/octet-stream";
}

function serveStatic(req, res, pathname) {
  const requested = pathname === "/" ? "/index.html" : pathname;
  const resolved = path.normalize(path.join(frontendDir, decodeURIComponent(requested)));

  if (!resolved.startsWith(frontendDir)) {
    sendError(res, 403, "Forbidden");
    return;
  }

  const filePath = fs.existsSync(resolved) && fs.statSync(resolved).isFile()
    ? resolved
    : path.join(frontendDir, "index.html");

  fs.readFile(filePath, (error, contents) => {
    if (error) {
      sendError(res, 500, "Unable to read frontend asset");
      return;
    }

    res.writeHead(200, {
      "Content-Type": contentType(filePath),
      "Cache-Control": filePath.endsWith("index.html") ? "no-cache" : "public, max-age=3600"
    });
    res.end(contents);
  });
}

function findProxyRule(pathname) {
  return proxyRules.find((rule) => {
    return pathname === rule.prefix || pathname.startsWith(`${rule.prefix}/`);
  });
}

async function login(req, res) {
  let body;

  try {
    body = await readBody(req);
  } catch (error) {
    sendError(res, 400, error.message);
    return;
  }

  const response = await requestJson(`${services.student}/auth/verify`, {
    method: "POST",
    body
  });

  if (!response.ok) {
    sendJson(res, response.status, response.json);
    return;
  }

  sendJson(res, 200, {
    token: signToken(response.json),
    user: response.json
  });
}

async function serviceStatus(res) {
  const checks = await Promise.all(Object.entries(services).map(async ([name, url]) => {
    try {
      const response = await requestJson(`${url}/health`, { timeoutMs: 1500 });
      return {
        name,
        url,
        ok: response.ok,
        detail: response.json
      };
    } catch (error) {
      return {
        name,
        url,
        ok: false,
        detail: error.message
      };
    }
  }));

  sendJson(res, 200, {
    gateway: "ok",
    services: checks
  });
}

async function overview(res) {
  const stats = {};
  await Promise.all(Object.entries(services).map(async ([name, url]) => {

```

```

    try {
      const response = await requestJson(`${url}/stats`, { timeoutMs: 2000 });
      stats[name] = response.ok ? response.json : { service: name, unavailable: true };
    } catch (error) {
      stats[name] = { service: name, unavailable: true, error: error.message };
    }
  });
}

sendJson(res, 200, {
  generatedAt: new Date().toISOString(),
  summary: {
    students: stats.student.studentCount || 0,
    occupiedBeds: stats.student.occupiedBeds || 0,
    totalBeds: stats.student.totalBeds || 0,
    occupancyRate: stats.student.occupancyRate || 0,
    pendingFeeAmount: stats.fee.pendingAmount || 0,
    studentsWithDues: stats.fee.studentsWithDues || 0,
    openComplaints: stats.complaint.openComplaints || 0,
    presentToday: stats.attendance.presentToday || 0,
    pendingLeaves: stats.attendance.pendingLeaves || 0,
    mealsRegisteredToday: stats.mess.registeredToday || 0,
    activeNotices: stats.notification.activeNotices || 0
  },
  stats
});
}

function authorize(rule, method, suffix, user) {
  const isRead = method === "GET";
  const allowed = isRead ? rule.read : rule.write;

  if (allowed.includes(user.role)) {
    return true;
  }

  return user.role === "student"
    && !isRead
    && (rule.studentWrite || []).some((allowedSuffix) => allowedSuffix === "" ? suffix === "" :
      suffix.endsWith(allowedSuffix));
}

async function proxy(req, res, parsedUrl, rule, user) {
  const suffix = parsedUrl.pathname.slice(rule.prefix.length);

  if (!authorize(rule, req.method, suffix, user)) {
    sendError(res, 403, `The ${user.role} role is not permitted to ${req.method} ${rule.prefix}`);
    return;
  }

  let body;

  try {
    body = ["POST", "PUT", "PATCH", "DELETE"].includes(req.method) ? await readBody(req) : undefined;
  } catch (error) {
    sendError(res, 400, error.message);
    return;
  }

  const search = new URLSearchParams(parsedUrl.search);

  if (user.role === "student") {
    if (rule.ownId && suffix && decodeURIComponent(suffix.slice(1)) !== user.sub) {
      sendError(res, 403, "Students can only view their own profile");
      return;
    }

    if (rule.ownId && !suffix && req.method === "GET") {
      search.set("studentId", user.sub);
    }

    if (rule.own && req.method === "GET") {
      search.set("studentId", user.sub);
    }

    // Students act only for themselves: the gateway stamps their id on writes and
    // the owning service rejects changes to records that belong to someone else.
    if (rule.own && body) {
      body.studentId = user.sub;
    }
  }

  if (body && req.method === "POST" && !body.recordedBy) {
    body.recordedBy = user.sub;
  }

  const query = search.toString();
  const targetUrl = `${services[rule.service]}${rule.target}${suffix}${query ? `?${query}` : ""}`;

  try {
    const response = await requestJson(targetUrl, {
      method: req.method,
      body,
      headers: { "X-User-Id": user.sub, "X-User-Role": user.role },
      timeoutMs: 6000
    });

    sendJson(res, response.status, response.json);
  } catch (error) {
    sendError(res, 502, `${rule.service} service unavailable`, error.message);
  }
}

const server = http.createServer(async (req, res) => {
  if (req.method === "OPTIONS") {
    sendJson(res, 204, {});
    return;
  }

  const parsedUrl = new URL(req.url, `http://${req.headers.host}`);
  const pathname = parsedUrl.pathname;

  try {
    if (pathname === "/api/auth/login" && req.method === "POST") {
      await login(req, res);
      return;
    }
  }

```

```

    if (pathname === "/api/health") {
      sendJson(res, 200, {
        service: "api-gateway",
        status: "ok",
        checkedAt: new Date().toISOString()
      });
      return;
    }

    if (pathname === "/api/service-status") {
      await serviceStatus(res);
      return;
    }

    if (pathname.startsWith("/api/")) {
      const user = authUser(req);

      if (!user) {
        sendError(res, 401, "Authentication required");
        return;
      }

      if (pathname === "/api/auth/me" && req.method === "GET") {
        sendJson(res, 200, { user });
        return;
      }

      if (pathname === "/api/overview" && req.method === "GET") {
        if (user.role === "student") {
          sendError(res, 403, "The student role is not permitted to view the administration overview");
          return;
        }
        await overview(res);
        return;
      }

      const rule = findProxyRule(pathname);
      if (!rule) {
        sendError(res, 404, "Gateway route not found");
        return;
      }

      await proxy(req, res, parsedUrl, rule, user);
      return;
    }

    serveStatic(req, res, pathname);
  } catch (error) {
    sendError(res, 500, error.message || "Unexpected gateway error");
  }
});

server.listen(port, () => {
  console.log(`api-gateway listening on http://localhost:${port}`);
});

```

## hostel/services/attendance-service/server.js

```

const path = require("path");
const {
  createId,
  createService,
  nowIso,
  requireFields,
  today
} = require("../shared/service-kit");

const dataFile = path.join(__dirname, "db.json");
const seedFile = path.join(__dirname, "seed.json");
const port = Number(process.env.ATTENDANCE_SERVICE_PORT || 4104);
const attendanceStatuses = ["Present", "Absent", "Late", "On Leave"];
const leaveStatuses = ["Pending", "Approved", "Rejected", "Cancelled"];
const isDate = (value) => /^d{4}-d{2}-d{2}$/.test(String(value)) && !Number.isNaN(Date.parse(value));

function upsert(db, { studentId, date, status, remark, markedBy }) {
  const existing = db.attendance.find((item) => item.studentId === studentId && item.date === date);

  if (existing) {
    existing.status = status;
    existing.remark = remark || "";
    existing.markedBy = markedBy || existing.markedBy;
    existing.updatedAt = nowIso();
    return existing;
  }

  const record = {
    id: createId("att"),
    studentId,
    date,
    status,
    remark: remark || "",
    markedBy: markedBy || "system",
    updatedAt: nowIso()
  };
  db.attendance.push(record);
  return record;
}

function countBy(records) {
  const counts = { Present: 0, Absent: 0, Late: 0, "On Leave": 0 };
  for (const record of records) {
    counts[record.status] += 1;
  }
  return counts;
}

function stats(db) {
  const todays = db.attendance.filter((item) => item.date === today());
  const counts = countBy(todays);

  return {
    service: "attendance-service",

```

```

    date: today(),
    markedToday: todays.length,
    presentToday: counts.Present + counts.Late,
    absentToday: counts.Absent,
    onLeaveToday: counts["On Leave"],
    pendingLeaves: db.leaves.filter((item) => item.status === "Pending").length,
    approvedLeaves: db.leaves.filter((item) => item.status === "Approved").length,
    recordCount: db.attendance.length
  };
}

createService({
  name: "attendance-service",
  port,
  dataFile,
  seedFile,
  routes: [
    {
      method: "GET",
      path: "/attendance",
      handler: ({ db, query, send }) => {
        const list = db.attendance
          .filter((item) => !query.studentId || item.studentId === query.studentId)
          .filter((item) => !query.date || item.date === query.date)
          .filter((item) => !query.from || item.date >= query.from)
          .filter((item) => !query.to || item.date <= query.to)
          .filter((item) => !query.status || item.status === query.status)
          .sort((a, b) => b.date.localeCompare(a.date));
        send(200, list);
      }
    },
    {
      method: "POST",
      path: "/attendance",
      handler: ({ db, body, save, send, error }) => {
        try {
          requireFields(body, ["studentId", "date", "status"]);
        } catch (validationError) {
          error(400, validationError.message);
          return;
        }

        if (!isDate(body.date)) {
          error(400, "Date must use the YYYY-MM-DD format");
          return;
        }

        if (body.date > today()) {
          error(400, "Attendance cannot be marked for a future date");
          return;
        }

        if (!attendanceStatuses.includes(body.status)) {
          error(400, `Status must be one of: ${attendanceStatuses.join(", ")}`);
          return;
        }

        const record = upsert(db, body);
        save();
        send(201, record);
      }
    },
    {
      method: "POST",
      path: "/attendance/bulk",
      handler: ({ db, body, save, send, error }) => {
        try {
          requireFields(body, ["date", "records"]);
        } catch (validationError) {
          error(400, validationError.message);
          return;
        }

        if (!isDate(body.date) || body.date > today()) {
          error(400, "Date must be a valid, non-future YYYY-MM-DD date");
          return;
        }

        if (!Array.isArray(body.records) || !body.records.length) {
          error(400, "records must be a non-empty array");
          return;
        }

        const invalid = body.records.find((item) => !item.studentId ||
!attendanceStatuses.includes(item.status));
        if (invalid) {
          error(400, "Every record needs a studentId and a valid status");
          return;
        }

        const saved = body.records.map((item) => upsert(db, { ...item, date: body.date, markedBy:
body.markedBy }));
        save();
        send(201, { date: body.date, saved: saved.length, counts: countBy(saved) });
      }
    },
    {
      method: "GET",
      path: "/attendance/summary",
      handler: ({ db, query, send }) => {
        const date = query.date || today();
        const records = db.attendance.filter((item) => item.date === date);
        send(200, { date, total: records.length, counts: countBy(records) });
      }
    },
    {
      method: "GET",
      path: "/attendance/summary/:studentId",
      handler: ({ db, params, query, send }) => {
        const records = db.attendance
          .filter((item) => item.studentId === params.studentId)
          .filter((item) => !query.from || item.date >= query.from)
          .filter((item) => !query.to || item.date <= query.to);
        const counts = countBy(records);
        const attended = counts.Present + counts.Late;
        const considered = records.length - counts["On Leave"];
      }
    }
  ]
});

```

```

        send(200, {
          studentId: params.studentId,
          days: records.length,
          counts,
          attendancePercentage: considered > 0 ? Math.round((attended / considered) * 100) : 100
        });
      }
    },
    {
      method: "GET",
      path: "/leaves",
      handler: ({ db, query, send }) => {
        const list = db.leaves
          .filter((item) => !query.studentId || item.studentId === query.studentId)
          .filter((item) => !query.status || item.status === query.status)
          .sort((a, b) => b.createdAt.localeCompare(a.createdAt));
        send(200, list);
      }
    },
    {
      method: "POST",
      path: "/leaves",
      handler: ({ db, body, save, send, error }) => {
        try {
          requireFields(body, ["studentId", "fromDate", "toDate", "reason"]);
        } catch (validationError) {
          error(400, validationError.message);
          return;
        }

        if (!isDate(body.fromDate) || !isDate(body.toDate) || body.toDate < body.fromDate) {
          error(400, "Leave dates must be valid and the end date cannot precede the start date");
          return;
        }

        const overlap = db.leaves.find((item) => {
          return item.studentId === body.studentId
            && ["Pending", "Approved"].includes(item.status)
            && body.fromDate <= item.toDate
            && item.fromDate <= body.toDate;
        });

        if (overlap) {
          error(409, "An active leave request already covers part of these dates", { conflictId:
            overlap.id });
          return;
        }

        const leave = {
          id: createId("lve"),
          studentId: body.studentId,
          fromDate: body.fromDate,
          toDate: body.toDate,
          reason: body.reason.trim(),
          destination: body.destination || "",
          status: "Pending",
          reviewedBy: null,
          createdAt: nowIso()
        };

        db.leaves.push(leave);
        save();
        send(201, leave);
      }
    },
    {
      method: "PATCH",
      path: "/leaves/:id/status",
      handler: ({ db, params, body, save, send, error }) => {
        const leave = db.leaves.find((item) => item.id === params.id);

        if (!leave) {
          error(404, "Leave request not found");
          return;
        }

        if (!leaveStatuses.includes(body.status)) {
          error(400, `Status must be one of: ${leaveStatuses.join(", ")}`);
          return;
        }

        leave.status = body.status;
        leave.reviewedBy = body.reviewedBy || leave.reviewedBy;
        leave.reviewedAt = nowIso();
        save();
        send(200, leave);
      }
    },
    {
      method: "GET",
      path: "/stats",
      handler: ({ db, send }) => send(200, stats(db))
    }
  ]
});

```

## hostel/services/complaint-service/server.js

```

const path = require("path");
const {
  createId,
  createService,
  nowIso,
  requireFields
} = require("../shared/service-kit");

const dataFile = path.join(__dirname, "db.json");
const seedFile = path.join(__dirname, "seed.json");
const port = Number(process.env.COMPLAINT_SERVICE_PORT || 4103);
const validStatuses = ["Pending", "Assigned", "In Progress", "Resolved"];

function complaintStats(db) {

```

```

const byStatus = validStatuses.reduce((accumulator, status) => {
  accumulator[status] = db.complaints.filter((item) => item.status === status).length;
  return accumulator;
}, {});

return {
  service: "complaint-service",
  complaintCount: db.complaints.length,
  openComplaints: db.complaints.filter((item) => item.status !== "Resolved").length,
  resolvedComplaints: byStatus.Resolved,
  highPriority: db.complaints.filter((item) => item.priority === "High" && item.status !== "Resolved").length,
  byStatus
};
}

function filterComplaints(complaints, query) {
  return complaints.filter((complaint) => {
    const residentMatch = !query.studentId || complaint.studentId === query.studentId;
    const statusMatch = !query.status || complaint.status === query.status;
    const categoryMatch = !query.category || complaint.category === query.category;
    return residentMatch && statusMatch && categoryMatch;
  });
}

createService({
  name: "complaint-service",
  port,
  dataFile,
  seedFile,
  routes: [
    {
      method: "GET",
      path: "/complaints",
      handler: ({ db, query, send }) => {
        const complaints = filterComplaints(db.complaints, query)
          .sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt));
        send(200, complaints);
      }
    },
    {
      method: "GET",
      path: "/complaints/:id",
      handler: ({ db, params, send, error }) => {
        const complaint = db.complaints.find((item) => item.id === params.id);

        if (!complaint) {
          error(404, "Complaint not found");
          return;
        }

        send(200, complaint);
      }
    },
    {
      method: "POST",
      path: "/complaints",
      handler: ({ db, body, save, send, error }) => {
        try {
          requireFields(body, ["studentId", "category", "description"]);
        } catch (validationError) {
          error(validationError.statusCode || 400, validationError.message);
          return;
        }

        const createdAt = nowIso();
        const complaint = {
          id: createId("cmp"),
          studentId: body.studentId,
          category: body.category,
          description: body.description.trim(),
          priority: body.priority || "Medium",
          status: "Pending",
          assignedTo: null,
          createdAt: createdAt,
          resolvedDate: null,
          activity: [
            {
              at: createdAt,
              message: "Complaint submitted"
            }
          ]
        };

        db.complaints.push(complaint);
        save();
        send(201, complaint);
      }
    },
    {
      method: "PATCH",
      path: "/complaints/:id/assign",
      handler: ({ db, params, body, save, send, error }) => {
        const complaint = db.complaints.find((item) => item.id === params.id);

        if (!complaint) {
          error(404, "Complaint not found");
          return;
        }

        if (!body.assignedTo) {
          error(400, "assignedTo is required");
          return;
        }

        complaint.assignedTo = body.assignedTo;
        complaint.status = "Assigned";
        complaint.activity.push({
          at: nowIso(),
          message: `Assigned to ${body.assignedTo}`
        });

        save();
        send(200, complaint);
      }
    }
  ],
});

```

```

{
  method: "PATCH",
  path: "/complaints/:id/status",
  handler: ({ db, params, body, save, send, error }) => {
    const complaint = db.complaints.find((item) => item.id === params.id);

    if (!complaint) {
      error(404, "Complaint not found");
      return;
    }

    if (!validStatuses.includes(body.status)) {
      error(400, `Status must be one of: ${validStatuses.join(", ")}`);
      return;
    }

    complaint.status = body.status;

    if (body.assignedTo !== undefined) {
      complaint.assignedTo = body.assignedTo;
    }

    if (body.status === "Resolved") {
      complaint.resolvedDate = nowIso();
    } else {
      complaint.resolvedDate = null;
    }

    complaint.activity.push({
      at: nowIso(),
      message: body.note || `Status changed to ${body.status}`
    });

    save();
    send(200, complaint);
  },
  {
    method: "GET",
    path: "/stats",
    handler: ({ db, send }) => {
      send(200, complaintStats(db));
    }
  }
]
});

```

## hostel/services/fee-service/server.js

```

const path = require("path");
const {
  clone,
  createId,
  createService,
  nowIso,
  replaceObject,
  requestJson,
  requireFields,
  today
} = require("../shared/service-kit");

const dataFile = path.join(__dirname, "db.json");
const seedFile = path.join(__dirname, "seed.json");
const port = Number(process.env.FEE_SERVICE_PORT || 4102);
const studentServiceUrl = process.env.STUDENT_SERVICE_URL || "http://localhost:4101";
const feeTypes = ["Hostel Rent", "Mess Charges", "Security Deposit", "Electricity", "Fine", "Other"];

// A pending fee whose due date has passed is reported as overdue.
function effectiveStatus(fee) {
  if (fee.status === "pending" && fee.dueDate < today()) {
    return "overdue";
  }
  return fee.status;
}

function view(fee) {
  return { ...fee, status: effectiveStatus(fee) };
}

function filtered(items, query) {
  return items.filter((item) => {
    return (!query.studentId || item.studentId === query.studentId)
      && (!query.status || (item.status ? effectiveStatus(item) : item.status) === query.status)
      && (!query.feeType || item.feeType === query.feeType)
      && (!query.term || item.term === query.term);
  });
}

function receiptNumber(studentId) {
  const short = studentId.replace(/[^0-9]/g, "").slice(-4).padStart(4, "0");
  return `RCP-${nowIso().slice(0, 7).replace("-", "")}-${short}-${Date.now().toString(36).toUpperCase()}`;
}

function feeStats(db) {
  const fees = db.fees.map(view);
  const unpaid = fees.filter((fee) => fee.status !== "paid");

  return {
    service: "fee-service",
    totalBilled: fees.reduce((sum, fee) => sum + Number(fee.amount), 0),
    totalCollected: db.payments.reduce((sum, payment) => sum + Number(payment.amount), 0),
    pendingAmount: unpaid.reduce((sum, fee) => sum + Number(fee.amount), 0),
    pendingFees: fees.filter((fee) => fee.status === "pending").length,
    overdueFees: fees.filter((fee) => fee.status === "overdue").length,
    studentsWithDues: new Set(unpaid.map((fee) => fee.studentId)).size,
    receiptCount: db.payments.length
  };
}

async function verifyStudent(studentId) {
  const response = await requestJson(`${studentServiceUrl}/students/${encodeURIComponent(studentId)}`);
}

```

```

    if (!response.ok) {
      throw Object.assign(new Error(response.status === 404 ? "Student not found" : "Student service
        unavailable"), {
        statusCode: response.status === 404 ? 400 : 503
      });
    }
  }

  return response.json;
}

createService({
  name: "fee-service",
  port,
  dataFile,
  seedFile,
  routes: [
    {
      method: "GET",
      path: "/fees",
      handler: ({ db, query, send }) => {
        send(200, filtered(db.fees, query).map(view).sort((a, b) => a.dueDate.localeCompare(b.dueDate)));
      }
    },
    {
      method: "POST",
      path: "/fees",
      handler: async ({ db, body, save, send, error }) => {
        try {
          requireFields(body, ["studentId", "feeType", "amount", "term", "dueDate"]);
        } catch (validationError) {
          error(400, validationError.message);
          return;
        }

        if (!feeTypes.includes(body.feeType)) {
          error(400, `Fee type must be one of: ${feeTypes.join(", ")}`);
          return;
        }

        if (!(Number(body.amount) > 0)) {
          error(400, "Amount must be a positive number");
          return;
        }

        try {
          await verifyStudent(body.studentId);
        } catch (verificationError) {
          error(verificationError.statusCode || 503, verificationError.message);
          return;
        }

        if (db.fees.some((fee) => fee.studentId === body.studentId && fee.feeType === body.feeType &&
          fee.term === body.term)) {
          error(409, "This fee has already been raised for the student and term");
          return;
        }

        const fee = {
          id: createId("fee"),
          studentId: body.studentId,
          feeType: body.feeType,
          amount: Number(body.amount),
          term: body.term,
          dueDate: body.dueDate,
          status: "pending",
          paymentDate: null,
          receiptNo: null,
          notes: body.notes || ""
        };

        db.fees.push(fee);
        save();
        send(201, view(fee));
      }
    },
    {
      method: "PATCH",
      path: "/fees/:id",
      handler: ({ db, params, body, save, send, error }) => {
        const fee = db.fees.find((item) => item.id === params.id);

        if (!fee) {
          error(404, "Fee record not found");
          return;
        }

        if (fee.status === "paid") {
          error(409, "A paid fee cannot be edited");
          return;
        }

        for (const field of ["amount", "dueDate", "notes"]) {
          if (body[field] !== undefined) {
            fee[field] = field === "amount" ? Number(body[field]) : body[field];
          }
        }

        save();
        send(200, view(fee));
      }
    },
    {
      method: "GET",
      path: "/payments",
      handler: ({ db, query, send }) => {
        const list = filtered(db.payments, query).sort((a, b) =>
          b.paymentDate.localeCompare(a.paymentDate));
        send(200, list);
      }
    },
    {
      method: "GET",
      path: "/payments/pending/:studentId",
      handler: ({ db, params, send }) => {
        const fees = db.fees.filter((fee) => fee.studentId === params.studentId && fee.status !==
          "paid").map(view);
      }
    }
  ]
});

```

```

    send(200, {
      studentId: params.studentId,
      pendingAmount: fees.reduce((sum, fee) => sum + Number(fee.amount), 0),
      fees
    });
  },
  {
    method: "POST",
    path: "/payments/pay",
    handler: async ({ db, body, save, send, error }) => {
      try {
        requireFields(body, ["studentId", "feeId", "method"]);
      } catch (validationError) {
        error(400, validationError.message);
        return;
      }

      let student;
      try {
        student = await verifyStudent(body.studentId);
      } catch (verificationError) {
        error(verificationError.statusCode || 503, verificationError.message);
        return;
      }

      // Snapshot gives all-or-nothing behaviour, mirroring a database transaction.
      const snapshot = clone(db);

      try {
        const fee = db.fees.find((item) => item.id === body.feeId);

        if (!fee) {
          throw Object.assign(new Error("Fee record not found"), { statusCode: 404 });
        }

        if (fee.studentId !== body.studentId) {
          throw Object.assign(new Error("Fee does not belong to this student"), { statusCode: 400 });
        }

        if (fee.status === "paid") {
          throw Object.assign(new Error("Fee is already paid"), { statusCode: 409 });
        }

        const paidAt = nowIso();
        const receiptNo = receiptNumber(body.studentId);
        const payment = {
          id: createId("pay"),
          studentId: body.studentId,
          studentName: student.name,
          feeId: fee.id,
          feeType: fee.feeType,
          term: fee.term,
          amount: Number(fee.amount),
          paymentDate: paidAt,
          method: body.method,
          recordedBy: body.recordedBy || "admin",
          receiptNo
        };

        db.payments.push(payment);
        fee.status = "paid";
        fee.paymentDate = paidAt;
        fee.receiptNo = receiptNo;

        save();
        send(201, payment);
      } catch (paymentError) {
        replaceObject(db, snapshot);
        error(paymentError.statusCode || 500, paymentError.message);
      }
    },
    {
      method: "GET",
      path: "/receipts/:paymentId",
      handler: ({ db, params, send, error }) => {
        const payment = db.payments.find((item) => item.id === params.paymentId || item.receiptNo ===
        params.paymentId);

        if (!payment) {
          error(404, "Receipt not found");
          return;
        }

        send(200, payment);
      }
    },
    {
      method: "GET",
      path: "/stats",
      handler: ({ db, send }) => send(200, feeStats(db))
    }
  ]
});

```

## hostel/services/mess-service/server.js

```

const path = require("path");
const {
  createId,
  createService,
  nowIso,
  requireFields,
  today
} = require("../shared/service-kit");

const dataFile = path.join(__dirname, "db.json");
const seedFile = path.join(__dirname, "seed.json");
const port = Number(process.env.MESS_SERVICE_PORT || 4105);
const mealTypes = ["Breakfast", "Lunch", "Snacks", "Dinner"];
const days = ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"];

```

```

const isDate = (value) => /^d{4}-d{2}-d{2}$/.test(String(value)) && !Number.isNaN(Date.parse(value));

function dayName(date) {
  return days[new Date(`${date}T00:00:00Z`).getUTCDay()];
}

function demand(db, date) {
  const result = {};

  for (const meal of mealTypes) {
    const registered = db.registrations.filter((item) => item.date === date && item.mealType === meal &&
      item.status === "Registered").length;
    const attended = db.mealAttendance.filter((item) => item.date === date && item.mealType ===
      meal).length;
    result[meal] = { registered, attended };
  }

  return { date, day: dayName(date), meals: result };
}

function stats(db) {
  const todayDemand = demand(db, today());
  const meals = Object.values(todayDemand.meals);

  return {
    service: "mess-service",
    date: today(),
    menuItems: db.menu.length,
    registeredToday: meals.reduce((sum, item) => sum + item.registered, 0),
    servedToday: meals.reduce((sum, item) => sum + item.attended, 0),
    todayDemand: todayDemand.meals
  };
}

createService({
  name: "mess-service",
  port,
  dataFile,
  seedFile,
  routes: [
    {
      method: "GET",
      path: "/menu",
      handler: ({ db, query, send }) => {
        const list = db.menu
          .filter((item) => !query.day || item.day === query.day)
          .filter((item) => !query.mealType || item.mealType === query.mealType)
          .sort((a, b) => days.indexOf(a.day) - days.indexOf(b.day) || mealTypes.indexOf(a.mealType) -
            mealTypes.indexOf(b.mealType));
        send(200, list);
      }
    },
    {
      method: "POST",
      path: "/menu",
      handler: ({ db, body, save, send, error }) => {
        try {
          requireFields(body, ["day", "mealType", "items"]);
        } catch (validationError) {
          error(400, validationError.message);
          return;
        }

        if (!days.includes(body.day) || !mealTypes.includes(body.mealType)) {
          error(400, "Invalid day or meal type");
          return;
        }

        if (db.menu.some((item) => item.day === body.day && item.mealType === body.mealType)) {
          error(409, "A menu already exists for this day and meal; edit it instead");
          return;
        }

        const entry = {
          id: createId("mnu"),
          day: body.day,
          mealType: body.mealType,
          items: body.items.trim(),
          isVeg: body.isVeg === undefined ? true : String(body.isVeg) !== "false",
          updatedAt: nowIso()
        };

        db.menu.push(entry);
        save();
        send(201, entry);
      }
    },
    {
      method: "PATCH",
      path: "/menu/:id",
      handler: ({ db, params, body, save, send, error }) => {
        const entry = db.menu.find((item) => item.id === params.id);

        if (!entry) {
          error(404, "Menu entry not found");
          return;
        }

        if (body.items !== undefined) {
          entry.items = String(body.items).trim();
        }

        if (body.isVeg !== undefined) {
          entry.isVeg = String(body.isVeg) !== "false";
        }

        entry.updatedAt = nowIso();
        save();
        send(200, entry);
      }
    },
    {
      method: "GET",
      path: "/registrations",
      handler: ({ db, query, send }) => {
        const list = db.registrations

```

```

        .filter((item) => !query.studentId || item.studentId === query.studentId)
        .filter((item) => !query.date || item.date === query.date)
        .filter((item) => !query.mealType || item.mealType === query.mealType)
        .filter((item) => !query.status || item.status === query.status)
        .sort((a, b) => b.date.localeCompare(a.date));
    send(200, list);
  }
},
{
  method: "POST",
  path: "/registrations",
  handler: ({ db, body, save, send, error }) => {
    try {
      requireFields(body, ["studentId", "date", "mealType"]);
    } catch (validationError) {
      error(400, validationError.message);
      return;
    }

    if (!isDate(body.date) || !mealTypes.includes(body.mealType)) {
      error(400, "Invalid date or meal type");
      return;
    }

    if (body.date < today()) {
      error(400, "Meals cannot be registered for a past date");
      return;
    }

    if (!db.menu.some((item) => item.day === dayName(body.date) && item.mealType === body.mealType)) {
      error(409, `No ${body.mealType} is scheduled on ${dayName(body.date)}`);
      return;
    }

    const existing = db.registrations.find((item) => {
      return item.studentId === body.studentId && item.date === body.date && item.mealType ===
body.mealType;
    });

    if (existing && existing.status === "Registered") {
      error(409, "Already registered for this meal");
      return;
    }

    if (existing) {
      existing.status = "Registered";
      existing.createdAt = nowIso();
      save();
      send(201, existing);
      return;
    }

    const registration = {
      id: createId("reg"),
      studentId: body.studentId,
      date: body.date,
      mealType: body.mealType,
      status: "Registered",
      createdAt: nowIso()
    };

    db.registrations.push(registration);
    save();
    send(201, registration);
  }
},
{
  method: "PATCH",
  path: "/registrations/:id/cancel",
  handler: ({ db, params, body, save, send, error }) => {
    const registration = db.registrations.find((item) => item.id === params.id);

    if (!registration) {
      error(404, "Registration not found");
      return;
    }

    if (body.studentId && registration.studentId !== body.studentId) {
      error(403, "You can only cancel your own registrations");
      return;
    }

    if (registration.date < today()) {
      error(409, "Past registrations cannot be cancelled");
      return;
    }

    registration.status = "Cancelled";
    save();
    send(200, registration);
  }
},
{
  method: "GET",
  path: "/meal-attendance",
  handler: ({ db, query, send }) => {
    const list = db.mealAttendance
      .filter((item) => !query.studentId || item.studentId === query.studentId)
      .filter((item) => !query.date || item.date === query.date)
      .filter((item) => !query.mealType || item.mealType === query.mealType)
      .sort((a, b) => b.servedAt.localeCompare(a.servedAt));
    send(200, list);
  }
},
{
  method: "POST",
  path: "/meal-attendance",
  handler: ({ db, body, save, send, error }) => {
    try {
      requireFields(body, ["studentId", "mealType"]);
    } catch (validationError) {
      error(400, validationError.message);
      return;
    }

    const date = body.date || today();

```

```

    if (!mealTypes.includes(body.mealType) || date > today()) {
      error(400, "Invalid meal type or future date");
      return;
    }

    if (db.mealAttendance.some((item) => item.studentId === body.studentId && item.date === date &&
item.mealType === body.mealType)) {
      error(409, "Meal already recorded for this student");
      return;
    }

    const registered = db.registrations.some((item) => {
      return item.studentId === body.studentId && item.date === date && item.mealType ===
body.mealType && item.status === "Registered";
    });

    const record = {
      id: createId("mel"),
      studentId: body.studentId,
      date,
      mealType: body.mealType,
      registered,
      servedBy: body.servedBy || "mess staff",
      servedAt: nowIso()
    };

    db.mealAttendance.push(record);
    save();
    send(201, record);
  },
  {
    method: "GET",
    path: "/demand",
    handler: ({ db, query, send, error }) => {
      const date = query.date || today();

      if (!isDate(date)) {
        error(400, "Date must use the YYYY-MM-DD format");
        return;
      }

      send(200, demand(db, date));
    }
  },
  {
    method: "GET",
    path: "/stats",
    handler: ({ db, send }) => send(200, stats(db))
  }
]
});

```

## hostel/services/notification-service/server.js

```

const path = require("path");
const {
  createId,
  createService,
  nowIso,
  requireFields
} = require("../shared/service-kit");

const dataFile = path.join(__dirname, "db.json");
const seedFile = path.join(__dirname, "seed.json");
const port = Number(process.env.NOTIFICATION_SERVICE_PORT || 4106);

function notificationStats(db) {
  return {
    service: "notification-service",
    noticeCount: db.notices.length,
    activeNotices: db.notices.filter((notice) => !notice.expiresOn || notice.expiresOn >=
nowIso().slice(0, 10)).length,
    unreadNotifications: db.notifications.filter((notification) => !notification.read).length,
    notificationCount: db.notifications.length
  };
}

createService({
  name: "notification-service",
  port,
  dataFile,
  seedFile,
  routes: [
    {
      method: "GET",
      path: "/notices",
      handler: ({ db, query, send }) => {
        const notices = db.notices.filter((notice) => {
          const priorityMatch = !query.priority || notice.priority === query.priority;
          const categoryMatch = !query.category || notice.category === query.category;
          return priorityMatch && categoryMatch;
        });

        send(200, notices.sort((a, b) => new Date(b.createdDate) - new Date(a.createdDate)));
      }
    },
    {
      method: "POST",
      path: "/notices",
      handler: ({ db, body, save, send, error }) => {
        try {
          requireFields(body, ["adminId", "title", "description"]);
        } catch (validationError) {
          error(validationError.statusCode || 400, validationError.message);
          return;
        }

        const notice = {
          id: createId("not"),
          adminId: body.adminId,
          title: body.title.trim(),

```

```

        description: body.description.trim(),
        category: body.category || "General",
        priority: body.priority || "Medium",
        audience: body.audience || "All Students",
        createdAt: nowIso(),
        expiresOn: body.expiresOn || null
    };

    db.notices.push(notice);
    save();
    send(201, notice);
}
},
{
    method: "PATCH",
    path: "/notices/:id",
    handler: ({ db, params, body, save, send, error }) => {
        const notice = db.notices.find((item) => item.id === params.id);

        if (!notice) {
            error(404, "Notice not found");
            return;
        }

        for (const field of ["title", "description", "category", "priority", "audience", "expiresOn"]) {
            if (body[field] !== undefined) {
                notice[field] = body[field];
            }
        }

        save();
        send(200, notice);
    }
},
{
    method: "GET",
    path: "/notifications",
    handler: ({ db, query, send }) => {
        const notifications = db.notifications.filter((notification) => {
            const residentMatch = !query.studentId || notification.studentId === query.studentId;
            const readMatch = query.read === undefined || String(notification.read) === query.read;
            return residentMatch && readMatch;
        });

        send(200, notifications.sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt)));
    }
},
{
    method: "POST",
    path: "/notifications",
    handler: ({ db, body, save, send, error }) => {
        try {
            requireFields(body, ["studentId", "title", "message"]);
        } catch (validationError) {
            error(validationError.statusCode || 400, validationError.message);
            return;
        }

        const notification = {
            id: createId("ntf"),
            studentId: body.studentId,
            title: body.title.trim(),
            message: body.message.trim(),
            channel: body.channel || "In App",
            read: false,
            createdAt: nowIso()
        };

        db.notifications.push(notification);
        save();
        send(201, notification);
    }
},
{
    method: "PATCH",
    path: "/notifications/:id/read",
    handler: ({ db, params, body, save, send, error }) => {
        const notification = db.notifications.find((item) => item.id === params.id);

        if (!notification) {
            error(404, "Notification not found");
            return;
        }

        if (body.studentId && notification.studentId !== body.studentId) {
            error(403, "You can only update your own notifications");
            return;
        }

        notification.read = true;
        save();
        send(200, notification);
    }
},
{
    method: "GET",
    path: "/stats",
    handler: ({ db, send }) => {
        send(200, notificationStats(db));
    }
}
];
});

```

## hostel/services/student-service/server.js

```

const path = require("path");
const {
    byId,
    createId,
    createService,
    requireFields,

```

```

    today,
    verifyPassword,
    hashPassword
  } = require("../shared/service-kit");

  const dataFile = path.join(__dirname, "db.json");
  const seedFile = path.join(__dirname, "seed.json");
  const port = Number(process.env.STUDENT_SERVICE_PORT || 4101);

  const studentFields = ["name", "phone", "email", "course", "year", "guardianName", "guardianPhone",
    "status"];

  function safe(user) {
    const { password, ...rest } = user;
    return rest;
  }

  function activeAllocation(db, studentId) {
    return db.allocations.find((item) => item.studentId === studentId && item.status === "Active") || null;
  }

  function occupancy(db, room) {
    const occupied = db.allocations.filter((item) => item.roomNo === room.roomNo && item.status ===
      "Active").length;
    let status = "Available";

    if (room.underMaintenance) {
      status = "Maintenance";
    } else if (occupied >= room.capacity) {
      status = "Full";
    }

    return { ...room, occupied, vacant: Math.max(0, room.capacity - occupied), status };
  }

  function withRoom(db, student) {
    const allocation = activeAllocation(db, student.id);
    return { ...safe(student), roomNo: allocation ? allocation.roomNo : null };
  }

  function stats(db) {
    const students = db.users.filter((user) => user.role === "student");
    const rooms = db.rooms.map((room) => occupancy(db, room));
    const beds = rooms.filter((room) => room.status !== "Maintenance").reduce((sum, room) => sum +
      room.capacity, 0);
    const occupied = rooms.reduce((sum, room) => sum + room.occupied, 0);

    return {
      service: "student-service",
      studentCount: students.length,
      activeStudents: students.filter((student) => student.status === "active").length,
      pendingRegistrations: students.filter((student) => student.status === "pending").length,
      roomCount: rooms.length,
      totalBeds: beds,
      occupiedBeds: occupied,
      vacantBeds: Math.max(0, beds - occupied),
      occupancyRate: beds ? Math.round((occupied / beds) * 100) : 0,
      unallocatedStudents: students.filter((student) => student.status === "active" &&
        !activeAllocation(db, student.id)).length
    };
  }

  createService({
    name: "student-service",
    port,
    dataFile,
    seedFile,
    routes: [
      {
        method: "POST",
        path: "/auth/verify",
        handler: ({ db, body, send, error }) => {
          try {
            requireFields(body, ["email", "password"]);
          } catch (validationError) {
            error(400, validationError.message);
            return;
          }

          const user = db.users.find((item) => item.email.toLowerCase() ===
            String(body.email).toLowerCase());

          if (!user || !verifyPassword(body.password, user.password)) {
            error(401, "Invalid email or password");
            return;
          }

          if (user.status === "vacated") {
            error(403, "This account is no longer active");
            return;
          }

          send(200, withRoom(db, user));
        },
      },
      {
        method: "GET",
        path: "/students",
        handler: ({ db, query, send }) => {
          const list = db.users
            .filter((user) => user.role === "student")
            .filter((user) => !query.status || user.status === query.status)
            .filter((user) => {
              const text = `${user.name} ${user.rollNo} ${user.email} ${user.course}`.toLowerCase();
              return !query.q || text.includes(String(query.q).toLowerCase());
            })
            .filter((user) => !query.studentId || user.id === query.studentId)
            .map((user) => withRoom(db, user))
            .sort((a, b) => a.rollNo.localeCompare(b.rollNo));
          send(200, list);
        },
      },
      {
        method: "GET",
        path: "/students/:id",
        handler: ({ db, params, send, error }) => {

```

```

const student = findById(db.users, params.id);
if (!student) {
  error(404, "Student not found");
  return;
}
send(200, withRoom(db, student));
},
{
  method: "POST",
  path: "/students",
  handler: ({ db, body, save, send, error }) => {
    try {
      requireFields(body, ["name", "rollNo", "phone", "email", "course"]);
    } catch (validationError) {
      error(400, validationError.message);
      return;
    }

    if (!/^d{10}$/.test(String(body.phone))) {
      error(400, "Phone number must contain 10 digits");
      return;
    }

    if (db.users.some((user) => user.email.toLowerCase() === String(body.email).toLowerCase())) {
      error(409, "A user with this email already exists");
      return;
    }

    if (db.users.some((user) => user.rollNo === String(body.rollNo).trim())) {
      error(409, "A student with this roll number already exists");
      return;
    }

    const student = {
      id: createId("stu"),
      rollNo: String(body.rollNo).trim(),
      name: body.name.trim(),
      phone: body.phone,
      email: body.email.trim().toLowerCase(),
      password: hashPassword(body.password || "student123"),
      role: "student",
      course: body.course,
      year: Number(body.year || 1),
      guardianName: body.guardianName || "",
      guardianPhone: body.guardianPhone || "",
      status: body.status || "active",
      joinedDate: today()
    };

    db.users.push(student);
    save();
    send(201, withRoom(db, student));
  },
  {
    method: "PATCH",
    path: "/students/:id",
    handler: ({ db, params, body, save, send, error }) => {
      const student = db.users.find((item) => item.id === params.id && item.role === "student");

      if (!student) {
        error(404, "Student not found");
        return;
      }

      for (const field of studentFields) {
        if (body[field] !== undefined) {
          student[field] = field === "year" ? Number(body[field]) : body[field];
        }
      }

      if (body.password) {
        student.password = hashPassword(body.password);
      }

      save();
      send(200, withRoom(db, student));
    }
  },
  {
    method: "GET",
    path: "/rooms",
    handler: ({ db, query, send }) => {
      const rooms = db.rooms
        .map((room) => occupancy(db, room))
        .filter((room) => !query.block || room.block === query.block)
        .filter((room) => !query.status || room.status === query.status)
        .sort((a, b) => a.roomNo.localeCompare(b.roomNo));
      send(200, rooms);
    }
  },
  {
    method: "POST",
    path: "/rooms",
    handler: ({ db, body, save, send, error }) => {
      try {
        requireFields(body, ["roomNo", "block", "floor", "capacity"]);
      } catch (validationError) {
        error(400, validationError.message);
        return;
      }

      if (db.rooms.some((room) => room.roomNo === body.roomNo)) {
        error(409, "Room already exists");
        return;
      }

      if (Number(body.capacity) < 1 || Number(body.capacity) > 6) {
        error(400, "Room capacity must be between 1 and 6");
        return;
      }

      const room = {
        roomNo: body.roomNo.trim(),
        block: body.block,

```

```

        floor: Number(body.floor),
        capacity: Number(body.capacity),
        roomType: body.roomType || "Standard",
        underMaintenance: false
    };

    db.rooms.push(room);
    save();
    send(201, occupancy(db, room));
},
{
    method: "PATCH",
    path: "/rooms/:roomNo",
    handler: ({ db, params, body, save, send, error }) => {
        const room = db.rooms.find((item) => item.roomNo === params.roomNo);

        if (!room) {
            error(404, "Room not found");
            return;
        }

        if (body.capacity !== undefined) {
            const occupied = occupancy(db, room).occupied;
            if (Number(body.capacity) < occupied) {
                error(409, `Capacity cannot be lower than current occupancy (${occupied})`);
                return;
            }
        }
        room.capacity = Number(body.capacity);

        if (body.underMaintenance !== undefined) {
            room.underMaintenance = Boolean(body.underMaintenance);
        }

        for (const field of ["roomType", "block", "floor"]) {
            if (body[field] !== undefined) {
                room[field] = field === "floor" ? Number(body[field]) : body[field];
            }
        }

        save();
        send(200, occupancy(db, room));
    }
},
{
    method: "GET",
    path: "/allocations",
    handler: ({ db, query, send }) => {
        const list = db.allocations
            .filter((item) => !query.studentId || item.studentId === query.studentId)
            .filter((item) => !query.roomNo || item.roomNo === query.roomNo)
            .filter((item) => !query.status || item.status === query.status)
            .sort((a, b) => b.allocatedOn.localeCompare(a.allocatedOn));
        send(200, list);
    }
},
{
    method: "POST",
    path: "/allocations",
    handler: ({ db, body, save, send, error }) => {
        try {
            requireFields(body, ["studentId", "roomNo"]);
        } catch (validationError) {
            error(400, validationError.message);
            return;
        }

        const student = db.users.find((item) => item.id === body.studentId && item.role === "student");
        const room = db.rooms.find((item) => item.roomNo === body.roomNo);

        if (!student) {
            error(404, "Student not found");
            return;
        }

        if (!room) {
            error(404, "Room not found");
            return;
        }

        if (student.status !== "active") {
            error(409, "Only active students can be allocated a room");
            return;
        }

        if (activeAllocation(db, student.id)) {
            error(409, "Student already has an active room allocation; vacate or transfer first");
            return;
        }

        const current = occupancy(db, room);
        if (current.status === "Maintenance") {
            error(409, "Room is under maintenance");
            return;
        }

        if (current.vacant < 1) {
            error(409, "Room is full");
            return;
        }

        const allocation = {
            id: createId("alc"),
            studentId: student.id,
            roomNo: room.roomNo,
            allocatedOn: body.allocatedOn || today(),
            vacatedOn: null,
            status: "Active"
        };

        db.allocations.push(allocation);
        save();
        send(201, allocation);
    }
},

```

```

{
  method: "PATCH",
  path: "/allocations/:id/vacate",
  handler: ({ db, params, save, send, error }) => {
    const allocation = db.allocations.find((item) => item.id === params.id);

    if (!allocation) {
      error(404, "Allocation not found");
      return;
    }

    if (allocation.status !== "Active") {
      error(409, "Allocation is already closed");
      return;
    }

    allocation.status = "Vacated";
    allocation.vacatedOn = today();
    save();
    send(200, allocation);
  },
},
{
  method: "GET",
  path: "/stats",
  handler: ({ db, send }) => send(200, stats(db))
}
]
});

```

## hostel/docker-compose.yml

```

services:
  api-gateway:
    image: node:20-alpine
    working_dir: /app
    volumes:
      - ./app
    command: ["node", "gateway/server.js"]
    environment:
      GATEWAY_PORT: 4000
      JWT_SECRET: development-demo-secret
      STUDENT_SERVICE_URL: http://student-service:4101
      FEE_SERVICE_URL: http://fee-service:4102
      COMPLAINT_SERVICE_URL: http://complaint-service:4103
      ATTENDANCE_SERVICE_URL: http://attendance-service:4104
      MESS_SERVICE_URL: http://mess-service:4105
      NOTIFICATION_SERVICE_URL: http://notification-service:4106
    ports:
      - "4000:4000"
    depends_on:
      - student-service
      - fee-service
      - complaint-service
      - attendance-service
      - mess-service
      - notification-service

  student-service:
    image: node:20-alpine
    working_dir: /app
    volumes:
      - ./app
    command: ["node", "services/student-service/server.js"]
    environment:
      STUDENT_SERVICE_PORT: 4101
    ports:
      - "4101:4101"

  fee-service:
    image: node:20-alpine
    working_dir: /app
    volumes:
      - ./app
    command: ["node", "services/fee-service/server.js"]
    environment:
      FEE_SERVICE_PORT: 4102
      STUDENT_SERVICE_URL: http://student-service:4101
    ports:
      - "4102:4102"

  complaint-service:
    image: node:20-alpine
    working_dir: /app
    volumes:
      - ./app
    command: ["node", "services/complaint-service/server.js"]
    environment:
      COMPLAINT_SERVICE_PORT: 4103
    ports:
      - "4103:4103"

  attendance-service:
    image: node:20-alpine
    working_dir: /app
    volumes:
      - ./app
    command: ["node", "services/attendance-service/server.js"]
    environment:
      ATTENDANCE_SERVICE_PORT: 4104
    ports:
      - "4104:4104"

  mess-service:
    image: node:20-alpine
    working_dir: /app
    volumes:
      - ./app
    command: ["node", "services/mess-service/server.js"]
    environment:
      MESS_SERVICE_PORT: 4105
    ports:

```

```

- "4105:4105"

notification-service:
  image: node:20-alpine
  working_dir: /app
  volumes:
    - ./app
  command: ["node", "services/notification-service/server.js"]
  environment:
    NOTIFICATION_SERVICE_PORT: 4106
  ports:
    - "4106:4106"

```

## hostel/docs/database-schema.sql

```

-- MySQL 8 schema for the Smart Hostel and Mess Administration Management Information System.
-- The runnable demo keeps each service's data in JSON files (database-per-service, no install needed);
-- this script is the relational design deliverable and the target for a production MySQL deployment.
-- Load with: mysql -u root -p < docs/database-schema.sql

```

```

CREATE DATABASE IF NOT EXISTS smart_hostel CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE smart_hostel;

--- ----- Users, rooms, allocation (student-
service)
CREATE TABLE users (
  id          VARCHAR(20) PRIMARY KEY,
  roll_no     VARCHAR(20) NOT NULL UNIQUE,
  name        VARCHAR(100) NOT NULL,
  phone       CHAR(10) NOT NULL,
  email       VARCHAR(120) NOT NULL UNIQUE,
  password_hash VARCHAR(200) NOT NULL,
  role        ENUM('admin','mess','student') NOT NULL DEFAULT 'student',
  course      VARCHAR(80) NOT NULL,
  study_year  TINYINT NOT NULL DEFAULT 1,
  guardian_name VARCHAR(100),
  guardian_phone VARCHAR(15),
  status      ENUM('active','pending','vacated') NOT NULL DEFAULT 'active',
  joined_date DATE NOT NULL,
  CONSTRAINT chk_phone CHECK (phone REGEXP '^[0-9]{10}$'),
  CONSTRAINT chk_year CHECK (study_year BETWEEN 0 AND 6)
) ENGINE=InnoDB;

CREATE TABLE rooms (
  room_no     VARCHAR(10) PRIMARY KEY,
  block       CHAR(1) NOT NULL,
  floor_no    TINYINT NOT NULL,
  capacity    TINYINT NOT NULL,
  room_type   VARCHAR(20) NOT NULL DEFAULT 'Standard',
  under_maintenance BOOLEAN NOT NULL DEFAULT FALSE,
  CONSTRAINT chk_capacity CHECK (capacity BETWEEN 1 AND 6)
) ENGINE=InnoDB;

CREATE TABLE allocations (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  student_id  VARCHAR(20) NOT NULL,
  room_no     VARCHAR(10) NOT NULL,
  allocated_on DATE NOT NULL,
  vacated_on  DATE NULL,
  status      ENUM('Active','Vacated') NOT NULL DEFAULT 'Active',
  -- generated column: only one ACTIVE allocation per student (NULL values never collide in a UNIQUE
  index)
  active_student VARCHAR(20) AS (CASE WHEN status = 'Active' THEN student_id END) STORED,
  UNIQUE KEY uq_one_active_room (active_student),
  KEY idx_alloc_room (room_no, status),
  CONSTRAINT fk_alloc_student FOREIGN KEY (student_id) REFERENCES users(id),
  CONSTRAINT fk_alloc_room FOREIGN KEY (room_no) REFERENCES rooms(room_no)
) ENGINE=InnoDB;

--- ----- Fees (fee-service)
CREATE TABLE fees (
  id          VARCHAR(30) PRIMARY KEY,
  student_id  VARCHAR(20) NOT NULL,
  fee_type    ENUM('Hostel Rent','Mess Charges','Security Deposit','Electricity','Fine','Other') NOT
  NULL,
  amount      DECIMAL(10,2) NOT NULL,
  term        VARCHAR(20) NOT NULL,
  due_date    DATE NOT NULL,
  status      ENUM('pending','paid') NOT NULL DEFAULT 'pending',
  payment_date DATETIME NULL,
  receipt_no  VARCHAR(40) NULL,
  notes       VARCHAR(200),
  UNIQUE KEY uq_fee_once (student_id, fee_type, term),
  KEY idx_fee_status_due (status, due_date),
  CONSTRAINT chk_amount CHECK (amount > 0),
  CONSTRAINT fk_fee_student FOREIGN KEY (student_id) REFERENCES users(id)
) ENGINE=InnoDB;

CREATE TABLE payments (
  id          VARCHAR(30) PRIMARY KEY,
  fee_id      VARCHAR(30) NOT NULL UNIQUE,
  student_id  VARCHAR(20) NOT NULL,
  amount      DECIMAL(10,2) NOT NULL,
  method      ENUM('UPI','Cash','Card','Bank Transfer') NOT NULL,
  payment_date DATETIME NOT NULL,
  receipt_no  VARCHAR(40) NOT NULL UNIQUE,
  recorded_by VARCHAR(20) NOT NULL,
  CONSTRAINT fk_pay_fee FOREIGN KEY (fee_id) REFERENCES fees(id),
  CONSTRAINT fk_pay_student FOREIGN KEY (student_id) REFERENCES users(id),
  CONSTRAINT fk_pay_by FOREIGN KEY (recorded_by) REFERENCES users(id)
) ENGINE=InnoDB;

--- ----- Attendance and leave (attendance-
service)
CREATE TABLE attendance (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  student_id  VARCHAR(20) NOT NULL,
  att_date    DATE NOT NULL,
  status      ENUM('Present','Absent','Late','On Leave') NOT NULL,
  remark      VARCHAR(200),
  marked_by   VARCHAR(20),
  UNIQUE KEY uq_att_day (student_id, att_date),

```

```

KEY idx_att_date (att_date),
CONSTRAINT fk_att_student FOREIGN KEY (student_id) REFERENCES users(id)
) ENGINE=InnoDB;

CREATE TABLE leave_requests (
  id INT AUTO_INCREMENT PRIMARY KEY,
  student_id VARCHAR(20) NOT NULL,
  from_date DATE NOT NULL,
  to_date DATE NOT NULL,
  reason VARCHAR(200) NOT NULL,
  destination VARCHAR(100),
  status ENUM('Pending','Approved','Rejected','Cancelled') NOT NULL DEFAULT 'Pending',
  reviewed_by VARCHAR(20),
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_leave_dates CHECK (to_date >= from_date),
  CONSTRAINT fk_leave_student FOREIGN KEY (student_id) REFERENCES users(id)
) ENGINE=InnoDB;

----- Mess (mess-service)
CREATE TABLE mess_menu (
  id INT AUTO_INCREMENT PRIMARY KEY,
  day_name ENUM('Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday') NOT NULL,
  meal_type ENUM('Breakfast','Lunch','Snacks','Dinner') NOT NULL,
  items VARCHAR(250) NOT NULL,
  is_veg BOOLEAN NOT NULL DEFAULT TRUE,
  UNIQUE KEY uq_menu_slot (day_name, meal_type)
) ENGINE=InnoDB;

CREATE TABLE meal_registrations (
  id INT AUTO_INCREMENT PRIMARY KEY,
  student_id VARCHAR(20) NOT NULL,
  meal_date DATE NOT NULL,
  meal_type ENUM('Breakfast','Lunch','Snacks','Dinner') NOT NULL,
  status ENUM('Registered','Cancelled') NOT NULL DEFAULT 'Registered',
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_meal_reg (student_id, meal_date, meal_type),
  KEY idx_meal_demand (meal_date, meal_type, status),
  CONSTRAINT fk_reg_student FOREIGN KEY (student_id) REFERENCES users(id)
) ENGINE=InnoDB;

CREATE TABLE meal_attendance (
  id INT AUTO_INCREMENT PRIMARY KEY,
  student_id VARCHAR(20) NOT NULL,
  meal_date DATE NOT NULL,
  meal_type ENUM('Breakfast','Lunch','Snacks','Dinner') NOT NULL,
  served_by VARCHAR(20),
  served_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_meal_served (student_id, meal_date, meal_type),
  CONSTRAINT fk_meal_student FOREIGN KEY (student_id) REFERENCES users(id)
) ENGINE=InnoDB;

----- Complaints and notices
CREATE TABLE complaints (
  id INT AUTO_INCREMENT PRIMARY KEY,
  student_id VARCHAR(20) NOT NULL,
  category ENUM('Room','Mess','Water','Electricity','Cleaning','Internet','Security','Other') NOT NULL,
  description TEXT NOT NULL,
  priority ENUM('Low','Medium','High') NOT NULL DEFAULT 'Medium',
  status ENUM('Pending','Assigned','In Progress','Resolved') NOT NULL DEFAULT 'Pending',
  assigned_to VARCHAR(80),
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  resolved_at DATETIME NULL,
  KEY idx_complaint_status (status, priority),
  CONSTRAINT fk_complaint_student FOREIGN KEY (student_id) REFERENCES users(id)
) ENGINE=InnoDB;

CREATE TABLE complaint_activity (
  id INT AUTO_INCREMENT PRIMARY KEY,
  complaint_id INT NOT NULL,
  message VARCHAR(250) NOT NULL,
  at_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_activity_complaint FOREIGN KEY (complaint_id) REFERENCES complaints(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE notices (
  id INT AUTO_INCREMENT PRIMARY KEY,
  admin_id VARCHAR(20) NOT NULL,
  title VARCHAR(150) NOT NULL,
  description TEXT NOT NULL,
  category VARCHAR(30) NOT NULL DEFAULT 'General',
  priority ENUM('Low','Medium','High') NOT NULL DEFAULT 'Medium',
  audience VARCHAR(40) NOT NULL DEFAULT 'All Students',
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  expires_on DATE NULL,
  CONSTRAINT fk_notice_admin FOREIGN KEY (admin_id) REFERENCES users(id)
) ENGINE=InnoDB;

CREATE TABLE notifications (
  id INT AUTO_INCREMENT PRIMARY KEY,
  student_id VARCHAR(20) NOT NULL,
  title VARCHAR(120) NOT NULL,
  message VARCHAR(250) NOT NULL,
  channel VARCHAR(20) NOT NULL DEFAULT 'In App',
  is_read BOOLEAN NOT NULL DEFAULT FALSE,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_notification_student FOREIGN KEY (student_id) REFERENCES users(id)
) ENGINE=InnoDB;

----- Views for dashboards and reports
CREATE VIEW v_room_occupancy AS
SELECT r.room_no, r.block, r.capacity, r.under_maintenance,
COUNT(a.id) AS occupied,
r.capacity - COUNT(a.id) AS vacant,
CASE WHEN r.under_maintenance THEN 'Maintenance'
WHEN COUNT(a.id) >= r.capacity THEN 'Full' ELSE 'Available' END AS room_status
FROM rooms r
LEFT JOIN allocations a ON a.room_no = r.room_no AND a.status = 'Active'
GROUP BY r.room_no, r.block, r.capacity, r.under_maintenance;

CREATE VIEW v_pending_fees AS
SELECT f.id, u.roll_no, u.name, f.fee_type, f.amount, f.term, f.due_date,
CASE WHEN f.due_date < CURDATE() THEN 'overdue' ELSE 'pending' END AS effective_status
FROM fees f JOIN users u ON u.id = f.student_id
WHERE f.status = 'pending';
CREATE VIEW v_attendance_summary AS

```

```

SELECT u.id AS student_id, u.roll_no, u.name,
       SUM(a.status = 'Present') AS present_days,
       SUM(a.status = 'Late') AS late_days,
       SUM(a.status = 'Absent') AS absent_days,
       ROUND(100 * SUM(a.status IN ('Present','Late')) / NULLIF(SUM(a.status <> 'On Leave'), 0), 1) AS
       attendance_pct
FROM users u JOIN attendance a ON a.student_id = u.id
GROUP BY u.id, u.roll_no, u.name;

CREATE VIEW v_meal_demand AS
SELECT meal_date, meal_type, COUNT(*) AS registered
FROM meal_registrations WHERE status = 'Registered'
GROUP BY meal_date, meal_type;

--- ..... Business rules in the database
DELIMITER //

-- Room capacity guard: reject an allocation that would over-fill a room or use a room under maintenance.
CREATE TRIGGER trg_allocation_capacity BEFORE INSERT ON allocations
FOR EACH ROW
BEGIN
    DECLARE v_capacity INT;
    DECLARE v_maint BOOLEAN;
    DECLARE v_occupied INT;

    SELECT capacity, under_maintenance INTO v_capacity, v_maint FROM rooms WHERE room_no = NEW.room_no FOR
    UPDATE;
    SELECT COUNT(*) INTO v_occupied FROM allocations WHERE room_no = NEW.room_no AND status = 'Active';

    IF v_maint THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Room is under maintenance';
    ELSEIF NEW.status = 'Active' AND v_occupied >= v_capacity THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Room is full';
    END IF;
END//

-- Attendance cannot be recorded for a future date.
CREATE TRIGGER trg_attendance_no_future BEFORE INSERT ON attendance
FOR EACH ROW
BEGIN
    IF NEW.att_date > CURDATE() THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Attendance cannot be marked for a future date';
    END IF;
END//

-- Fee payment as one atomic transaction: create the receipt and mark the fee paid, or change nothing.
CREATE PROCEDURE sp_record_payment(IN p_fee_id VARCHAR(30), IN p_method VARCHAR(20), IN p_admin
    VARCHAR(20), OUT p_receipt VARCHAR(40))
BEGIN
    DECLARE v_student VARCHAR(20);
    DECLARE v_amount DECIMAL(10,2);
    DECLARE v_status VARCHAR(10);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;
    SELECT student_id, amount, status INTO v_student, v_amount, v_status FROM fees WHERE id = p_fee_id FOR
    UPDATE;

    IF v_status IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Fee record not found';
    ELSEIF v_status = 'paid' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Fee is already paid';
    END IF;

    SET p_receipt = CONCAT('RCP-', DATE_FORMAT(NOW(), '%Y%m'), '-', LPAD(FLOOR(RAND() * 100000), 5, '0'));
    INSERT INTO payments (id, fee_id, student_id, amount, method, payment_date, receipt_no, recorded_by)
    VALUES (CONCAT('pay-', UUID_SHORT()), p_fee_id, v_student, v_amount, p_method, NOW(), p_receipt,
        p_admin);
    UPDATE fees SET status = 'paid', payment_date = NOW(), receipt_no = p_receipt WHERE id = p_fee_id;
    COMMIT;
END//

DELIMITER ;

--- ..... Sample report queries
-- Students without a room:
--   SELECT u.roll_no, u.name FROM users u LEFT JOIN allocations a ON a.student_id = u.id AND a.status =
--   'Active'
--   WHERE u.role = 'student' AND u.status = 'active' AND a.id IS NULL;
-- Food requirement for tomorrow:
--   SELECT meal_type, registered FROM v_meal_demand WHERE meal_date = CURDATE() + INTERVAL 1 DAY;
-- Students with attendance below 75%:
--   SELECT * FROM v_attendance_summary WHERE attendance_pct < 75;

```

## hostel/frontend/app.js

```

(function () {
    const root = document.getElementById("app-root");
    const toastRegion = document.getElementById("toast-region");

    const credentials = {
        admin: ["admin@hostel.test", "admin123"],
        mess: ["mess@hostel.test", "mess123"],
        student: ["aarav@student.test", "student123"]
    };

    const allViews = {
        dashboard: ["Dashboard", "home", "Live summary across all hostel and mess services."],
        students: ["Students", "users", "Student registration, profiles, and guardian details."],
        rooms: ["Rooms", "bed", "Room availability, occupancy, and allocation."],
        fees: ["Fees", "credit", "Hostel fee ledger, pending dues, and receipts."],
        attendance: ["Attendance", "clipboard", "Daily attendance and leave requests."],
        mess: ["Mess", "utensils", "Weekly menu, meal registration, and demand."],
        complaints: ["Complaints", "alert", "Issue tracking from submission to resolution."],
        notices: ["Notices", "bell", "Announcements and personal notifications."],
        architecture: ["Architecture", "database", "Services, database ownership, and access control."]
    };
})();

```

```

const menus = {
  admin: ["dashboard", "students", "rooms", "fees", "attendance", "mess", "complaints", "notices",
    "architecture"],
  mess: ["dashboard", "mess", "complaints", "notices"],
  student: ["dashboard", "fees", "attendance", "mess", "complaints", "notices"]
};

const mealTypes = ["Breakfast", "Lunch", "Snacks", "Dinner"];
const weekDays = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"];
const statusColors = { good: ["paid", "approved", "resolved", "available", "active", "present",
  "registered", "served"], warn: ["pending", "assigned", "late", "maintenance", "on leave"], bad:
  ["overdue", "rejected", "absent", "full", "cancelled", "vacated", "unavailable"], info: ["in
  progress", "ok"] };

const emptyData = () => ({
  students: [], rooms: [], allocations: [], fees: [], payments: [], attendance: [], leaves: [],
  menu: [], registrations: [], mealAttendance: [], complaints: [], notices: [], notifications: [],
  demand: {}, health: null, overview: null
});

const state = {
  token: localStorage.getItem("hostel-token") || "",
  user: JSON.parse(localStorage.getItem("hostel-user")) || "null",
  view: "dashboard",
  loading: false,
  query: "",
  attDate: new Date().toISOString().slice(0, 10),
  attDraft: {},
  data: emptyData()
};

const iconPaths = {
  home: '<path d="m3 10.8 9-7.2 9 7.2"/><path d="M5 10v10h14V10"/><path d="M9 20v-6h6v6"/>',
  users: '<path d="M16 21v-2a4 4 0 0 0-4-4h6a4 4 0 0 0-4 4v2"/><circle cx="9" cy="7" r="4"/><path
  d="M22 21v-2a4 4 0 0 0-3-3.8"/><path d="M16 3.2a4 4 0 0 1 0 7.6"/>',
  credit: '<rect x="2" y="5" width="20" height="14" rx="2"/><path d="M2 10h20"/><path d="M6 15h4"/>',
  alert: '<path d="M10.3 3.9 1.8 18a2 2 0 0 1 1.7 3h17a2 2 0 0 1 1.7-3L13.7 3.9a2 2 0 0 0-3.4
  0z"/><path d="M12 9v4"/><path d="M12 17h.01"/>',
  bed: '<path d="M2 20v-8a2 2 0 0 1 2-2h16a2 2 0 0 1 2 2v8"/><path d="M2 16h20"/><path d="M6 10V6a2 2 0
  1 2-2h8a2 2 0 0 1 2 2v4"/>',
  utensils: '<path d="M3 2v7c0 1.1 9 2 2 2h2a2 2 0 0 0 2-2V2"/><path d="M6 2v20"/><path d="M19 2c-2.2
  1.3-3 3.6-3 6.5V14h3v8"/>',
  clipboard: '<rect x="8" y="2" width="8" height="4" rx="1"/><path d="M16 4h2a2 2 0 0 1 2 2v14a2 2 0 0
  1-2 2H6a2 2 0 0 1-2-2V6a2 2 0 0 1 2-2h2"/><path d="m9 14 2 2 4-4"/>',
  bell: '<path d="M18 8a6 6 0 0 0-12 0c 7-3 9h18c0-2-3-2-3-9"/><path d="M13.7 21a2 2 0 0 1-3.4
  0"/>',
  database: '<ellipse cx="12" cy="5" rx="9" ry="3"/><path d="M3 5v14c0 1.7 4 3 9 3s9-1.3 9-3V5"/><path
  d="M3 12c0 1.7 4 3 9 3s9-1.3 9-3"/>',
  shield: '<path d="M12 22s8-4 8-10V5l-8-3-8 3v7c0 6 8 10 8 10z"/><path d="M9 12l2 2 4-4"/>',
  refresh: '<path d="M21 12a9 9 0 0 1-15.2 6.5"/><path d="M3 12A9 9 0 0 1 18.2 5.5"/><path d="M3
  3v6h6"/><path d="M21 21v-6h-6"/>',
  logout: '<path d="M9 21H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h4"/><path d="M16 17l5-5-5-5"/><path d="M21
  12H9"/>',
  plus: '<path d="M12 5v14"/><path d="M5 12h14"/>',
  check: '<path d="M20 6 9 17l-5-5"/>',
  x: '<path d="M18 6 6 18"/><path d="m6 6 12 12"/>'
};

const icon = (name) => `<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round" aria-
hidden="true">${iconPaths[name]}</svg>`;
const esc = (value) => String(value ?? "").replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g,
"&gt;").replace(/"/g, "&quot;").replace(/'/g, "&#039;");
const money = (value) => new Intl.NumberFormat("en-IN", { style: "currency", currency: "INR",
  maximumFractionDigits: 0 }).format(Number(value || 0));
const today = () => new Date().toISOString().slice(0, 10);
const addDays = (n) => new Date(Date.now() + n * 86400000).toISOString().slice(0, 10);
const dayName = (date) => new Date(`${date}T00:00:00Z`).toLocaleDateString("en-US", { weekday: "long",
  timeZone: "UTC" });
const dateOnly = (value) => value ? new Intl.DateTimeFormat("en-IN", { day: "2-digit", month: "short",
  year: "numeric", timeZone: "UTC" }).format(new Date(value)) : "Not set";
const dateTime = (value) => value ? new Intl.DateTimeFormat("en-IN", { day: "2-digit", month: "short",
  hour: "2-digit", minute: "2-digit" }).format(new Date(value)) : "Not recorded";
const role = () => (state.user ? state.user.role : "");
const isStaff = () => role() === "admin" || role() === "mess";

function statusPill(value) {
  const key = String(value || "").toLowerCase();
  const tone = Object.keys(statusColors).find((name) => statusColors[name].includes(key)) || "neutral";
  return `<span class="status ${tone}">${esc(value || "Unknown")}</span>`;
}

function toast(message) {
  const item = document.createElement("div");
  item.className = "toast";
  item.textContent = message;
  toastRegion.appendChild(item);
  setTimeout(() => item.remove(), 3500);
}

async function api(endpoint, options = {}) {
  const headers = {
    Accept: "application/json",
    ...(options.body ? { "Content-Type": "application/json" } : {}),
    ...(options.auth === false || !state.token ? {} : { Authorization: `Bearer ${state.token}` })
  };
  const response = await fetch(endpoint, { method: options.method || "GET", headers, body: options.body
    ? JSON.stringify(options.body) : undefined });
  const payload = await response.json().catch(() => null);

  if (!response.ok) {
    throw new Error((payload && payload.error) || `Request failed with status ${response.status}`);
  }

  return payload;
}

const formData = (form) => Object.fromEntries(new FormData(form).entries());
const options = (list, value, label, selected) => list.map((item) => `<option
  value="${esc(value(item))}" ${value(item) === selected ? "selected" :
  ""}>${esc(label(item))}</option>`).join("");
const studentName = (id) => (const s = state.data.students.find((item) => item.id === id); return s ?
`${s.name} (${s.rollNo})` : id);
const activeStudents = () => state.data.students.filter((s) => s.role === "student" && s.status ===
"active");
const table = (inner) => `<div class="table-scroll"><table class="data-table">${inner}</table></div>`;

```

```

const empty = (message) => `<div class="empty-state">${esc(message)}</div>`;

function kpi(label, value, iconName, tone) {
  return `<article class="kpi"><div class="kpi-icon" style="${tone} ?
  `background:${tone[0]};color:${tone[1]}` : """>${icon(iconName)}</div><div><strong>${esc(value)}</st
  rong><span>${esc(label)}</span></div></article>`;
}

const AMBER = ["#ffffd6", "var(--amber)"];
const CORAL = ["#ffe2dc", "var(--coral)"];
const VIOLET = ["#e5e7ff", "var(--violet)"];
const GREEN = ["#e8f4dd", "var(--green)"];

async function loadData() {
  state.loading = true;
  render();

  const r = role();
  const plan = {
    students: ["/api/students", true],
    rooms: ["/api/rooms", r === "admin"],
    allocations: ["/api/allocations", r !== "mess"],
    fees: ["/api/fees", r !== "mess"],
    payments: ["/api/payments", r !== "mess"],
    attendance: ["/api/attendance", r !== "mess"],
    leaves: ["/api/leaves", r !== "mess"],
    menu: ["/api/menu", true],
    registrations: ["/api/registrations", true],
    mealAttendance: ["/api/meal-attendance", isStaff()],
    complaints: ["/api/complaints", true],
    notices: ["/api/notices", true],
    notifications: ["/api/notifications", r !== "mess"],
    overview: ["/api/overview", isStaff()],
    health: ["/api/service-status", r === "admin"],
    demandToday: ["/api/demand?date=${today()}", isStaff()],
    demandTomorrow: ["/api/demand?date=${addDays(1)}", isStaff()]
  };

  const keys = Object.keys(plan).filter((key) => plan[key][1]);
  const results = await Promise.allSettled(keys.map((key) => api(plan[key][0])));
  const data = emptyData();
  const failures = [];

  results.forEach((result, index) => {
    if (result.status === "fulfilled") {
      data[keys[index]] = result.value;
    } else {
      failures.push(result.reason.message);
    }
  });

  state.data = data;
  state.loading = false;

  if (failures.some((message) => message.includes("Authentication"))) {
    logout(false);
    return;
  }

  if (failures.length) {
    toast(`Some data could not be loaded: ${failures[0]}`);
  }

  render();
}

function renderLogin() {
  root.innerHTML = `
    <main class="login-screen">
      <section class="login-visual">
        <div class="brand-lockup">
          <div class="brand-mark">${icon("bed")}</div>
          <h1>Smart Hostel & Mess Console</h1>
          <p>Student records, room allocation, fees, attendance, mess planning, complaints, and notices
          in one integrated administration system.</p>
        </div>
        <div class="system-map" aria-label="Service map">
          <div class="map-row"><div class="map-node"><strong>Gateway</strong><span>Port
          4000</span></div><div class="map-node"><strong>Auth + RBAC</strong><span>Signed
          token</span></div><div class="map-node"><strong>Frontend</strong><span>Web
          dashboard</span></div></div>
          <div class="map-row"><div class="map-node"><strong>Students &
          Rooms</strong><span>DB-1</span></div><div class="map-
          node"><strong>Fees</strong><span>DB-2</span></div><div class="map-
          node"><strong>Complaints</strong><span>DB-3</span></div></div>
          <div class="map-row"><div class="map-
          node"><strong>Attendance</strong><span>DB-4</span></div><div class="map-
          node"><strong>Mess</strong><span>DB-5</span></div><div class="map-
          node"><strong>Notices</strong><span>DB-6</span></div></div>
        </div>
      </section>
      <section class="login-panel">
        <form class="auth-box" id="login-form">
          <h2>Sign in</h2>
          <p>Use a demo role or enter seeded credentials.</p>
          <div class="preset-grid">
            <button type="button" class="preset-btn" data-login-preset="admin">${icon("database")}
            Admin</button>
            <button type="button" class="preset-btn" data-login-preset="mess">${icon("utensils")} Mess
            Staff</button>
            <button type="button" class="preset-btn" data-login-preset="student">${icon("users")}
            Student</button>
          </div>
          <div class="form-grid">
            <div class="field full"><label for="email">Email</label><input id="email" name="email"
            autocomplete="username" value="admin@hostel.test" required></div>
            <div class="field full"><label for="password">Password</label><input id="password"
            name="password" type="password" autocomplete="current-password" value="admin123" required></div>
          </div>
          <p class="auth-error" id="auth-error"></p>
          <button class="primary-btn" type="submit">${icon("check")} Sign in</button>
        </form>
      </section>
    </main>`;
}

function renderShell() {

```

```

const menu = menus[role()] || menus.student;
if (!menu.includes(state.view)) {
  state.view = "dashboard";
}
const current = allViews[state.view];

root.innerHTML = `
  <div class="app-shell">
    <aside class="sidebar">
      <div class="sidebar-brand"><div class="brand-mark">${icon("bed")}</div><div><span>Smart
Hostel</span><strong>Mess Console</strong></div></div>
      <nav class="nav" aria-label="Primary">
        ${menu.map((id) => `<button class="nav-btn ${state.view === id ? "active" : ""}>
nav="${id}" title="${esc(allViews[id][0])}">${icon(allViews[id][1])}</button>`)}
      </nav>
      <div class="sidebar-footer">
        <div class="user-
chip"><strong>${esc(state.user.name)}</strong><span>${esc(state.user.role)}</span></div>
Room ${esc(state.user.roomNo)} : ""</span></div>
        <button class="ghost-btn" data-logout>${icon("logout")} Sign out</button>
      </div>
    </aside>
    <main class="main-area">
      <header class="topbar">
        <div class="view-title"><h2>${esc(current[0])}</h2><p>${esc(current[2])}</p></div>
        <div class="topbar-actions">
          <button class="icon-btn" title="Refresh data" data-refresh>${icon("refresh")}</button>
          <button class="icon-btn mobile-nav" title="Sign out" data-logout>${icon("logout")}</button>
        </div>
      </header>
      <div class="content">${state.loading ? `<section class="band"><div class="empty-state">Loading
live data from the gateway and services...</div></section>` : currentView()}</div>
    </main>
  </div>`;

function currentView() {
  const map = { dashboard: dashboardView, students: studentsView, rooms: roomsView, fees: feesView,
attendance: attendanceView, mess: messView, complaints: complaintsView, notices: noticesView,
architecture: architectureView };
  return map[state.view]();
}

const meters = (rows, total) => `<div class="meter-list">${rows.map(([label, count]) => `<div
class="meter-row"><span class="small">${esc(label)}</span><div class="meter"><span
style="width:${Math.max(3, Math.round((count / Math.max(1, total)) *
100))}%></span></div><strong>${count}</strong></div>`)}</div>`;

function noticeCard(notice) {
  return `<article class="item-card"><div class="item-
head"><strong>${esc(notice.title)}</strong><div>${statusFill(notice.priority)}</div><span class="small
muted">${esc(notice.category)} - ${dateOnly(notice.createdAt)}</span><div>${notice.expiresOn ? " - until
${dateOnly(notice.expiresOn)}" : ""}</span><p class="small"
style="margin:0">${esc(notice.description)}</p></article>`;
}

function menuFor(date) {
  const day = dayName(date);
  return mealTypes.map((meal) => [meal, state.data.menu.find((item) => item.day === day &&
item.mealType === meal)]);
}

const menuList = (date) => `<div class="list">${menuFor(date).map(([meal, entry]) => `<div class="item-
card"><div class="item-head"><strong>${meal}</strong><div>${entry ? statusFill(entry.isVeg ? "Veg" : "Non-
veg") : ""}</div><span class="small">${entry ? esc(entry.items) : "Not
scheduled"}</span></div>`)}</div>`;

function demandPanel(demand, title) {
  const meals = (demand && demand.meals) || {};
  const total = Math.max(1, ...Object.values(meals).map((m) => m.registered));
  return `<div class="panel"><div class="panel-title"><h3>${esc(title)}</h3><span>${esc(demand &&
demand.day ? ` ${demand.day}, ${dateOnly(demand.date)}` :
"")}</span></div>${meters(mealTypes.map((meal) => [`${meal} (served ${meals[meal] ?
meals[meal].attended : 0})`, meals[meal] ? meals[meal].registered : 0]), total)}</div>`;
}

function dashboardView() {
  const d = state.data;
  const s = d.overview ? d.overview.summary : {};
  const openComplaints = d.complaints.filter((c) => c.status !== "Resolved");
  const flow = ["Pending", "Assigned", "In Progress", "Resolved"].map((status) => [status,
d.complaints.filter((c) => c.status === status).length]);
  const notices = `<div class="panel"><div class="panel-title"><h3>Latest
Notices</h3><span>${d.notices.length} published</span></div><div class="list">${d.notices.slice(0,
3).map(noticeCard).join("")} || empty("No notices")</div></div>`;
  const complaintPanel = `<div class="panel"><div class="panel-title"><h3>Complaint
Flow</h3><span>${openComplaints.length} open</span></div>${meters(flow, d.complaints.length)}</div>`;

  if (role() === "student") {
    const me = d.students[0] || state.user;
    const unpaid = d.fees.filter((f) => f.status !== "paid");
    const marked = d.attendance.filter((a) => a.status !== "On Leave");
    const percent = marked.length ? Math.round((marked.filter((a) => a.status !== "Absent").length /
marked.length) * 100) : 100;
    const mine = d.registrations.filter((r) => r.status === "Registered" && r.date >= today()).slice(0,
5);
    return `
      <section class="kpi-grid">
        ${kpi("My room", me.roomNo || "Not allocated", "bed")}
        ${kpi("Pending fees", money(unpaid.reduce((sum, f) => sum + f.amount, 0)), "credit", AMBER)}
        ${kpi("Attendance", `${percent}%`, "clipboard", GREEN)}
        ${kpi("Open complaints", openComplaints.length, "alert", CORAL)}
      </section>
      <section class="grid-2">
        <div class="panel"><div class="panel-title"><h3>Today's
Menu</h3><span>${dayName(today())}</span></div>${menuList(today())}</div>
        <div class="panel"><div class="panel-title"><h3>My Upcoming Meals</h3><span>${mine.length}
registered</span></div>${mine.length ?
table(`<thead><tr><th>Date</th><th>Meal</th></tr></thead><tbody>${mine.map((r) =>
`<tr><td>${dateOnly(r.date)}</td><td>${esc(r.mealType)}</td></tr>`)}</tbody>`) : empty("No
upcoming meal registrations")}</div>
      </section>
      <section class="grid-2">${notices}<div class="panel"><div class="panel-
title"><h3>Notifications</h3><span>${d.notifications.filter((n) => !n.read).length}
unread</span></div><div class="list">${d.notifications.slice(0, 4).map(notificationCard).join("")} ||

```

```

empty("No notifications"))}</div></div></section>`;
}

const kpis = role() === "admin"
  ? `${kpi("Students", s.students || 0, "users")}${kpi("Bed occupancy", `${s.occupancyRate || 0}%`
    `${s.occupiedBeds || 0}/${s.totalBeds || 0})`, "bed", VIOLET)}${kpi("Pending fees",
    money(s.pendingFeeAmount || 0), "credit", AMBER)}${kpi("Open complaints", s.openComplaints || 0,
    "alert", CORAL)}${kpi("Present today", s.presentToday || 0, "clipboard")}${kpi("Meals registered
    today", s.mealsRegisteredToday || 0, "utensils", GREEN)}`
  : `${kpi("Meals registered today", s.mealsRegisteredToday || 0, "utensils", GREEN)}${kpi("Meals
    served today", d.overview?.stats?.mess?.servedToday || 0, "check")}${kpi("Menu slots",
    d.overview?.stats?.mess?.menuItems || 0, "clipboard", VIOLET)}${kpi("Open complaints",
    openComplaints.length, "alert", CORAL)}`;

const health = role() === "admin" ? `
  <section class="band"><div class="panel-title"><h3>Service Health</h3><span>${d.overview ? `Updated
    ${dateTime(d.overview.generatedAt)}` : ""}</span></div>
    <div class="service-grid">${(d.health?.services || []).map((sv) => `<div class="service-
    tile"><strong>${esc(sv.name)}</strong>${statusPill(sv.ok ? "ok" :
    "unavailable")}</span>${esc(sv.url)}</span></div>`)}</div></section>` : "";

const dues = d.fees.filter((f) => f.status !== "paid").slice(0, 5);
const duesPanel = role() === "admin" ? `<div class="panel"><div class="panel-title"><h3>Pending
    Dues</h3><span>${s.studentsWithDues || 0} students</span></div>${dues.length ? table(`<thead><tr><th>
    Student</th><th>Fee</th><th>Amount</th><th>Status</th></tr></thead><tbody>${dues.map((f) => `<tr><td>
    ${esc(studentName(f.studentId))}</td><td>${esc(f.feeType)}</td><td>${money(f.amount)}</td><td>${statu
    sPill(f.status)}</td></tr>`)}</tbody>`)}</div>` : `<div
    class="panel"><div class="panel-title"><h3>Today's
    Menu</h3><span>${dayName(today())}</span></div>${menuList(today())}</div>`;

return `
  <section class="kpi-grid">${kpis}</section>${health}
  <section class="grid-2">${complaintPanel}${duesPanel}</section>
  <section class="grid-2">${demandPanel(d.demandToday, "Meal Demand -
    Today")}${demandPanel(d.demandTomorrow, "Meal Demand - Tomorrow")}</section>
  <section class="grid-2">${notices}${role() === "admin" ? `<div class="panel"><div class="panel-
    title"><h3>Leave Requests</h3><span>${s.pendingLeaves || 0}
    pending</span></div>${leaveTable(d.leaves.filter((l) => l.status === "Pending").slice(0, 4),
    true)}</div>` : ""}</section>`;
}

function notificationCard(n) {
  return `<article class="item-card"><div class="item-head"><strong>${esc(n.title)}</strong>${n.read ?
    statusPill("Read") : `<button class="ghost-btn" data-read="${esc(n.id)}">${icon("check")}`} Mark
    read</button>`}</div><span class="small">${esc(n.message)}</span><span class="small
    muted">${dateTime(n.createdAt)}</span></article>`;
}

function studentsView() {
  const q = state.query.toLowerCase();
  const list = state.data.students.filter((s) => !q || `${s.name} ${s.rollNo} ${s.email}
    ${s.course}`.toLowerCase().includes(q));
  return `
    <section class="grid-2">
      <form class="panel" id="student-form">
        <div class="panel-title"><h3>Register Student</h3><span>${state.data.students.filter((s) =>
          s.status === "pending").length} pending</span></div>
        <div class="form-grid">
          <div class="field"><label>Full name</label><input name="name" required</div>
          <div class="field"><label>Roll number</label><input name="rollNo" required</div>
          <div class="field"><label>Phone (10 digits)</label><input name="phone" pattern="\d{10}"
            required</div>
          <div class="field"><label>Email</label><input name="email" type="email" required</div>
          <div class="field"><label>Course</label><input name="course" value="B.Tech CSE"
            required</div>
          <div class="field"><label>Year</label><input name="year" type="number" min="1" max="6"
            value="1"></div>
          <div class="field"><label>Guardian name</label><input name="guardianName"></div>
          <div class="field"><label>Guardian phone</label><input name="guardianPhone"></div>
          <div class="field"><label>Status</label><select
            name="status"><option>active</option><option>pending</option></select></div>
        </div>
        <div class="row-actions" style="margin-top:14px"><button class="primary-btn"
          type="submit">${icon("plus")} Register</button></div>
      </form>
      <div class="panel"><div class="panel-title"><h3>Overview</h3><span>${state.data.students.length}
        students</span></div>
        <div class="kpi-grid" style="grid-template-columns:repeat(2,minmax(120px,1fr))">
          ${kpi("Active", state.data.students.filter((s) => s.status === "active").length, "users")}
          ${kpi("Without room", state.data.students.filter((s) => s.status === "active" &&
            !s.roomNo).length, "bed", AMBER)}
        </div>
        <p class="small muted">New students receive the default password <strong>student123</strong>
        (stored hashed).</p>
      </div>
    </section>
    <section class="panel">
      <div class="panel-title"><h3>Student Directory</h3><input id="student-search" placeholder="Search
        name, roll no, course" value="${esc(state.query)}" style="max-width:260px"></div>
      <div id="students-table">${studentsTable(list)}</div>
    </section>`;
}

function studentsTable(list) {
  return table(`<thead><tr><th>Roll no</th><th>Name</th><th>Course</th><th>Room</th><th>Guardian</th><th>
    Status</th><th>Action</th></tr></thead><tbody>${list.map((s) => `
    <tr><td>${esc(s.rollNo)}</td><td><strong>${esc(s.name)}</strong><br><span class="small
    muted">${esc(s.email)} / ${esc(s.phone)}</span></td><td>${esc(s.course)} (Y${s.year})</td>
    <td>${s.roomNo ? esc(s.roomNo) : `<span
    class="muted">`}</span>`}</td><td>${esc(s.guardianName)}<br><span class="small
    muted">${esc(s.guardianPhone)}</span></td><td>${statusPill(s.status)}</td>
    <td>${s.status === "pending" ? `<button class="primary-btn" data-student-status="${esc(s.id)}"
    data-value="active">${icon("check")} Approve</button>` : s.status === "active" ? `<button
    class="ghost-btn" data-student-status="${esc(s.id)}" data-value="vacated">Mark vacated</button>` :
    ""}</td></tr>`)}</tbody>`;
}

function roomsView() {
  const d = state.data;
  const unallocated = activeStudents().filter((s) => !s.roomNo);
  const open = d.rooms.filter((r) => r.status === "Available");
  const current = d.allocations.filter((a) => a.status === "Active");
  return `
    <section class="grid-2">
      <form class="panel" id="allocation-form">
        <div class="panel-title"><h3>Allocate Room</h3><span>${unallocated.length} students

```

[illegible]

```

<\/><\/tr><\/tbody><\/>`'.join("")<\/tbody><\/>`;
}

function attendanceView() {
const d = state.data;
if (role() === "student") {
const counts = { Present: 0, Absent: 0, Late: 0, "On Leave": 0 };
d.attendance.forEach((a) => { counts[a.status] += 1 });
const considered = d.attendance.length - counts["On Leave"];
const percent = considered ? Math.round(((counts.Present + counts.Late) / considered) * 100) : 100;
return `
<section class="kpi-grid">${kpi("Attendance", `${percent}%`, "clipboard", GREEN)}${kpi("Present", counts.Present, "check")}${kpi("Late", counts.Late, "alert", AMBER)}${kpi("Absent", counts.Absent, "x", CORAL)}</section>
<section class="grid-2">
<form class="panel" id="leave-form"><div class="panel-title"><h3>Request Leave<\/h3><span>Warden approval required<\/span><\/div>
<div class="form-grid">
<div class="field"><label>From<\/label><input name="fromDate" type="date" min="${today()}" value="${today()}> required<\/div>
<div class="field"><label>To<\/label><input name="toDate" type="date" min="${today()}" value="${today()}> required<\/div>
<div class="field"><label>Destination<\/label><input name="destination"><\/div>
<div class="field"><label>Reason<\/label><input name="reason" required><\/div>
<\/div><div class="row-actions" style="margin-top:14px"><button class="primary-btn" type="submit">${icon("plus")} Submit request<\/button><\/div><\/form>
<div class="panel"><div class="panel-title"><h3>My Leave Requests<\/h3><span>${d.leaves.length}<\/span><\/div>${leaveTable(d.leaves, false)}<\/div>
<\/section>
<section class="panel"><div class="panel-title"><h3>Recent Attendance<\/h3><span>Last ${Math.min(14, d.attendance.length)} days<\/span><\/div>
Date
 Status | Remark |
<\/tr>
<\/thead>
${d.attendance.slice(0, 14).map((a) => `<tr><td>{dateOnly(a.date)}<\/td><td>{statusFill(a.status)}<\/td><td>{esc(a.remark)}<\/td><\/tr>`).join("")}<\/tbody>`<\/table>`;
}

const residents = activeStudents().filter((s) => s.roomNo);
const saved = Object.fromEntries(d.attendance.filter((a) => a.date === state.attDate).map((a) => [a.studentId, a.status]));
const pct = (id) => {
const rows = d.attendance.filter((a) => a.studentId === id && a.status !== "On Leave");
return rows.length ? Math.round((rows.filter((a) => a.status !== "Absent").length / rows.length) * 100) : 100;
};
return `
<section class="panel">
<div class="panel-title"><h3>Mark Attendance<\/h3><div class="row-actions"><input type="date" id="att-date" value="${state.attDate}" max="${today()}"><button class="primary-btn" data-save-attendance>${icon("check")} Save ${residents.length} records<\/button><\/div><\/div>
Student
 Room | Status | Overall |
<\/tr>
<\/thead>
${residents.map((s) => {
const value = state.attDraft[s.id] || saved[s.id] || "Present";
return `<tr><td>{esc(s.name)}<\/td><td>{small muted}>{esc(s.rollNo)}<\/td><td>{esc(s.roomNo)}<\/td><td>{select data-att=${esc(s.id)}>{"Present", "Absent", "Late", "On Leave"}.map(o) => `option {o === value ? "selected" : ""}>{o}<\/option>`.join("")}<\/td><td>{saved[s.id] ? "" : '<span class="small muted">not marked<\/span>'}`<\/td><td>{pct(s.id)}%<\/td><\/tr>`;
}).join("")}<\/tbody>`<\/table>
<\/section>
<section class="panel"><div class="panel-title"><h3>Leave Requests<\/h3><span>${d.leaves.filter((l) => l.status === "Pending").length} pending<\/span><\/div>${leaveTable(d.leaves, true)}<\/section>`;
}

function messView() {
const d = state.data;
const student = role() === "student";
const weekly = table(`<thead><tr><th>Day<\/th>${mealTypes.map((m) => `<th>{m}<\/th>`).join("")}<\/tr><\/thead><tbody>${weekDays.map((day) => `<tr><td><strong>{day}<\/strong><\/td>${mealTypes.map((m) => { const e = d.menu.find(x) => x.day === day && x.mealType === m; return `<td class="small">{e ? esc(e.items) : "-"}<\/td>`; }).join("")}<\/tr>`).join("")}<\/tbody>`;
const menuPanel = `<section class="panel"><div class="panel-title"><h3>Weekly Menu<\/h3><span>${d.menu.length} entries<\/span><\/div>${weekly}<\/section>`;

if (student) {
const mine = d.registrations.filter((r) => r.status === "Registered" && r.date >= today());
return `
<section class="grid-2">
<form class="panel" id="register-form"><div class="panel-title"><h3>Register for a Meal<\/h3><span>Helps the mess plan food<\/span><\/div>
<div class="form-grid">
<div class="field"><label>Date<\/label><input name="date" type="date" min="${today()}" value="${addDays(1)}> required<\/div>
<div class="field"><label>Meal<\/label><select name="mealType">${mealTypes.map((m) => `<option>{m}<\/option>`).join("")}<\/select><\/div>
<\/div><div class="row-actions" style="margin-top:14px"><button class="primary-btn" type="submit">${icon("plus")} Register<\/button><\/div><\/form>
<div class="panel"><div class="panel-title"><h3>My Registrations<\/h3><span>${mine.length} upcoming<\/span><\/div>
${mine.length ?
table(`<thead><tr><th>Date<\/th><th>Meal<\/th><th><\/th><\/tr><\/thead><tbody>${mine.map((r) => `<tr><td>{dateOnly(r.date)}<\/td><td>{esc(r.mealType)}<\/td><td><button class="ghost-btn" data-cancel-reg=${esc(r.id)}>Cancel<\/button><\/td><\/tr>`).join("")}<\/tbody>`: empty("No upcoming registrations")}<\/div>
<\/section>${menuPanel}`;
}

const served = d.mealAttendance.filter((m) => m.date === today());
return `
<section class="grid-2">
<form class="panel" id="menu-form"><div class="panel-title"><h3>Set Menu<\/h3><span>Creates or updates a slot<\/span><\/div>
<div class="form-grid">
<div class="field"><label>Day<\/label><select name="day">${weekDays.map((x) => `<option>{x}<\/option>`).join("")}<\/select><\/div>
<div class="field"><label>Meal<\/label><select name="mealType">${mealTypes.map((m) => `<option>{m}<\/option>').join("")}<\/select><\/div>
<div class="field full"><label>Items<\/label><input name="items" required><\/div>
<div class="field"><label>Type<\/label><select name="isVeg"><option value="true">Veg<\/option><option value="false">Non-veg<\/option><\/select><\/div>
<\/div><div class="row-actions" style="margin-top:14px"><button class="primary-btn" type="submit">${icon("check")} Save menu<\/button><\/div><\/form>
<form class="panel" id="served-form"><div class="panel-title"><h3>Record Meal Served<\/h3><span>${served.length} served today<\/span><\/div>
<div class="form-grid">
<div class="field"><label>Student<\/label><select name="studentId"

```

```

required>${options(activeStudents().filter((s) => s.roomNo), (s) => s.id, (s) => `${s.name}
(${s.rollNo})`)}</select></div>
<div class="field"><label>Meal</label><select name="mealType">${mealTypes.map((m) =>
<option>${m}</option>`)}</select></div>
</div><div class="row-actions" style="margin-top:14px"><button class="primary-btn"
type="submit">${icon("utensils")} Record</button></div></form>
</section>
<section class="grid-2">${demandPanel(d.demandToday, "Meal Demand -
Today")}${demandPanel(d.demandTomorrow, "Meal Demand - Tomorrow")}</section>
${menuPanel}
<section class="panel"><div class="panel-title"><h3>Served
Today</h3><span>${served.length}</span></div>
${served.length ? table(`<thead><tr><th>Student</th><th>Meal</th><th>Registered</th><th>Time</th>
</tr></thead><tbody>${served.map((m) => `<tr><td>${esc(studentName(m.studentId))}</td><td>${esc(m.meal
Type)}</td><td>${statusPill(m.registered ? "Registered" : "Walk-
in")}</td><td>${dateTime(m.servedAt)}</td></tr>`)}</tbody>`) : empty("No meals recorded yet
today")}</section>`;
}

function complaintsView() {
const d = state.data;
const next = { Pending: "Assigned", Assigned: "In Progress", "In Progress": "Resolved" };
const student = role() === "student";
return `
<section class="grid-2">
  ${student || role() === "admin" ? `<form class="panel" id="complaint-form"><div class="panel-
title"><h3>${student ? "Submit Complaint" : "Log Complaint"}</h3><span>${d.complaints.filter((c) =>
c.status !== "Resolved").length} open</span></div>
<div class="form-grid">
  ${student ? " " : `<div class="field full"><label>Student</label><select name="studentId"
required>${options(activeStudents(), (s) => s.id, (s) => `${s.name} (${s.rollNo})`)}</select></div>`}
  <div class="field"><label>Category</label><select name="category">${["Room", "Mess", "Water",
"Electricity", "Cleaning", "Internet", "Security", "Other"].map((c) =>
<option>${c}</option>`)}</select></div>
  <div class="field"><label>Priority</label><select name="priority"><option>Low</option><option
selected>Medium</option><option>High</option></select></div>
  <div class="field full"><label>Description</label><textarea name="description" rows="3"
required></textarea></div>
</div><div class="row-actions" style="margin-top:14px"><button class="primary-btn"
type="submit">${icon("plus")} Submit</button></div></form>` : " "}
  <div class="panel"><div class="panel-title"><h3>Complaint Flow</h3><span>${d.complaints.length}
total</span></div><div>${meters(["Pending", "Assigned", "In Progress", "Resolved"].map((s) => [s,
d.complaints.filter((c) => c.status === s).length]), d.complaints.length)}</div>
</section>
<section class="panel"><div class="panel-title"><h3>${student ? "My Complaints" : "All
Complaints"}</h3><span>${d.complaints.length}</span></div>
<div class="list">${d.complaints.map((c) => `<article class="item-card"><div class="item-
head"><strong>${esc(c.category)}</strong><div class="item-meta"><div class="row-actions">${statusPill(c.priority)}</div><div>
<p class="small" style="margin:0">${esc(c.description)}</p><span class="small muted">Raised
${dateTime(c.createdAt)}</div><div class="row-actions">${c.assignedTo ? ` - Assigned to ${esc(c.assignedTo)} : "${c.assignedTo}"` : ""}</div>
<div class="timeline">${c.activity.map((a) => `<span>${dateTime(a.at)} -
${esc(a.message)}</span>`)}</div>
</div>
  ${student && next[c.status] ? `<div class="row-actions"><button class="primary-btn" data-
complaints="${esc(c.id)}" data-next="${next[c.status]}" data-category="${esc(c.category)}">Move to
${next[c.status]}</button></div>` : ""}</article>`)}</div> || empty("No
complaints")</div></section>`;
}

function noticesView() {
const d = state.data;
return `
  ${role() === "admin" ? `<section class="panel"><form id="notice-form"><div class="panel-
title"><h3>Publish Notice</h3><span>${d.notices.length} published</span></div>
<div class="form-grid">
  <div class="field"><label>Title</label><input name="title" required></div>
  <div class="field"><label>Category</label><select name="category"><option>Hostel</option><optio
n>Mess</option><option>Fees</option><option>Maintenance</option><option>Events</option></select></div>
  <div class="field"><label>Priority</label><select name="priority"><option>Low</option><option
selected>Medium</option><option>High</option></select></div>
  <div class="field"><label>Expires on</label><input name="expiresOn" type="date"
value="${addDays(14)}"></div>
  <div class="field full"><label>Description</label><textarea name="description" rows="3"
required></textarea></div>
</div><div class="row-actions" style="margin-top:14px"><button class="primary-btn"
type="submit">${icon("plus")} Publish</button></div></form>` : " "}
  <section class="grid-2">
    <div class="panel"><div class="panel-title"><h3>Notice
Board</h3><span>${d.notices.length}</span></div><div>
      class="list">${d.notices.map(noticeCard).join("")} || empty("No notices")</div></div>
    <div class="panel"><div class="panel-title"><h3>Notifications</h3><span>${d.notifications.filter((n) => !n.read).length}
unread</span></div><div>
      class="list">${d.notifications.map(notificationCard).join("")} || empty("No
notifications")</div></div>` : " "}
  </section>`;
}

function architectureView() {
const node = (title, sub) => `<div class="topology-
node"><strong>${title}</strong><span>${sub}</span></div>`;
return `
<section class="architecture-grid">
  <div class="panel"><div class="panel-title"><h3>Service Topology</h3><span>6 services +
gateway</span></div>
  <div class="topology"><div class="topology-row"><div class="topology-node gateway"><strong>API
Gateway</strong>: 4000</div><div>Auth, RBAC, routing, overview</div></div>
  <div class="topology-row">${node("Student Service :4101", "students, rooms,
allocations")}</div>
  <div class="topology-row">${node("Fee Service :4102", "fees, payments, receipts")}</div>
  <div class="topology-row">${node("Attendance Service :4104", "attendance,
leaves")}</div>
  <div class="topology-row">${node("Mess Service :4105", "menu, registrations, demand")}</div>
  <div class="topology-row">${node("Notification Service :4106", "notices, notifications")}</div>
  <div class="panel"><div class="panel-title"><h3>Design Concepts</h3><span>DBMS and software
engineering</span></div>
  <div class="concept-list"><span>Database per service</span><span>Role-based access
control</span><span>Hashed passwords (scrypt)</span><span>Room capacity rules</span><span>Unique
attendance per day</span><span>Transactional fee payment</span><span>Duplicate meal-registration
guard</span><span>Fault isolation per service</span><span>Validation at UI and
server</span><span>Aggregated dashboard API</span></div>
  <p class="small muted">The relational MySQL schema for the same design is in
<strong>docs/database-schema.sql</strong></p>
</div>
  <div class="panel"><div class="panel-title"><h3>Access Matrix</h3><span>Enforced by the
gateway</span></div>
  <table>
    <thead>
      <tr>
        <th>Area</th>
        <th>Admin</th>
        <th>Mess</th>
    </thead>
  </table>
</section>

```

```

staff</th><th>Student</th></tr></thead><tbody>
  <tr><td>Students and rooms</td><td>Full</td><td>Read students</td><td>Own profile</td></tr>
  <tr><td>Fees and payments</td><td>Full</td><td>None</td><td>Own, read-only</td></tr>
  <tr><td>Attendance and leave</td><td>Full</td><td>None</td><td>Own; request leave</td></tr>
  <tr><td>Mess menu and meals</td><td>Full</td><td>Full</td><td>View menu; own
registrations</td></tr>
  <tr><td>Complaints</td><td>Full</td><td>Review and update</td><td>Submit and view own</td></tr>
  <tr><td>Notices</td><td>Publish</td><td>Read</td><td>Read</td></tr></tbody>`))</section>`;
}

async function login(email, password) {
  const result = await api("/api/auth/login", { method: "POST", auth: false, body: { email, password }
  });
  state.token = result.token;
  state.user = result.user;
  state.view = "dashboard";
  localStorage.setItem("hostel-token", result.token);
  localStorage.setItem("hostel-user", JSON.stringify(result.user));
  await loadData();
  toast(`Signed in as ${result.user.role}`);
}

function logout(showToast = true) {
  state.token = "";
  state.user = null;
  state.data = emptyData();
  localStorage.removeItem("hostel-token");
  localStorage.removeItem("hostel-user");
  render();
  if (showToast) {
    toast("Signed out");
  }
}

async function mutate(endpoint, opts, message) {
  try {
    await api(endpoint, opts);
    toast(message);
    await loadData();
    return true;
  } catch (error) {
    toast(error.message);
    return false;
  }
}

const assignee = { Mess: "Mess Committee", Internet: "IT Support", Security: "Warden Office", Other:
  "Warden Office" };

async function submitForm(event) {
  const form = event.target;
  const id = form.id;

  if (!id) {
    return;
  }

  event.preventDefault();
  const body = formData(form);

  try {
    if (id === "login-form") {
      const errorNode = document.getElementById("auth-error");
      errorNode.textContent = "";
      await login(body.email, body.password).catch((error) => { errorNode.textContent = error.message;
    });
    return;
  }

  if (id === "menu-form") {
    const existing = state.data.menu.find((m) => m.day === body.day && m.mealType === body.mealType);
    const ok = existing
      ? await mutate(`/api/menu/${existing.id}`, { method: "PATCH", body: { items: body.items, isVeg:
body.isVeg } }, "Menu updated")
      : await mutate("/api/menu", { method: "POST", body }, "Menu added");
    if (ok) form.reset();
    return;
  }

  const actions = [
    "student-form": ["/api/students", "Student registered"],
    "room-form": ["/api/rooms", "Room added"],
    "allocation-form": ["/api/allocations", "Room allocated"],
    "fee-form": ["/api/fees", "Fee raised"],
    "leave-form": ["/api/leaves", "Leave request submitted"],
    "register-form": ["/api/registrations", "Meal registered"],
    "served-form": ["/api/meal-attendance", "Meal recorded"],
    "complaint-form": ["/api/complaints", "Complaint submitted"],
    "notice-form": ["/api/notices", "Notice published"]
  ];

  if (!actions[id]) {
    return;
  }

  if (id === "notice-form") body.adminId = state.user.id;
  if (id === "served-form") body.servedBy = state.user.id;

  if (await mutate(actions[id][0], { method: "POST", body }, actions[id][1])) {
    form.reset();
  }
} catch (error) {
  toast(error.message);
}

}

async function handleClick(event) {
  const t = event.target;
  const hit = (selector) => t.closest(selector);
  let el;

  if ((el = hit("[data-nav]"))) { state.view = el.dataset.nav; state.query = ""; render(); return; }

  if ((el = hit("[data-login-preset]"))) {
    const [email, password] = credentials[el.dataset.loginPreset];
    document.querySelector("#email").value = email;
  }
}

```

```

    document.querySelector("#password").value = password;
    await login(email, password).catch((error) => toast(error.message));
    return;
}

if (hit("[data-logout]")) { logout(); return; }
if (hit("[data-refresh]")) { await loadData(); toast("Data refreshed"); return; }

if ((el = hit("[data-pay]"))) {
    const method = document.getElementById(`method-${el.dataset.pay}`).value;
    await mutate("/api/payments/pay", { method: "POST", body: { studentId: el.dataset.student, feeId:
el.dataset.pay, method } }, "Payment recorded");
    return;
}

if ((el = hit("[data-complaint]"))) {
    const status = el.dataset.next;
    await mutate(`/api/complaints/${el.dataset.complaint}/status`, { method: "PATCH", body: { status,
assignedTo: status === "Assigned" ? assignee[el.dataset.category] || "Maintenance Team" : undefined,
note: `Status changed to ${status} by ${state.user.name}` }, "Complaint updated");
    return;
}

if ((el = hit("[data-leave]"))) { await mutate(`/api/leaves/${el.dataset.leave}/status`, { method:
"PATCH", body: { status: el.dataset.value, reviewedBy: state.user.id }, `Leave
${el.dataset.value.toLowerCase()}`); return; }
if ((el = hit("[data-vacate]"))) { await mutate(`/api/allocations/${el.dataset.vacate}/vacate`, {
method: "PATCH", body: { } }, "Room vacated"); return; }
if ((el = hit("[data-maint]"))) { await mutate(`/api/rooms/${encodeURIComponent(el.dataset.maint)}`,
{ method: "PATCH", body: { underMaintenance: el.dataset.value === "true" }, "Room updated");
return; }
if ((el = hit("[data-student-status]"))) { await mutate(`/api/students/${el.dataset.studentStatus}`,
{ method: "PATCH", body: { status: el.dataset.value }, "Student updated"); return; }
if ((el = hit("[data-cancel-reg]"))) { await
mutate(`/api/registrations/${el.dataset.cancelReg}/cancel`, { method: "PATCH", body: { } },
"Registration cancelled"); return; }
if ((el = hit("[data-read]"))) { await mutate(`/api/notifications/${el.dataset.read}/read`, { method:
"PATCH", body: { } }, "Marked as read"); return; }

if (hit("[data-save-attendance]")) {
    const records = [...document.querySelectorAll("[data-att]").map((select) => ({ studentId:
select.dataset.att, status: select.value }));
    state.attDraft = {};
    await mutate("/api/attendance/bulk", { method: "POST", body: { date: state.attDate, records,
markedBy: state.user.id }, "Attendance saved for ${state.attDate}");
}
}

function handleInput(event) {
    if (event.target.id === "student-search") {
        state.query = event.target.value;
        const q = state.query.toLowerCase();
        document.getElementById("students-table").innerHTML = studentsTable(state.data.students.filter((s)
=> !q || `${s.name} ${s.rollNo} ${s.email} ${s.course}`.toLowerCase().includes(q)));
    }
}

function handleChange(event) {
    const el = event.target;

    if (el.id === "att-date" && el.value) {
        state.attDate = el.value;
        state.attDraft = {};
        render();
    } else if (el.dataset && el.dataset.att) {
        state.attDraft[el.dataset.att] = el.value;
    }
}

function render() {
    if (!state.token || !state.user) {
        renderLogin();
        return;
    }

    renderShell();
}

document.addEventListener("submit", submitForm);
document.addEventListener("click", handleClick);
document.addEventListener("input", handleInput);
document.addEventListener("change", handleChange);

if (state.token && state.user) {
    loadData();
} else {
    render();
}
})();

```

## hostel/frontend/index.html

```

<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Smart Hostel and Mess Administration MIS</title>
    <link rel="stylesheet" href="/styles.css">
  </head>
  <body>
    <div id="app-root"></div>
    <div id="toast-region" class="toast-region" aria-live="polite"></div>
    <script src="/app.js"></script>
  </body>
</html>

```

## hostel/frontend/styles.css

```

:root {
  --paper: #f7f7f0;
  --surface: #ffffff;
  --surface-2: #f1f5f1;
  --ink: #20241f;
  --muted: #687067;
  --line: #d7ded4;
  --teal: #0f766e;
  --teal-2: #d9f0ec;
  --coral: #c45a41;
  --amber: #b7791f;
  --green: #26734d;
  --red: #b42318;
  --violet: #595ebf;
  --shadow: 0 18px 45px rgba(31, 41, 34, 0.12);
}

* {
  box-sizing: border-box;
}

html {
  min-height: 100%;
  background: var(--paper);
  color: var(--ink);
  font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  letter-spacing: 0;
}

body {
  min-height: 100vh;
  margin: 0;
}

button,
input,
select,
textarea {
  font: inherit;
}

button {
  cursor: pointer;
}

.login-screen {
  min-height: 100vh;
  display: grid;
  grid-template-columns: minmax(320px, 0.85fr) minmax(360px, 1.15fr);
  background:
    linear-gradient(130deg, rgba(15, 118, 110, 0.10), transparent 38%),
    linear-gradient(320deg, rgba(196, 90, 65, 0.12), transparent 34%),
    var(--paper);
}

.login-visual {
  display: flex;
  flex-direction: column;
  justify-content: space-between;
  padding: 44px;
  border-right: 1px solid var(--line);
  background: #eef3ed;
}

.brand-lockup {
  display: grid;
  gap: 12px;
}

.brand-mark {
  width: 48px;
  height: 48px;
  display: grid;
  place-items: center;
  border-radius: 8px;
  background: var(--teal);
  color: #ffffff;
}

.login-visual h1 {
  max-width: 560px;
  margin: 0;
  font-size: clamp(2.2rem, 4vw, 4.8rem);
  line-height: 0.95;
  font-weight: 820;
}

.login-visual p {
  max-width: 520px;
  margin: 18px 0 0;
  color: var(--muted);
  font-size: 1.03rem;
  line-height: 1.6;
}

.system-map {
  display: grid;
  gap: 10px;
  max-width: 540px;
}

.map-row {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 10px;
}

.map-node {
  min-height: 64px;
  display: grid;
  align-content: center;
  gap: 5px;
  padding: 12px;
  border: 1px solid var(--line);
  border-radius: 8px;
}

```

```

    background: rgba(255, 255, 255, 0.7);
  }

.map-node strong {
  font-size: 0.86rem;
}

.map-node span {
  color: var(--muted);
  font-size: 0.72rem;
}

.login-panel {
  display: grid;
  place-items: center;
  padding: 32px;
}

.auth-box {
  width: min(460px, 100%);
  padding: 28px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--surface);
  box-shadow: var(--shadow);
}

.auth-box h2,
.view-title h2,
.panel-title h3 {
  margin: 0;
}

.auth-box h2 {
  font-size: 1.35rem;
}

.auth-box > p {
  margin: 8px 0 20px;
  color: var(--muted);
}

.preset-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 8px;
  margin-bottom: 18px;
}

.preset-btn,
.icon-btn,
.primary-btn,
.ghost-btn,
.danger-btn,
.nav-btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  min-height: 38px;
  border: 1px solid transparent;
  border-radius: 8px;
  font-weight: 700;
}

.preset-btn {
  padding: 10px;
  background: var(--surface-2);
  color: var(--ink);
}

.preset-btn:hover {
  border-color: var(--teal);
  color: var(--teal);
}

.form-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 12px;
}

.field {
  display: grid;
  gap: 6px;
}

.field.full {
  grid-column: 1 / -1;
}

.field label {
  color: var(--muted);
  font-size: 0.78rem;
  font-weight: 750;
  text-transform: uppercase;
}

.field input,
.field select,
.field textarea {
  width: 100%;
  min-height: 40px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: #ffffff;
  color: var(--ink);
  padding: 9px 11px;
  outline: none;
}

.field textarea {
  min-height: 96px;
  resize: vertical;
}

.field input:focus,

```

```

.field select:focus,
.field textarea:focus {
  border-color: var(--teal);
  box-shadow: 0 0 3px rgba(15, 118, 110, 0.12);
}

.primary-btn {
  padding: 0 14px;
  background: var(--teal);
  color: #ffffff;
}

.ghost-btn {
  padding: 0 12px;
  background: #ffffff;
  border-color: var(--line);
  color: var(--ink);
}

.danger-btn {
  padding: 0 12px;
  background: #fff1ee;
  border-color: #ffd0c7;
  color: var(--red);
}

.icon-btn {
  width: 38px;
  padding: 0;
  background: #ffffff;
  border-color: var(--line);
  color: var(--ink);
}

.primary-btn:hover,
.ghost-btn:hover,
.danger-btn:hover,
.icon-btn:hover {
  transform: translateY(-1px);
}

.auth-error {
  min-height: 20px;
  margin: 10px 0 0;
  color: var(--red);
  font-size: 0.9rem;
}

.app-shell {
  min-height: 100vh;
  display: grid;
  grid-template-columns: 248px minmax(0, 1fr);
}

.sidebar {
  position: sticky;
  top: 0;
  height: 100vh;
  display: flex;
  flex-direction: column;
  padding: 18px;
  border-right: 1px solid var(--line);
  background: #eef3ed;
}

.sidebar-brand {
  display: flex;
  align-items: center;
  gap: 10px;
  min-height: 52px;
  margin-bottom: 22px;
}

.sidebar-brand span {
  display: block;
  color: var(--muted);
  font-size: 0.76rem;
  font-weight: 700;
  text-transform: uppercase;
}

.sidebar-brand strong {
  display: block;
  font-size: 1rem;
}

.nav {
  display: grid;
  gap: 6px;
}

.nav-btn {
  width: 100%;
  justify-content: flex-start;
  min-height: 42px;
  padding: 0 11px;
  background: transparent;
  color: #36413a;
}

.nav-btn.active,
.nav-btn:hover {
  background: #ffffff;
  color: var(--teal);
  box-shadow: 0 8px 20px rgba(28, 50, 43, 0.07);
}

.sidebar-footer {
  margin-top: auto;
  display: grid;
  gap: 10px;
}

.user-chip {
  display: grid;
  gap: 3px;
}

```

```

padding: 10px;
border: 1px solid var(--line);
border-radius: 8px;
background: rgba(255, 255, 255, 0.72);
}

.user-chip strong {
  font-size: 0.88rem;
}

.user-chip span {
  color: var(--muted);
  font-size: 0.76rem;
  text-transform: capitalize;
}

.main-area {
  min-width: 0;
}

.topbar {
  position: sticky;
  top: 0;
  z-index: 10;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  min-height: 72px;
  padding: 16px 24px;
  border-bottom: 1px solid var(--line);
  background: rgba(247, 247, 240, 0.86);
  backdrop-filter: blur(12px);
}

.view-title {
  min-width: 0;
}

.view-title h2 {
  font-size: 1.38rem;
}

.view-title p {
  margin: 4px 0 0;
  color: var(--muted);
}

.topbar-actions {
  display: flex;
  align-items: center;
  gap: 8px;
}

.content {
  display: grid;
  gap: 18px;
  padding: 24px;
}

.kpi-grid {
  display: grid;
  grid-template-columns: repeat(6, minmax(140px, 1fr));
  gap: 12px;
}

.kpi {
  min-height: 112px;
  display: grid;
  align-content: space-between;
  gap: 10px;
  padding: 16px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--surface);
}

.kpi .kpi-icon {
  width: 34px;
  height: 34px;
  display: grid;
  place-items: center;
  border-radius: 8px;
  background: var(--teal-2);
  color: var(--teal);
}

.kpi strong {
  display: block;
  font-size: 1.55rem;
}

.kpi span {
  color: var(--muted);
  font-size: 0.82rem;
  font-weight: 700;
}

.band {
  display: grid;
  gap: 14px;
  padding: 18px;
  border-top: 1px solid var(--line);
  border-bottom: 1px solid var(--line);
  background: rgba(255, 255, 255, 0.52);
}

.grid-2 {
  display: grid;
  grid-template-columns: minmax(0, 1fr) minmax(0, 1fr);
  gap: 16px;
}

.grid-3 {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
}

```

```

    gap: 16px;
  }

  .panel {
    min-width: 0;
    padding: 16px;
    border: 1px solid var(--line);
    border-radius: 8px;
    background: var(--surface);
  }

  .panel-title {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 12px;
    margin-bottom: 14px;
  }

  .panel-title h3 {
    font-size: 1rem;
  }

  .panel-title span {
    color: var(--muted);
    font-size: 0.82rem;
  }

  .data-table {
    width: 100%;
    border-collapse: collapse;
    table-layout: fixed;
  }

  .data-table th,
  .data-table td {
    padding: 11px 9px;
    border-bottom: 1px solid var(--line);
    text-align: left;
    vertical-align: top;
    overflow-wrap: anywhere;
  }

  .data-table th {
    color: var(--muted);
    font-size: 0.72rem;
    text-transform: uppercase;
  }

  .data-table tr:last-child td {
    border-bottom: 0;
  }

  .table-scroll {
    overflow-x: auto;
  }

  .status {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    min-height: 24px;
    max-width: 100%;
    padding: 3px 8px;
    border-radius: 999px;
    background: var(--surface-2);
    color: var(--ink);
    font-size: 0.76rem;
    font-weight: 800;
    text-transform: capitalize;
  }

  .status.good {
    background: #dff3e6;
    color: var(--green);
  }

  .status.warn {
    background: #fffld6;
    color: var(--amber);
  }

  .status.bad {
    background: #ffe2dc;
    color: var(--red);
  }

  .status.info {
    background: #e5e7ff;
    color: var(--violet);
  }

  .status.neutral {
    background: var(--surface-2);
    color: var(--muted);
  }

  .list {
    display: grid;
    gap: 10px;
  }

  .item-card {
    display: grid;
    gap: 10px;
    padding: 14px;
    border: 1px solid var(--line);
    border-radius: 8px;
    background: var(--surface);
  }

  .item-head {
    display: flex;
    justify-content: space-between;
    gap: 12px;
  }

```

```

.item-head strong {
  min-width: 0;
  overflow-wrap: anywhere;
}

.muted {
  color: var(--muted);
}

.small {
  font-size: 0.82rem;
}

.row-actions {
  display: flex;
  align-items: center;
  flex-wrap: wrap;
  gap: 8px;
}

.service-grid {
  display: grid;
  grid-template-columns: repeat(6, minmax(120px, 1fr));
  gap: 10px;
}

.service-tile {
  min-height: 78px;
  display: grid;
  align-content: center;
  gap: 6px;
  padding: 12px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--surface);
}

.service-tile strong {
  font-size: 0.85rem;
}

.service-tile span {
  color: var(--muted);
  font-size: 0.76rem;
}

.meter-list {
  display: grid;
  gap: 12px;
}

.meter-row {
  display: grid;
  grid-template-columns: 110px minmax(0, 1fr) 34px;
  align-items: center;
  gap: 10px;
}

.meter {
  height: 9px;
  overflow: hidden;
  border-radius: 999px;
  background: var(--surface-2);
}

.meter span {
  display: block;
  height: 100%;
  background: var(--teal);
}

.amenity-grid,
.flat-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(210px, 1fr));
  gap: 12px;
}

.amenity-card,
.flat-card {
  min-height: 136px;
  display: grid;
  gap: 9px;
  padding: 14px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--surface);
}

.amenity-card strong,
.flat-card strong {
  font-size: 1rem;
}

.timeline {
  display: grid;
  gap: 8px;
  padding-left: 12px;
  border-left: 2px solid var(--line);
}

.timeline span {
  color: var(--muted);
  font-size: 0.78rem;
}

.architecture-grid {
  display: grid;
  grid-template-columns: 0.9fr 1.1fr;
  gap: 16px;
}

.topology {
  display: grid;
  gap: 10px;
}

```

```

.topology-row {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 10px;
}

.topology-node {
  min-height: 76px;
  display: grid;
  align-content: center;
  gap: 4px;
  padding: 12px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--surface);
}

.topology-node.gateway {
  background: var(--teal);
  color: #ffffff;
}

.topology-node span {
  color: var(--muted);
  font-size: 0.76rem;
}

.topology-node.gateway span {
  color: rgba(255, 255, 255, 0.78);
}

.concept-list {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 8px;
}

.concept-list span {
  padding: 9px 10px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--surface);
  color: #354039;
  font-size: 0.88rem;
}

.empty-state {
  padding: 18px;
  border: 1px dashed var(--line);
  border-radius: 8px;
  color: var(--muted);
  text-align: center;
}

.toast-region {
  position: fixed;
  right: 18px;
  bottom: 18px;
  z-index: 40;
  display: grid;
  gap: 8px;
  width: min(360px, calc(100vw - 36px));
}

.toast {
  padding: 12px 14px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--surface);
  box-shadow: var(--shadow);
  color: var(--ink);
}

.mobile-nav {
  display: none;
}

svg {
  flex: 0 0 auto;
}

@media (max-width: 1180px) {
  .kpi-grid,
  .service-grid {
    grid-template-columns: repeat(3, minmax(140px, 1fr));
  }

  .grid-3,
  .architecture-grid {
    grid-template-columns: 1fr;
  }
}

@media (max-width: 880px) {
  .login-screen,
  .app-shell,
  .grid-2 {
    grid-template-columns: 1fr;
  }

  .login-visual {
    min-height: 420px;
    border-right: 0;
    border-bottom: 1px solid var(--line);
  }

  .sidebar {
    position: static;
    height: auto;
    border-right: 0;
    border-bottom: 1px solid var(--line);
  }

  .nav {
    grid-template-columns: repeat(4, minmax(0, 1fr));
  }
}

```

```

.nav-btn {
  justify-content: center;
}

.nav-btn span {
  display: none;
}

.sidebar-footer {
  display: none;
}

.topbar {
  position: static;
  align-items: flex-start;
}

.mobile-nav {
  display: block;
}
}

@media (max-width: 620px) {
  .login-visual,
  .login-panel,
  .content,
  .topbar {
    padding: 18px;
  }

  .auth-box {
    padding: 18px;
  }

  .preset-grid,
  .form-grid,
  .kpi-grid,
  .service-grid,
  .topology-row,
  .concept-list,
  .nav {
    grid-template-columns: 1fr;
  }

  .map-row {
    grid-template-columns: 1fr;
  }

  .meter-row {
    grid-template-columns: 86px minmax(0, 1fr) 30px;
  }

  .topbar {
    flex-direction: column;
  }

  .topbar-actions {
    width: 100%;
    justify-content: space-between;
  }
}
}

```

## hostel/scripts/generate-seed.js

```

// Generates seed.json (and db.json) for every service with dates relative to today,
// so attendance, meals and dues always look current in a demo.
const path = require("path");
const { hashPassword, writeJson } = require("../shared/service-kit");

const servicesDir = path.join(__dirname, "..", "services");
const now = new Date();
const dayOffset = (n) => new Date(now.getTime() + n * 86400000).toISOString().slice(0, 10);
const stamp = (n, time = "09:00") => `${dayOffset(n)}T${time}:00.000Z`;
const weekday = (n) => ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"][new
  Date(`${dayOffset(n)}T00:00:00Z`).getUTCDay()];
const month = dayOffset(0).slice(0, 7);

const students = [
  ["Aarav Sharma", "B.Tech CSE", 2, "Rajesh Sharma"],
  ["Ishita Verma", "B.Tech ECE", 1, "Sunil Verma"],
  ["Rohan Reddy", "B.Tech CSE", 2, "Venkat Reddy"],
  ["Sneha Iyer", "B.Tech IT", 3, "Lakshmi Iyer"],
  ["Karthik Nair", "B.Tech ME", 2, "Suresh Nair"],
  ["Priya Singh", "B.Tech CSE", 1, "Amit Singh"],
  ["Mohammed Faiz", "B.Tech EEE", 3, "Imran Faiz"],
  ["Ananya Das", "B.Tech CSE", 2, "Bikash Das"],
  ["Vikas Patel", "B.Tech CE", 4, "Harish Patel"],
  ["Meera Joshi", "B.Tech IT", 1, "Nitin Joshi"],
  ["Sai Teja", "B.Tech AI&DS", 2, "Ramana Teja"],
  ["Divya Menon", "B.Tech CSE", 1, "Gopal Menon"]
].map(([name, course, year, guardianName], index) => {
  const n = index + 1;
  return {
    id: `stu-${2600 + n}`,
    rollNo: `25200301${String(n).padStart(2, "0")}`,
    name,
    phone: `98765432${String(10 + n).padStart(2, "0")}`,
    email: `${name.split(" ")[0].toLowerCase()}@student.test`,
    password: hashPassword("student123"),
    role: "student",
    course,
    year,
    guardianName,
    guardianPhone: `980000000${String(10 + n).padStart(2, "0")}`,
    status: n === 12 ? "pending" : "active",
    joinedDate: dayOffset(-120 + n)
  };
});

const staff = [
  ["adm-0001", "Dr. Suresh Kumar", "admin@hostel.test", "admin123", "admin", "9000000001"],
  ["msx-0001", "Lakshmi Devi", "mess@hostel.test", "mess123", "mess", "9000000002"]
]

```

```

.map(({id, name, email, password, role, phone}) => ({
  id, rollNo: `STAFF-${id.slice(-4)}`, name, phone, email, password: hashPassword(password), role,
  course: role === "admin" ? "Hostel Administration" : "Mess Administration", year: 0,
  guardianName: "", guardianPhone: "", status: "active", joinedDate: dayOffset(-400)
})));

const rooms = [
  ["A-101", "A", 1, 2], ["A-102", "A", 1, 2], ["A-103", "A", 1, 3], ["A-104", "A", 1, 3],
  ["A-201", "A", 2, 2], ["A-202", "A", 2, 2], ["B-101", "B", 1, 3], ["B-102", "B", 1, 3],
  ["B-201", "B", 2, 2], ["B-202", "B", 2, 4]
].map(({roomNo, block, floor, capacity}) => ({
  roomNo, block, floor, capacity,
  roomType: capacity === 2 ? "Double" : capacity === 3 ? "Triple" : "Dormitory",
  underMaintenance: roomNo === "B-201"
})));

const roomPlan = ["A-101", "A-101", "A-102", "A-102", "A-103", "A-103", "A-104", "B-101", "B-101",
  "B-102", "B-102"];
const active = students.filter((s) => s.status === "active");
const allocations = active.slice(0, 10).map((student, i) => ({
  id: `alc-${1001 + i}`, studentId: student.id, roomNo: roomPlan[i],
  allocatedOn: dayOffset(-110), vacatedOn: null, status: "Active"
})));
const allocated = allocations.map((a) => a.studentId);

// Fees and payments
const fees = [];
const payments = [];
let receipt = 1;
active.forEach((student, i) => {
  const rent = { id: `fee-r${i + 1}`, studentId: student.id, feeType: "Hostel Rent", amount: 42000, term:
    "Sem 1 2026-27", dueDate: dayOffset(-40), status: "pending", paymentDate: null, receiptNo: null,
    notes: "Semester hostel rent" };
  const mess = { id: `fee-m${i + 1}`, studentId: student.id, feeType: "Mess Charges", amount: 3600, term:
    month, dueDate: dayOffset(i % 3 === 0 ? -3 : 12), status: "pending", paymentDate: null, receiptNo:
    null, notes: "Monthly mess charges" };
  for (const fee of [rent, mess]) {
    const paid = fee === rent ? i % 4 !== 3 : i % 2 === 1;
    if (paid) {
      const receiptNo = `RCP-${month.replace("-", "")}-${String(receipt++).padStart(4, "0")}`;
      fee.status = "paid";
      fee.paymentDate = stamp(-(20 + i));
      fee.receiptNo = receiptNo;
      payments.push({ id: `pay-${1000 + payments.length + 1}`, studentId: student.id, studentName:
        student.name, feeId: fee.id, feeType: fee.feeType, term: fee.term, amount: fee.amount, paymentDate:
        fee.paymentDate, method: ["UPI", "Cash", "Bank Transfer", "Card"][i % 4], recordedBy: "adm-0001",
        receiptNo });
      fees.push(fee);
    }
  }
});

// Attendance for the last 7 days (today is partly marked so the desk can finish it)
const attendance = [];
allocated.forEach((studentId, i) => {
  for (let d = -7; d <= 0; d += 1) {
    if (d === 0 && i > 5) continue;
    const roll = (i * 7 + Math.abs(d) * 3) % 11;
    const status = roll === 0 ? "Absent" : roll === 1 ? "Late" : "Present";
    attendance.push({ id: `att-${studentId}-${d}`, studentId, date: dayOffset(d), status, remark: "",
      markedBy: "adm-0001", updatedAt: stamp(d, "21:00") });
  }
});

const leaves = [
  { id: "lve-1001", studentId: "stu-2603", fromDate: dayOffset(2), toDate: dayOffset(4), reason: "Family
    function", destination: "Warangal", status: "Pending", reviewedBy: null, createdAt: stamp(-1) },
  { id: "lve-1002", studentId: "stu-2605", fromDate: dayOffset(-3), toDate: dayOffset(-1), reason:
    "Medical checkup", destination: "Kochi", status: "Approved", reviewedBy: "adm-0001", createdAt:
    stamp(-6) },
  { id: "lve-1003", studentId: "stu-2607", fromDate: dayOffset(5), toDate: dayOffset(6), reason:
    "Hackathon travel", destination: "Bengaluru", status: "Pending", reviewedBy: null, createdAt:
    stamp(0, "07:30") }
];

// Weekly mess menu
const menuPlan = {
  Breakfast: ["Idli, sambar, coconut chutney", "Poori and aloo masala", "Upma and banana", "Dosa with
    tomato chutney", "Pongal and vada", "Paratha with curd", "Bread, omelette or jam, tea"],
  Lunch: ["Rice, dal, cabbage fry, curd", "Veg pulao, raita, papad", "Rice, sambar, beans poriyal",
    "Chapati, paneer curry, rice", "Rice, rasam, potato fry", "Veg biryani, raita", "Rice, dal tadka,
    mixed veg curry"],
  Snacks: ["Tea and biscuits", "Samosa and tea", "Sprouts chaat", "Bajji and tea", "Fruit bowl and tea",
    "Pav bhaji bites", "Corn chaat and tea"],
  Dinner: ["Chapati, dal, aloo gobi", "Rice, sambar, egg curry or paneer", "Fried rice and manchurian",
    "Chapati, chole", "Rice, dal fry, cauliflower fry", "Chapati, mixed veg, curd rice", "Khichdi and
    papad"]
};
const dayNames = ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"];
const menu = [];
dayNames.forEach((day, d) => {
  Object.entries(menuPlan).forEach(([mealType, list]) => {
    menu.push({ id: `mnu-${day.slice(0, 3).toLowerCase()}-${mealType.slice(0, 3).toLowerCase()}`, day,
      mealType, items: list[d], isVeg: !/egg|omelette/.test(list[d]), updatedAt: stamp(-30) });
  });
});

const registrations = [];
const mealAttendance = [];
allocated.forEach((studentId, i) => {
  [0, 1].forEach((offset) => {
    ["Lunch", "Dinner"].forEach((mealType) => {
      if ((i + offset + mealType.length) % 5 !== 0) {
        registrations.push({ id: `reg-${studentId}-${offset}-${mealType}`, studentId, date:
          dayOffset(offset), mealType, status: "Registered", createdAt: stamp(-1, "18:00") });
      }
    });
  });
  registrations.push({ id: `reg-${studentId}-0-Breakfast`, studentId, date: dayOffset(0), mealType:
    "Breakfast", status: "Registered", createdAt: stamp(-1, "18:00") });
  if (i % 4 !== 3) {
    mealAttendance.push({ id: `mel-${studentId}-b`, studentId, date: dayOffset(0), mealType: "Breakfast",
      registered: true, servedBy: "msx-0001", servedAt: stamp(0, "07:45") });
  }
});
const complaints = [

```

```

["stu-2601", "Water", "No hot water in the block A geyser since yesterday.", "High", "In Progress",
  "Maintenance Team"],
["stu-2604", "Mess", "Dinner was served cold on Tuesday; please review timing.", "Medium", "Assigned",
  "Mess Committee"],
["stu-2606", "Room", "Ceiling fan makes a loud noise at night.", "Low", "Pending", null],
["stu-2608", "Internet", "Wi-Fi drops repeatedly on the first floor.", "Medium", "Resolved", "IT
  Support"],
["stu-2610", "Cleaning", "Washroom on B-block first floor needs daily cleaning.", "Medium", "Pending",
  null]
].map(([studentId, category, description, priority, status, assignedTo], i) => ({
  id: `cmp-${1001 + i}`, studentId, category, description, priority, status, assignedTo,
  createdAt: stamp(-(i + 1), "10:00"), resolvedDate: status === "Resolved" ? stamp(0, "08:00") : null,
  activity: [{ at: stamp(-(i + 1), "10:00"), message: "Complaint submitted" }].concat(assignedTo ? [{ at:
    stamp(-(i + 1), "12:00"), message: `Assigned to ${assignedTo}` } ] : []
})));

const notices = [
  ["Hostel fee due date", "Semester hostel rent dues must be cleared at the hostel office within the due
    date to avoid fines.", "Fees", "High", 7],
  ["Special dinner on Sunday", "A festival dinner will be served on Sunday. Register for the meal before
    Saturday evening.", "Mess", "Medium", 6],
  ["Water tank cleaning", "Water supply will be interrupted from 10:00 to 12:00 tomorrow for overhead
    tank cleaning.", "Maintenance", "High", 2],
  ["Night attendance timings", "Night attendance is taken at 21:00. Inform the warden in advance if you
    will be late.", "Hostel", "Medium", 30]
].map(([title, description, category, priority, days], i) => ({
  id: `not-${1001 + i}`, adminId: "adm-0001", title, description, category, priority, audience: "All
    Students",
  createdAt: stamp(-(i + 1), "12:00"), expiresOn: dayOffset(days)
})));

const notifications = [
  ["stu-2601", "Complaint update", "Your water complaint is now In Progress."],
  ["stu-2603", "Leave request received", "Your leave request is pending warden approval."],
  ["stu-2605", "Leave approved", "Your medical leave request was approved."],
  ["stu-2604", "Fee reminder", "Mess charges for this month are due soon."]
].map(([studentId, title, message], i) => ({ id: `ntf-${1001 + i}`, studentId, title, message, channel:
  "In App", read: false, createdAt: stamp(-i, "11:00") }));

const seeds = {
  "student-service": { users: [...staff, ...students], rooms, allocations },
  "fee-service": { fees, payments },
  "complaint-service": { complaints },
  "attendance-service": { attendance, leaves },
  "mess-service": { menu, registrations, mealAttendance },
  "notification-service": { notices, notifications }
};

for (const [service, data] of Object.entries(seeds)) {
  writeJson(path.join(servicesDir, service, "seed.json"), data);
  writeJson(path.join(servicesDir, service, "db.json"), data);
  console.log(`Seeded ${service}`);
}

```

## hostel/scripts/reset-data.js

```

// Restores every service database to fresh demo data (dates are relative to today).
require("../generate-seed");

```

## hostel/scripts/smoke-test.js

```

const { spawn } = require("child_process");
const http = require("http");
const path = require("path");
const { requestJson } = require("../shared/service-kit");

const rootDir = path.join(__dirname, "..");
const gateway = "http://localhost:4000";
let child;
let passed = 0;

function check(condition, label) {
  if (!condition) {
    throw new Error(`FAILED: ${label}`);
  }
  passed += 1;
  console.log(` ok ${label}`);
}

async function waitForGateway() {
  const deadline = Date.now() + 12000;

  while (Date.now() < deadline) {
    try {
      const response = await requestJson(`${gateway}/api/service-status`, { timeoutMs: 1000 });
      if (response.ok && response.json.services.every((service) => service.ok)) {
        return;
      }
    } catch (error) {
      // gateway not up yet
    }
    await new Promise((resolve) => setTimeout(resolve, 300));
  }

  throw new Error("Gateway or services did not become ready");
}

async function login(email, password) {
  const response = await requestJson(`${gateway}/api/auth/login`, { method: "POST", body: { email,
    password } });
  return response;
}

const call = (token, url, method = "GET", body) =>
  requestJson(`${gateway}${url}`, { method, body, headers: { Authorization: `Bearer ${token}` } });

async function run() {
  child = spawn(process.execPath, [path.join(rootDir, "scripts", "start-all.js")], { cwd: rootDir, stdio:

```

```

["ignore", "pipe", "pipe"] });
child.stdout.on("data", () => {});
child.stderr.on("data", (chunk) => process.stderr.write(chunk));

await waitForGateway();
console.log("Authentication");
check((await login("admin@hostel.test", "wrong")).status === 401, "wrong password is rejected");
const admin = await login("admin@hostel.test", "admin123");
const student = await login("aarav@student.test", "student123");
const mess = await login("mess@hostel.test", "mess123");
check(admin.ok && student.ok && mess.ok, "admin, student and mess staff can sign in");
const A = admin.json.token;
const S = student.json.token;
const M = mess.json.token;
check((await requestJson(`${gateway}/api/students`)).status === 401, "requests without a token get 401");

console.log("Student and room management");
check((await call(A, "/api/overview")).json.summary.students >= 11, "overview aggregates all services");
const dup = await call(A, "/api/students", "POST", { name: "X", rollNo: "2520030101", phone: "9999999999", email: "x@student.test", course: "CSE" });
check(dup.status === 409, "duplicate roll number is rejected");
const badPhone = await call(A, "/api/students", "POST", { name: "X", rollNo: "999", phone: "12", email: "y@student.test", course: "CSE" });
check(badPhone.status === 400, "invalid phone number is rejected");
const created = await call(A, "/api/students", "POST", { name: "Test Student", rollNo: "2520039999", phone: "9123456780", email: "test@student.test", course: "B.Tech CSE", year: 1 });
check(created.status === 201 && created.json.password === undefined, "student registered without leaking the password");
const full = await call(A, "/api/allocations", "POST", { studentId: created.json.id, roomNo: "A-101" });
check(full.status === 409, "allocation to a full room is rejected");
const maint = await call(A, "/api/allocations", "POST", { studentId: created.json.id, roomNo: "B-201" });
check(maint.status === 409, "allocation to a room under maintenance is rejected");
const ok = await call(A, "/api/allocations", "POST", { studentId: created.json.id, roomNo: "B-202" });
check(ok.status === 201, "valid room allocation succeeds");
check((await call(A, "/api/allocations", "POST", { studentId: created.json.id, roomNo: "B-202" })).status === 409, "double allocation is rejected");
check((await call(A, `/api/allocations/${ok.json.id}/vacate`, "PATCH", {})).ok, "room can be vacated");

console.log("Fees");
const fee = await call(A, "/api/fees", "POST", { studentId: created.json.id, feeType: "Hostel Rent", amount: 1000, term: "Smoke", dueDate: "2099-01-01" });
check(fee.status === 201, "fee record created");
check((await call(A, "/api/fees", "POST", { studentId: created.json.id, feeType: "Hostel Rent", amount: -5, term: "Smoke2", dueDate: "2099-01-01" })).status === 400, "negative fee amount is rejected");
const pay = await call(A, "/api/payments/pay", "POST", { studentId: created.json.id, feeId: fee.json.id, method: "UPI" });
check(pay.status === 201 && pay.json.receiptNo, "payment recorded with receipt");
check((await call(A, "/api/payments/pay", "POST", { studentId: created.json.id, feeId: fee.json.id, method: "UPI" })).status === 409, "paying twice is rejected");

console.log("Attendance");
const today = new Date().toISOString().slice(0, 10);
check((await call(A, "/api/attendance", "POST", { studentId: "stu-2601", date: "2099-01-01", status: "Present" })).status === 400, "future attendance is rejected");
check((await call(A, "/api/attendance/bulk", "POST", { date: today, records: [{ studentId: "stu-2609", status: "Present" }, { studentId: "stu-2610", status: "Absent" }] })).status === 201, "bulk attendance saved");
const leave = await call(S, "/api/leaves", "POST", { fromDate: "2099-02-01", toDate: "2099-02-03", reason: "Smoke test", destination: "Home" });
check(leave.status === 201 && leave.json.studentId === "stu-2601", "student can request leave (id stamped by gateway)");
check((await call(S, "/api/leaves", "POST", { fromDate: "2099-02-02", toDate: "2099-02-04", reason: "Overlap" })).status === 409, "overlapping leave is rejected");
check((await call(A, `/api/leaves/${leave.json.id}/status`, "PATCH", { status: "Approved" })).ok, "admin approves leave");

console.log("Mess");
const day = ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"][new Date(Date.now() + 86400000).getUTCDay()];
const menu = await call(S, "/api/menu");
check(menu.ok && menu.json.length === 28, "student can view the weekly menu");
const tomorrow = new Date(Date.now() + 2 * 86400000).toISOString().slice(0, 10);
const reg = await call(S, "/api/registrations", "POST", { date: tomorrow, mealType: "Breakfast" });
check(reg.status === 201, "student registers for a meal");
check((await call(S, "/api/registrations", "POST", { date: tomorrow, mealType: "Breakfast" })).status === 409, "duplicate meal registration is rejected");
check((await call(M, `/api/demand?date=${tomorrow}`)).json.meals.Breakfast.registered >= 1, "mess staff can see meal demand");
check((await call(M, "/api/meal-attendance", "POST", { studentId: "stu-2609", mealType: "Dinner" })).status === 201, "mess staff records a served meal");
console.log(` (menu check for ${day} ok)`);

console.log("Complaints and notices");
const complaint = await call(S, "/api/complaints", "POST", { category: "Room", description: "Smoke test complaint", priority: "Low" });
check(complaint.status === 201 && complaint.json.studentId === "stu-2601", "student raises complaint");
check((await call(A, `/api/complaints/${complaint.json.id}/status`, "PATCH", { status: "Resolved" })).ok, "admin resolves complaint");
check((await call(A, "/api/notices", "POST", { adminId: "adm-0001", title: "Smoke", description: "Notice" })).status === 201, "admin publishes notice");

console.log("Role-based access control");
const ownList = await call(S, "/api/students");
check(ownList.ok && ownList.json.length === 1 && ownList.json[0].id === "stu-2601", "student list is limited to the student's own record");
check((await call(S, "/api/students/stu-2602")).status === 403, "student cannot view another profile");
check((await call(S, "/api/students/stu-2601")).ok, "student can view own profile");
check((await call(S, "/api/overview")).status === 403, "student cannot open the admin overview");
check((await call(S, "/api/payments/pay", "POST", { feeId: "x", method: "Cash" })).status === 403, "student cannot record payments");
check((await call(S, "/api/fees")).json.every((item) => item.studentId === "stu-2601"), "student sees only own fees");
check((await call(S, "/api/attendance")).json.every((item) => item.studentId === "stu-2601"), "student sees only own attendance");
check((await call(M, "/api/fees")).status === 403, "mess staff cannot view fees");
check((await call(M, "/api/menu", "POST", { day: "Monday", mealType: "Lunch", items: "x" })).status === 409, "duplicate menu slot is rejected");
const other = (await call(A, "/api/registrations?studentId=stu-2609")).json.find((r) => r.status === "Registered" && r.date >= today);
check(other && (await call(S, `/api/registrations/${other.id}/cancel`, "PATCH", {})).status === 403, "student cannot cancel another student's meal");
const frontend = await new Promise((resolve, reject) => {

```

```

    http.get(gateway, (response) => {
      let body = "";
      response.on("data", (chunk) => { body += chunk; });
      response.on("end", () => resolve({ status: response.statusCode, body }));
    }).on("error", reject);
  });
  check(frontend.status === 200 && frontend.body.includes("app-root"), "frontend is served by the
  gateway");
  console.log(`\nSmoke test passed (${passed} checks).`);
}

run()
  .catch((error) => {
    console.error(error.message);
    process.exitCode = 1;
  })
  .finally(() => {
    if (child && !child.killed) {
      child.kill("SIGTERM");
    }
  });
});

```

## hostel/scripts/start-all.js

```

const { spawn } = require("child_process");
const path = require("path");

const fs = require("fs");

const rootDir = path.join(__dirname, "..");

// First run (no databases yet): create demo data with dates relative to today.
if (!fs.existsSync(path.join(rootDir, "services", "student-service", "db.json"))) {
  require("./generate-seed");
}

const services = [
  ["student-service", "services/student-service/server.js"],
  ["fee-service", "services/fee-service/server.js"],
  ["complaint-service", "services/complaint-service/server.js"],
  ["attendance-service", "services/attendance-service/server.js"],
  ["mess-service", "services/mess-service/server.js"],
  ["notification-service", "services/notification-service/server.js"],
  ["api-gateway", "gateway/server.js"]
];

const env = {
  ...process.env,
  GATEWAY_PORT: process.env.GATEWAY_PORT || "4000",
  STUDENT_SERVICE_PORT: process.env.STUDENT_SERVICE_PORT || "4101",
  FEE_SERVICE_PORT: process.env.FEE_SERVICE_PORT || "4102",
  COMPLAINT_SERVICE_PORT: process.env.COMPLAINT_SERVICE_PORT || "4103",
  ATTENDANCE_SERVICE_PORT: process.env.ATTENDANCE_SERVICE_PORT || "4104",
  MESS_SERVICE_PORT: process.env.MESS_SERVICE_PORT || "4105",
  NOTIFICATION_SERVICE_PORT: process.env.NOTIFICATION_SERVICE_PORT || "4106",
  STUDENT_SERVICE_URL: process.env.STUDENT_SERVICE_URL || "http://localhost:4101",
  FEE_SERVICE_URL: process.env.FEE_SERVICE_URL || "http://localhost:4102",
  COMPLAINT_SERVICE_URL: process.env.COMPLAINT_SERVICE_URL || "http://localhost:4103",
  ATTENDANCE_SERVICE_URL: process.env.ATTENDANCE_SERVICE_URL || "http://localhost:4104",
  MESS_SERVICE_URL: process.env.MESS_SERVICE_URL || "http://localhost:4105",
  NOTIFICATION_SERVICE_URL: process.env.NOTIFICATION_SERVICE_URL || "http://localhost:4106",
  JWT_SECRET: process.env.JWT_SECRET || "development-demo-secret"
};

const children = services.map(([name, script]) => {
  const child = spawn(process.execPath, [path.join(rootDir, script)], {
    cwd: rootDir,
    env,
    stdio: ["ignore", "pipe", "pipe"]
  });

  child.stdout.on("data", (chunk) => process.stdout.write(`${name} ${chunk}`));
  child.stderr.on("data", (chunk) => process.stderr.write(`${name} ${chunk}`));
  child.on("exit", (code) => {
    if (code !== 0 && code !== null) {
      console.error(`${name} exited with code ${code}`);
    }
  });

  return child;
});

function shutdown() {
  for (const child of children) {
    if (!child.killed) {
      child.kill("SIGTERM");
    }
  }
}

process.on("SIGINT", () => { shutdown(); process.exit(0); });
process.on("SIGTERM", () => { shutdown(); process.exit(0); });

console.log("Smart Hostel and Mess Administration MIS starting...");
console.log(`Open http://localhost:${env.GATEWAY_PORT}`);

```

## hostel/shared/service-kit.js

```

const crypto = require("crypto");
const fs = require("fs");
const http = require("http");
const https = require("https");
const path = require("path");

function ensureDir(dirPath) {
  fs.mkdirSync(dirPath, { recursive: true });
}

function clone(value) {

```

```

    return JSON.parse(JSON.stringify(value));
  }

  function replaceObject(target, source) {
    for (const key of Object.keys(target)) {
      delete target[key];
    }

    Object.assign(target, clone(source));
  }

  function readJson(filePath, fallback) {
    ensureDir(path.dirname(filePath));

    if (!fs.existsSync(filePath)) {
      writeJson(filePath, fallback);
      return clone(fallback);
    }

    const raw = fs.readFileSync(filePath, "utf8").trim();
    return raw ? JSON.parse(raw) : clone(fallback);
  }

  function writeJson(filePath, payload) {
    ensureDir(path.dirname(filePath));
    const tempPath = `${filePath}.tmp`;
    fs.writeFileSync(tempPath, JSON.stringify(payload, null, 2));
    fs.renameSync(tempPath, filePath);
  }

  function createId(prefix) {
    const stamp = Date.now().toString(36);
    const random = crypto.randomBytes(3).toString("hex");
    return `${prefix}-${stamp}-${random}`;
  }

  function nowIso() {
    return new Date().toISOString();
  }

  function sendJson(res, statusCode, payload, extraHeaders = {}) {
    res.writeHead(statusCode, {
      "Access-Control-Allow-Origin": "*",
      "Access-Control-Allow-Methods": "GET,POST,PATCH,PUT,DELETE,OPTIONS",
      "Access-Control-Allow-Headers": "Content-Type, Authorization",
      "Content-Type": "application/json; charset=utf-8",
      ...extraHeaders
    });
    res.end(JSON.stringify(payload));
  }

  function sendError(res, statusCode, message, details) {
    sendJson(res, statusCode, {
      error: message,
      details: details || null
    });
  }

  function readBody(req) {
    return new Promise((resolve, reject) => {
      let body = "";

      req.on("data", (chunk) => {
        body += chunk;
        if (body.length > 1_000_000) {
          req.destroy();
          reject(new Error("Request body too large"));
        }
      });

      req.on("end", () => {
        if (!body.trim()) {
          resolve({});
          return;
        }

        try {
          resolve(JSON.parse(body));
        } catch (error) {
          reject(new Error("Invalid JSON body"));
        }
      });

      req.on("error", reject);
    });
  }

  function normalizePath(pathname) {
    if (pathname.length > 1 && pathname.endsWith("/")) {
      return pathname.slice(0, -1);
    }

    return pathname;
  }

  function matchPath(pattern, pathname) {
    const patternParts = normalizePath(pattern).split("/").filter(Boolean);
    const pathParts = normalizePath(pathname).split("/").filter(Boolean);

    if (patternParts.length !== pathParts.length) {
      return null;
    }

    const params = {};

    for (let index = 0; index < patternParts.length; index += 1) {
      const patternPart = patternParts[index];
      const pathPart = pathParts[index];

      if (patternPart.startsWith(":") {
        params[patternPart.slice(1)] = decodeURIComponent(pathPart);
        continue;
      }

      if (patternPart !== pathPart) {
        return null;
      }
    }
  }

```

```

    }
  }

  return params;
}

function byId(collection, id) {
  return collection.find((item) => item.id === id || item.flatNo === id);
}

function createService({ name, port, dataFile, seedFile, routes }) {
  const seed = readJson(seedFile, {});
  const db = readJson(dataFile, seed);

  function save() {
    writeJson(dataFile, db);
  }

  const normalizedRoutes = [
    {
      method: "GET",
      path: "/health",
      handler: ({ send }) => send(200, {
        service: name,
        status: "ok",
        checkedAt: nowIso()
      })
    },
    ...routes
  ];

  const server = http.createServer(async (req, res) => {
    if (req.method === "OPTIONS") {
      sendJson(res, 204, {});
      return;
    }

    const parsedUrl = new URL(req.url, `http://${req.headers.host}`);
    const pathname = normalizePath(parsedUrl.pathname);
    const route = normalizedRoutes.find((candidate) => {
      return candidate.method === req.method && matchPath(candidate.path, pathname);
    });

    if (!route) {
      sendError(res, 404, `${name} route not found`);
      return;
    }

    const params = matchPath(route.path, pathname);
    const query = Object.fromEntries(parsedUrl.searchParams.entries());

    try {
      const body = ["POST", "PUT", "PATCH", "DELETE"].includes(req.method)
        ? await readBody(req)
        : {};

      await route.handler({
        req,
        res,
        params,
        query,
        body,
        db,
        save,
        send: (status, payload, headers) => sendJson(res, status, payload, headers),
        error: (status, message, details) => sendError(res, status, message, details)
      });
    } catch (error) {
      sendError(res, 500, error.message || "Unexpected service error");
    }
  });

  server.listen(port, () => {
    console.log(`${name} listening on http://localhost:${port}`);
  });

  return server;
}

function requestJson(url, options = {}) {
  const parsed = new URL(url);
  const transport = parsed.protocol === "https:" ? https : http;
  const method = options.method || "GET";
  const body = options.body === undefined ? undefined : JSON.stringify(options.body);
  const headers = {
    Accept: "application/json",
    ...(body ? { "Content-Type": "application/json", "Content-Length": Buffer.byteLength(body) } : {}),
    ...(options.headers || {})
  };

  return new Promise((resolve, reject) => {
    const request = transport.request({
      method,
      hostname: parsed.hostname,
      port: parsed.port,
      path: `${parsed.pathname}${parsed.search}`,
      headers,
      timeout: options.timeoutMs || 5000
    }, (response) => {
      let responseBody = "";

      response.on("data", (chunk) => {
        responseBody += chunk;
      });

      response.on("end", () => {
        let json = null;

        try {
          json = responseBody ? JSON.parse(responseBody) : null;
        } catch (error) {
          json = responseBody;
        }

        resolve({
          status: response.statusCode || 0,

```

```

        ok: Boolean(response.statusCode && response.statusCode >= 200 && response.statusCode < 300),
        headers: response.headers,
        json
    });
    });
});

request.on("timeout", () => {
    request.destroy(new Error(`Request timed out for ${url}`));
});

request.on("error", reject);

if (body) {
    request.write(body);
}

request.end();
});
}

function requireFields(body, fields) {
    const missing = fields.filter((field) => {
        return body[field] === undefined || body[field] === null || body[field] === "";
    });

    if (missing.length) {
        const error = new Error(`Missing required fields: ${missing.join(", ")}`);
        error.statusCode = 400;
        throw error;
    }
}

function hashPassword(password, salt = crypto.randomBytes(8).toString("hex")) {
    const hash = crypto.scryptSync(String(password), salt, 32).toString("hex");
    return `${salt}:${hash}`;
}

function verifyPassword(password, stored) {
    if (!stored || !stored.includes(":")) {
        return false;
    }

    const [salt, hash] = stored.split(":");
    const candidate = crypto.scryptSync(String(password), salt, 32);
    const expected = Buffer.from(hash, "hex");
    return expected.length === candidate.length && crypto.timingSafeEqual(expected, candidate);
}

function today() {
    return new Date().toISOString().slice(0, 10);
}

module.exports = {
    hashPassword,
    verifyPassword,
    today,
    byId,
    clone,
    createId,
    createService,
    nowIso,
    readBody,
    readJson,
    replaceObject,
    requestJson,
    requireFields,
    sendError,
    sendJson,
    writeJson
};

```